学校代码	10699
分 类 号	O242
密　　级	公开
学　　号	2022264024

题目　图文融合的虚假信息检测方法研究与实现

作者　朱沂原

专 业 领 域　软件工程

指 导 教 师　李勇军　副教授

培 养 单 位　软件学院

申 请 日 期　2024 年 12 月

西 北 工 业 大 学

硕 士 学 位 论 文

题目：图文融合的虚假信息检测方法研究与实现

专业领域：软件工程

作　　者：朱沂原

指导教师：李勇军

2024 年 12 月

Title: Research and Implementation of Fake News Detection Base on Image and Text Fusion

By

Yiyuan Zhu

Under the Supervision of Professor

Yongjun Li

A Dissertation Submitted to

Northwestern Polytechnical University

In Partial Fulfillment of The Requirement

For The Degree of

Master of Software Engineering

Xi'an, P.R. China

December/2024

学位论文评阅人和答辩委员会名单

学位论文评阅人名单

姓名	职称	工作单位
x	x	西北工业大学
x	x	x

答辩委员会名单

答辩日期	2025 年 3 月 y 日		
答辩委员会	**姓名**	**职称**	**工作单位**
主席	x	x	西北工业大学
委员	x	x	西北工业大学
委员	x	x	西北工业大学
秘书	x	x	西北工业大学

摘　要

随着移动互联网的发展，越来越多的人开始在个人社交帐号上发布各种帖子，包括文字、图片等信息，社交媒体已经成为人们获取信息、交流观点的重要平台。但是其中也存在不少虚假信息，给人们的生活和工作带来了诸多负面影响。鉴于人工核查往往成本高昂、效率低下，如何利用计算机技术自动检测虚假信息成为了当前亟待解决的问题。

目前已有许多关于虚假信息检测的研究，但部分研究仅关注纯文本之类的单模态虚假信息检测，忽略了多模态信息的重要性，而现有的多模态检测算法又存在模态惰性等问题，导致模型性能不佳、泛化能力差。此外，现有虚假信息检测算法往往在不同的框架下开发，缺乏统一的框架和接口，给算法的复现和比较带来了困难。而且开发这些算法模型时，在模型训练等方面也存在大量重复性的编码工作，降低了研究效率。针对上述问题，本文以基于图文融合的虚假信息检测方法研究与实现为研究课题，进行了研究探讨，主要工作体现在以下两个方面：

（1）本文提出了一种基于单模态交替的多模态虚假信息检测方法。该方法采用单模态交替学习的方式有效地解决了模态惰性的问题。在训练阶段，每轮训练时先提取文本特征，送入共享分类层完成文本单模态的虚假信息检测，计算文本分类损失并更新对应参数，从而完成文本模态的学习；然后提取图片特征，采用同样的方式进行图片模态的分类，并引入对比学习作为辅助任务，提升模型对图片特征的学习能力。而在推理阶段则将文本和图片模态的预测结果加权综合作为样本分类结果，以兼顾各模态对最终结果的贡献。本文在两个真实世界的数据集上进行了实验，结果表明本方法相较于其他基线模型具有更好的分类性能和更小的计算开销，具有较大的实际应用价值。

（2）本文设计并实现了虚假信息检测领域的首个开源算法库，集成了几十余种常用的虚假信息检测算法模型。作为一套通用的框架，该开源库提供了统一的模型训练和推理接口，规范了内部模型的实现，使得不同算法模型之间可以方便地进行比较和复现，其中包含的可视化等丰富辅助功能也保证了算法库的易用性和实用性。使用者亦可在算法库的基础上，复用已有的代码来快速开发新的算法模型，这也体现了算法库的可拓展性。本文通过大量复现性实验验证了算法库中模型的有效性，并提供了使用示例来说明算法库的使用方法，以此帮助更多研究者快速上手使用。

上述工作为虚假信息检测领域的研究提供了一些新的思路和方法，同时也提供了一套新的工具资源，有助于推动虚假信息检测的研究进展，也为相关领域的研究者提供了一些参考和帮助。

关键词：虚假信息检测；社交媒体；多模态；交替学习；算法库

本研究得到陕西省重点研发计划（编号 2024GX-YBXM-124）资助。

Abstract

With the growth of mobile internet, an increasing number of individuals are publishing various posts on their personal social media accounts, including text, images, and other types of information. Social media has thus become a significant platform for information sharing and discussion. However, it also contains substantial fake news, negatively affecting people's lives and work. Given the high cost and low efficiency of manual verification, the need to leverage computer technology to automatically detect fake news has become an urgent issue.

Many studies have been conducted on fake news detection; however, some of these focus exclusively on uni-modal detection, such as analyzing plain text, while overlooking the significance of multi-modal information. Furthermore, existing multi-modal detection algorithms often encounter issues such as modal laziness, which negatively impacts model performance and generalization capabilities. Additionally, many current fake news detection algorithms are developed within various frameworks, resulting in a lack of a unified framework and interface. This disparity complicates the reproduction of algorithms and hinders the comparison of different models. Moreover, the development of these algorithmic models involves considerable redundant coding efforts related to tasks like model training, which decreases research efficiency. To address these challenges, this thesis focuses on the research and implementation of fake news detection based on image and text fusion, and the primary contributions are reflected in the following two aspects:

(1) This thesis introduces a multi-modal fake news detection method based on uni-modal alternative learning. This approach employs uni-modal alternative learning to effectively address the issue of modal laziness. During each training epoch, text features are first extracted and processed through the shared classification layer to perform textual uni-modal fake news detection. This step includes calculating the textual classification loss and updating the corresponding parameters to complete the learning for the text modality. Subsequently, image features are extracted, and image modality classification is conducted similarly. Additionally, contrastive learning is incorporated as an auxiliary task to enhance the model's ability to learn image features. In the inference stage, this method weights and integrates the prediction results from both text and image modalities to generate the final classification results. The thesis presents a series of experiments conducted on two real-world datasets, demonstrating that the proposed method outperforms other baseline models in terms of classification performance while maintaining lower computational overhead. This indicates good feasibility for practical applications.

(2) This thesis also introduces the first open-source algorithm library for fake news detection, integrating more than a dozen widely used detection algorithms. As a general framework, the library offers a unified interface for model training and inference, standardizes the implementation of internal models, and facilitates easy comparison and reproduction of different algorithms. It also includes various auxiliary functions, such as visualization tools, to enhance usability and practicality. Users can leverage the existing code within the library to quickly develop new detection models, highlighting the library's scalability and versatility. A series of experiments are conducted to validate the effectiveness of the models in this library, and examples are provided to illustrate how to use the library. This initiative aims to assist researchers in efficiently conducting studies in the field of fake news detection.

The work presented above offers new ideas and methodologies for research in fake news detection, along with a comprehensive set of toolkits. These advancements can facilitate progress in the field of fake news detection and provide valuable references and support for researchers in related domains.

Key Words: Fake news detection; Social media; Multi-modal; Alternative learning; Algorithms library

The present work is supported by Key Research and Development Program in Shaanxi Province of China (Project No.2024GX-YBXM-124).

目 录

摘 要 I
ABSTRACT III
图目录 VII
表目录 IX
第 1 章 绪论 1
1.1 研究背景 1
1.2 研究内容 4
1.3 本文结构 5
第 2 章 相关研究 7
2.1 虚假信息检测算法 7
2.1.1 基于内容的虚假信息检测 7
2.1.2 基于社交上下文的虚假信息检测 9
2.2 深度学习算法库 10
2.3 现有研究的问题 12
2.4 本章小结 13
第 3 章 基于单模态交替的多模态虚假信息检测算法 15
3.1 多模态虚假信息检测的研究背景 15
3.2 单模态交替学习方法 16
3.2.1 符号定义及问题描述 16
3.2.2 算法流程 17
3.2.3 文本单模态特征学习 18
3.2.4 图片单模态特征学习 20
3.2.5 带梯度修正的图文共享层 22
3.2.6 推理阶段的多模态动态融合 23
3.2.7 复杂度分析 24
3.3 实验结果及分析 26
3.3.1 实验设置 27
3.3.2 对比实验结果及分析 29
3.3.3 消融实验结果及分析 32
3.3.4 案例分析 34
3.3.5 模型轻量性比较 38

3.3.6 模态缺失情况下的推理结果及分析 39
3.4 本章小结 41
第 4 章 基于深度学习的虚假信息检测算法库 43
4.1 虚假信息检测算法库的研究背景 43
4.2 算法库框架及主要功能模块 44
4.2.1 数据模块 45
4.2.2 模型模块 48
4.2.3 评估模块 50
4.2.4 训练器模块 52
4.3 可复现性实验 54
4.3.1 实现细节 54
4.3.2 数据集 54
4.3.3 实验结果及分析 57
4.4 使用示例 59
4.4.1 运行内置模型 59
4.4.2 开发新模型 63
4.5 本章小结 65
第 5 章 总结与展望 67
5.1 本文总结 67
5.2 未来展望 68
参考文献 69
附录 A 源代码 77
致 谢 79
在学期间发表的学术成果和参加科研情况 81

图目录

1-1 2022.6-2024.6 网民规模和互联网普及率 1
1-2 新浪微博社区管理中心公示的虚假信息 3

2-1 不同类型的虚假信息检测算法 7

3-1 多模态融合的不同方式 15
3-2 基于单模态交替学习的多模态虚假信息检测算法 17
3-3 用于文本特征编码的 TextCNN 结构 19
3-4 基于 MoCo 的图片对比学习 20
3-5 混淆矩阵 31
3-6 ROC 曲线 32
3-7 PR 曲线 32
3-8 消融实验的分类结果柱状图 34
3-9 Weibo21 数据集中的真实信息案例 35
3-10 Weibo21 数据集中的虚假信息案例 36
3-11 Weibo21 数据集中的模型分类错误案例 37
3-12 不同模型中的参数数量 38
3-13 不同模型在 Weibo21 测试集上的推理耗时 39
3-14 缺失不同模态时的推理结果柱状图 40

4-1 算法库框架示意图 45
4-2 `MultimodalDataset`对多模态数据的处理流程 46
4-3 `tokenize`函数与`transform`函数的不同处理方式 47
4-4 `AbstractModel`的 UML 类图 49
4-5 `calculate_loss`方法和`predict`方法的调用过程 50
4-6 自定义评估指标的计算函数 51
4-7 评估器类`Evaluator` 51
4-8 `Trainer`的工作流程 52
4-9 SpotFake 训练日志记录 61
4-10 TensorBoard 训练曲线记录 61

表目录

3-1 本章符号定义 16
3-2 数据集的数据汇总情况 27
3-3 本文方法 AlLNet 与其他基线方法的对比实验结果 30
3-4 AlLNet 不同变种的消融实验结果 33
3-5 模态缺失时的推理结果 40

4-1 FaKnow 中集成的算法模型 48
4-2 FaKnow 内置模型的复现性实验结果 58

第 1 章　绪论

1.1 研究背景

随着手机等电子设备的普及以及通信技术的发展，移动互联网已经成为人们日常生活中不可或缺的一部分，网民规模也在不断增长。中国互联网络信息中心于 2024 年 8 月发布了第 54 次《中国互联网络发展状况统计报告》。如图1-1所示，截至 2024 年 6 月，我国网民规模近 11 亿人（10.9967 亿人），较 2023 年 12 月增长 742 万人，互联网普及率达 78.0%，较 2023 年 12 月提升 0.5 个百分点。其中我国手机网民规模达 10.96 亿人，较 2023 年 12 月增长 528 万人，网民中使用手机上网的比例为 99.7%。与此同时，我国网民的人均每周上网时长为 29.0 个小时，较 2023 年 12 月提升 2.9 个小时[1]。

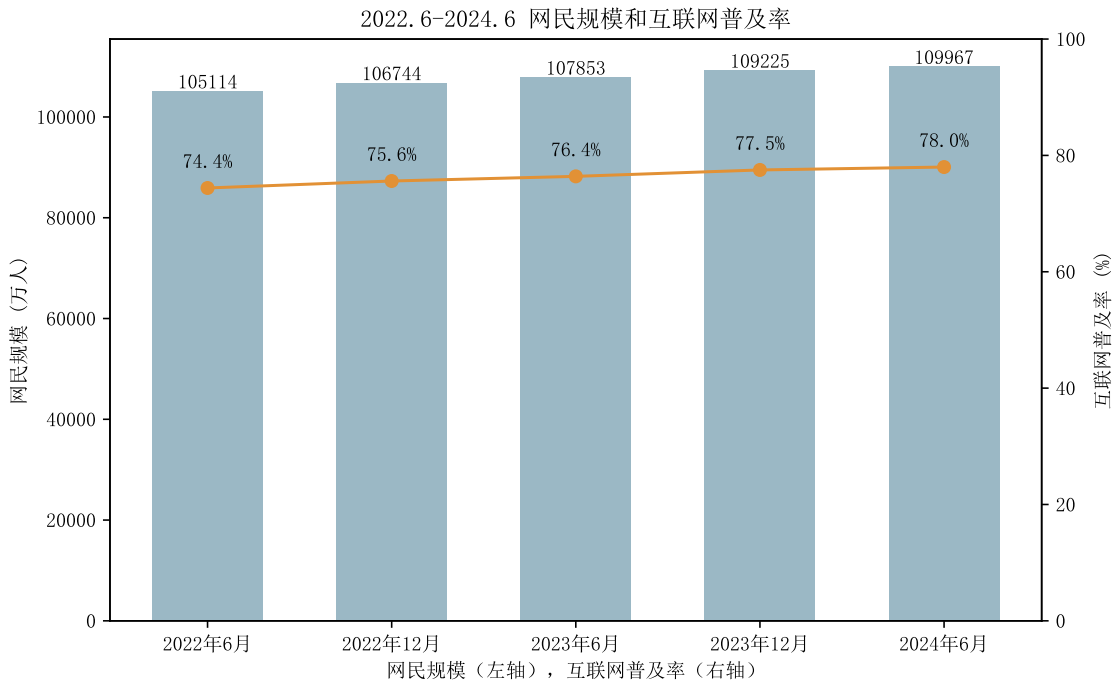


图 1-1　2022.6-2024.6 网民规模和互联网普及率

随之而来的是如微博、微信、推特（Twitter）、脸书（Facebook）等国内外社交媒体平台的快速发展，这些网络平台为网民提供了一个分享信息、交流观点的平台。越来越多的人开始在自己的社交帐号上发布各种帖子，包括文字、图片、视频等信息，分享个人的日常生活或者表达自己对时事新闻的观点看法。即便是从不在网络平台上发表帖子的社交媒体轻度使用者，也会通过浏览他人发布的内容来获取信息，了解社会热点、时事新闻等。

然而由于社交媒体的开放性和自由性，任何人都可以在社交媒体上发布信息，无需像报纸、书籍等传统媒体一样需要经过严格的编辑、审核、出版等环节，这就导致了社

交媒体上信息的真实性和可信度难以保证，各大网络平台成为了滋生虚假信息的温床。这些虚假信息可能是无意的错误信息，也可能是恶意的谣言，例如阴谋论（Conspiracy theories）、标题党（Clickbait）、伪科学（Peudoscience），甚至蓄意捏造的“假新闻”（Fake news）等[2]。尤其是现阶段中文互联网中存在许多“营销号”和社交机器人，它们通常捏造或者歪曲事实，使用颇具煽动性、夸张性的言论，发表带有仇恨或偏见歧视的错误观点，以此达到快速吸引流量，博人眼球的目的。同时由于回音室效应和信息茧房，虚假信息在社交媒体上的传播速度很快，很多未经证实的谣言在短时间内就会被大量转发，甚至引发舆论热议。Vosoughi 等人于 2018 年在《科学》杂志上发表的一篇文章指出，由于虚假信息比真实信息往往更新颖奇特，通常它的传播范围更大、传播速度也更快。他们收集整理了 2006 年至 2017 年 Twitter 上的谣言传播数据集，发现共有约 12.6 万条谣言被 300 多万人传播。在这些数据中，前 1% 的虚假信息的传播范围在 1000 到 100000 人之间，而真实信息的传播人数很少超过 1000，假新闻被转发的概率比真实信息高出 70%[3]。

这些虚假信息不仅会误导网民，可能对个人的生命财产安全造成危害，还会对社会稳定和国家安全造成威胁，引发社会恐慌，造成不良的社会影响。在 2016 美国总统大选期间，推特上就有大量虚假信息，其中包括一些虚假新闻网站发布的虚假新闻。同时也有许多社交机器人混在真实用户中，发布大量支持特定候选人的信息，并与真实用户进行评论、回复以及转发等互动。这些虚假信息可能会影响选民的投票决策[4]。在美国大选结束后，奥巴马和希拉里都在不同场合表达过“虚假新闻对大选结果产生了直接影响”的观点。在 2017 年法国总统大选期间也发生了类似的情况，有大量虚假信息在社交媒体上传播，试图影响选民的投票决策[5]。在新冠疫情期间，不少“营销号”利用疫情传播恐慌情绪，发布虚假疫情数据，甚至散布关于疫苗的谣言，中国社交媒体上也出现了大量的虚假信息，例如“抗生素能预防和治疗新冠病毒感染”、“吃大蒜能预防感染新冠病毒”、“定期用生理盐水清洗鼻子有助预防感染新冠病毒”、“肺炎疫苗能预防感染新冠病毒”等。这些有关新冠病毒的不实信息加大了医卫人员的工作难度，并向公众传播恐惧，引起不少群众大量抢购大蒜、食用盐以及抗生素等，甚至出现了一些因误信谣言而导致的悲剧[6]。而随着如 ChatGPT 等生成式人工智能技术的发展，造谣的成本大大降低，虚假信息的制造者还可以利用这些技术生成更加逼真的虚假信息，例如伪造图片或视频，使得虚假信息更加难以辨别。《中国互联网络发展状况统计报告》也指出 2024 年上半年青少年和“银发族”是新增的 742 万网民重要来源，青少年占新增网民的 49.0%，50-59 岁、60 岁及以上群体分别占新增网民的 15.2% 和 20.8%[1]。这些群体由于自身的认知水平和信息获取渠道的局限性，对网络信息的辨别能力较弱，更容易受到虚假信息的影响。因此，如何有效识别和过滤社交媒体上的虚假信息，保护网民的合法权益，维护社会稳定，成为当前亟待解决的问题。

目前，社交媒体平台和各国政府部门都在采取措施来应对虚假信息的传播。在 2016

年美国总统大选后，脸书和谷歌都采取了针对虚假信息的应对措施。脸书允许用户将帖子标记为虚假信息，这一标记过程需要经过人工事实核查，以确保标记的准确性；在脸书的基础之上，谷歌更加详细地展示了检测结果，不仅展示了经过事实核查的新闻片段，也展示了完成核查标记的人员信息。我国政府也高度重视网络虚假信息的治理工作，主要采取举报、监测等手段进行处理。中央网信办每年都会开展“网络清朗行动”，整治网络虚假信息，维护网络空间秩序。2023 年开展的“清朗・从严整治‘自媒体’乱象”和“清朗・打击网络水军操纵信息内容”专项行动旨在破解“自媒体”信息内容失真问题，全面清理网络水军违法违规信息。而 2024 年开展的“清朗・整治‘自媒体’无底线博流量”和“清朗・规范生成合成内容标识”专项行动则集中整治虚构摆拍等问题，处置利用生成合成技术制造谣言、营销炒作的违规账号，净化网络空间[7]。同时中央网信办（国家网信办）违法和不良信息举报中心和新华网也于 2018 年联合推出了中国互联网联合辟谣平台[1]，该平台依托于由 104 家单位组成的全国网络辟谣联动机制，实现谣言信息联动发现、联动查证和联动辟谣，为广大群众识谣辨谣、举报谣言提供了权威平台。如图1-2所示，新浪微博也鼓励用户举报投诉不实信息，并在在微博社区管理中心[2]对这些不实信息进行公示处理，对热点事件和谣言进行辟谣，提供权威信息，引导网民理性看待网络信息。

图 1-2　新浪微博社区管理中心公示的虚假信息

然而上述处理方案大多依赖人工核查和平台用户的举报投诉，但是这样进行逐条核对的成本过高，通常费时费力。因此目前急需自动化检测虚假信息的技术，能够快速、

[1]https://www.piyao.org.cn/
[2]https://service.account.weibo.com/

准确地识别社交媒体上的虚假信息，以帮助用户识别不实信息，并提高虚假信息的检测效率，减轻人工核查的负担。而深度学习则能很好地解决这一问题，通过先进的神经网络模型，可以在大幅降低人工成本和时间成本的同时，较为准确地从海量的社交网络信息中识别出潜在的虚假信息，揭露这些错误的言论或观点，有助于遏制虚假信息的传播。目前的研究主要集中在文本信息的虚假信息检测上，但是社交媒体上的信息往往是多模态的，包括文本、图片、视频等信息。文本信息可能会被伪造或篡改，图片和视频也可能被人工合成或编辑，单一模态的虚假信息检测技术难以适应社交媒体上的多模态信息。因此，如何有效地结合文本、图片等多模态信息，提高虚假信息检测的准确性和鲁棒性，成为当前亟待解决的问题。同时现阶段学术界涌现的各类虚假信息检测算法通常在不同框架下开发，复现困难并且其中存在大量重复性工作，目前也缺乏统一的基准（benchmark）将这些算法整合统一化。

1.2 研究内容

为解决上文提到的问题，本文以图文融合的虚假信息检测方法研究与实现作为研究内容，可分为两大版块：算法上，本文的研究主要聚焦以社交媒体上包含图片和文本的帖子作为学习数据，如何有效地利用其中的多模态信息来完成虚假信息检测任务，即判断给定的帖子内容是否为虚假信息；工程上，本文的研究主要聚焦在如何将现阶段种类繁多的虚假信息检测算法进行处理与整合，为它们设计一套统一的框架，省去算法开发过程中不必要的重复性工作。具体来说，本文的研究工作和研究特点如下所述。

- **基于单模态交替的多模态虚假信息检测方法：**传统的单模态虚假信息检测算法只能处理图片或者文本这种单一模态的信息，无法学习丰富的多模态特征，因而其识别虚假信息的能力较弱。而现有的多模态方法通常拼接图片和文本特征作为多模态表征，或者使用注意力等加权机制融合图文特征，存在模态惰性问题，学习不够充分，从而导致模型的判别能力有限。本文则提出了一种基于单模态交替的多模态虚假信息检测方法，该方法抛弃了传统的多模态加权融合的训练方式，转而采用全新的交替训练手段。每轮训练过程中，模型在分别提取图片和文本单模态特征后，依次将它们送入共享的分类头完成虚假信息检测任务，然后计算分类损失并更新对应参数。在反复迭代训练过程中，图片和文本模态的特征学习过程彼此独立、交替进行，从而避免了多模态加权融合，在能够学习丰富的多模态特征的前提下，有效地解决了模态惰性问题，同时图文共享的分类头也可以捕获到多模态交互信息。本方法也引入了基于动态字典的动量对比学习，将正负样本判别作为除虚假信息检测之外的辅助任务，从而提升模型对于图片特征的学习能力。由于不同模态信息对于最终分类结果的贡献比重不同，本方法在推理阶段将图片和文本模态各自的预测结果加权综合作为最终分类结果。考虑到当一个模态输出的熵越大时，它越容易产生错误的预测，本方法会对这个模态赋予越小的推理权重，以保证最终结果的准确性。同时本方法也十分轻量，模型参数量少且推理速

度快，还能应对模态缺失问题，非常适合真实世界应用场景。与一系列基线方法的对比试验均证明了此方法的优越性。

- **基于深度学习的虚假信息检测算法库：**近年虚假信息检测领域中涌现了许多优秀的虚假信息检测算法，这些虚假信息检测算法往往在不同的框架下开发，其使用方式也有较大区别，给想要复现它们的人带来许多困扰，同时这些虚假信息检测模型在开发过程中也存在着大量重复性的工作。为了解决这些问题，本文推出了首个虚假信息检测研究领域的深度学习算法库，这个库中包含了基于内容和基于社交上下文两大种类的常见的虚假信息检测模型，覆盖了模型训练和评估的全过程工作，并将数据、模型、训练与评估组织成一个统一的框架。它还提供了一系列丰富的辅助功能和工具，如通用的数据结构、可视化等。这样的统一的框架和接口既方便了用户快速上手，又为用户自定义新功能提供了良好的拓展性。此外，本文还设计了一系列复现性实验以验证算法库中集成模型的有效性和正确性，并提供了一些使用示例以说明该算法库的易用性和拓展性。此算法库既能帮助用户快速复现已有的虚假信息检测算法，又能为研究者提供一个统一的开发平台，省去了开发新模型中的不必要的重复性工作。

1.3 本文结构

本文共分为五个章节，各章节的内容安排如下所述。

第 1 章为绪论，以社交媒体作为引入，聚焦于社交媒体上虚假信息的泛滥现象以及虚假信息对社会产生的负面影响，介绍了本文的研究背景以及意义。然后分两个方面介绍了本文的研究内容，包括基于单模态交替的多模态虚假信息检测方法和基于深度学习的虚假信息检测算法库，并阐述其创新点。最后介绍论文的组织结构。

第 2 章为相关研究工作，分别从虚假信息检测算法和深度学习算法库两个方面介绍了与文本相关的研究工作。这一章首先介绍了基于内容和基于社交上下文两大类别的虚假新信息检测算法，对每个类别都论述了一些具有代表性的经典研究，包括它们的创新点和不足之处，并重点比较了本文的研究内容相较于前人工作的区别和优势。然后本章回顾了其他领域已有的深度学习算法库以及各自的功能特点，指出虚假信息检测领域也急需类似的开源库。

第 3 章则详细介绍了本文提出的基于单模态交替的多模态虚假信息检测方法。首先阐述这项工作的研究动机并对比此算法与以往研究的不同之处。然后讲解了算法的整体流程，包括图文的单模态学习、图文共享层的梯度修正机制以及推理时的加权融合步骤，并用算法伪代码来更好地说明单模态交替学习的特殊之处。最后给出了对比试验、消融实验、模型轻量性以及模态缺失等一系列实验结果及分析，证明了本章提出的方法的优越性。

第 4 章则以工程系统的视角阐述了本文提出的基于深度学习的虚假信息检测算法库。首先介绍了目前虚假信息检测研究中存在的痛点，从而说明这项工作的必要性。随

后介绍了本章提出的算法库的五大功能特点以及项目整体框架，并就算法库中最核心的数据、模型、训练器、评估这四个模块展开论述，着重介绍了它们的设计思路和实现方案，包括多模态数据集、统一的模型接口以及训练器工作流程等多个方面。同时也通过一系列复现性实验证明此算法库的有效性。最后还给出了算法库的简略使用说明。

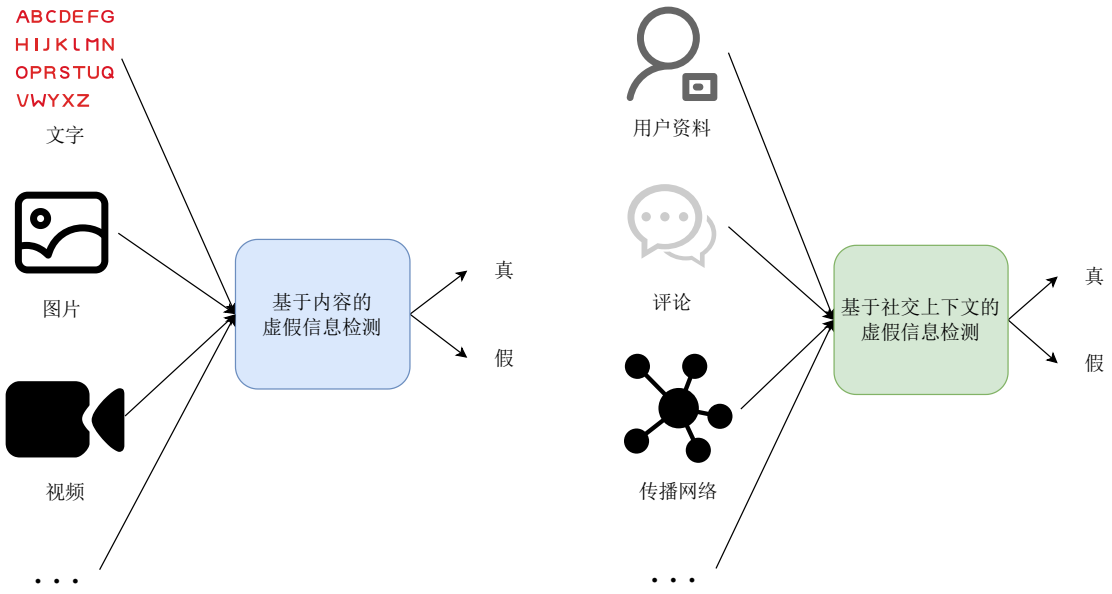
第 5 章为总结与展望，主要回顾了本文的两项研究工作，总结它们的研究特点与意义。同时分析了其中存在的不足以及改进之处，对未来的研究作了进一步展望。

第 2 章　相关研究

本章将回顾与本文所提出的基于单模态交替学习的多模态虚假信息检测算法、虚假信息检测算法库两个工作相关的研究，主要集中在两个方面：虚假信息检测算法和深度学习算法库。

2.1 虚假信息检测算法

如图2-1所示，目前主流的虚假信息检测算法可分两类：基于内容的检测算法和基于社交上下文的检测算法。下文将分别介绍这两类算法的研究现状，并分析它们的特点。



(a) 基于内容　　(b) 基于社交上下文

图 2-1　不同类型的虚假信息检测算法

2.1.1 基于内容的虚假信息检测

基于内容的虚假信息检测算法关注点主要在帖子本身的信息，它们通常从待检测帖子的内容出发，学习内容中的特征来完成分类任务，是最为简单直接的检测方式，特别适合早期检测场景。根据算法中使用到的内容特征，可以将其分为单模态和多模态两类。

(1) 单模态检测

单模态虚假信息检测算法通常只使用帖子中的文本或图片，提取单一模态的特征完成分类任务。虚假信息检测最早作为自然语言处理领域的研究问题，比较常见的是基于各类文本的检测方法，它们将帖子的文本内容作为模型输入，以 Word2Vec[8] 或者 BERT[9] 等方式获得词嵌入向量，并通过 LSTM 等网络结构提取文本特征来对样本分类。

马晶等人[10] 在 2016 年首次抛弃了传统手工特征的方式，将基于传统机器学习的

虚假信息检测方法升级为基于深度学习的方法，使用 RNN、LSTM 和 GRU 对帖子中的句子进行建模，以其中的隐含层输出作为样本的表征，最后送入分类器完成检测任务。CAMI[11] 模型使用 CNN 灵活地提取分散在输入序列中的关键特征，通过卷积核的滑动窗口扫描，得到了更加丰富的文本表征，能有效地识别错误信息并实现早期检测。ENDEF[12] 提出了一种实体去偏新闻检测框架，提取帖子中的实体信息，并引入辅助任务移除它们带来的偏差，提高了模型的泛化能力。受混合专家模型（MoE, Mixture of Expert）的启发，MDFEND[13] 利用领域门来聚合多个专家网络提取到的不同领域的特征，从而应对多领域虚假信息检测。

而基于图片的虚假信息检测算法则关注图片的特征，通常使用 CNN 提取图片的特征，再通过分类器完成检测任务。MVNN[14] 额外关注了那些经过伪造或者篡改的不实图片，分别设计频域子网络和像素子网络来提取图片的频域和空间域特征，再将它们以注意力机制融合送入分类器，提高了模型的检测效果。

(2) 多模态检测

随着社交网络的发展，用户发布的帖子往往图文并茂，图片和文本特征能彼此互补，在多模态交互过程中提供了丰富的信息，有利于后续分类任务，因此多模态虚假信息检测逐渐成为研究的热点。

SpotFake[15] 使用预训练的 VGG[16] 和 BERT 模型作为编码器，分别从图片和文本中提取单模态特征，然后将二者拼接作为多模态表征送入分类器。由于模型容易在不同领域的新闻样本上过拟合，学习到来自新闻领域的噪声，EANN[17] 使用 TextCNN[18] 从文本中提取特征，同时还引入了事件对抗学习作为辅助任务，通过事件对抗神经网络让模型学习到未见过的事件可迁移特征，以提高模型在不同新闻领域帖子的检测的泛化能力。SAFE[19] 则未直接使用图像特征，而是更关注不同模态之间的内容一致性，创新性地使用预训练模型 ShowAndTell[20] 为图片生成文字标题，将其与帖子中的文本信息计算特征的余弦相似度，以此作为多模态一致性的度量，提高了模型的检测效果。由于变分自编码器（VAE, Variational Autoencoder）能通过优化观测数据的边际似然性来学习概率潜变量模型，MVAE[21] 在拼接图文特征的基础上，引入了变分自编码器这一结构，从多模态特征中重构出原始数据，进而让模型能学习到跨模态的共享表征。

MCAN[22] 同样兼顾了图片的频域和空间域特征，使用离散余弦变换和卷积神经网络提取前者，同时通过预训练 VGG 模型得到后者。然后将这两个特征与文本特征一起，不断地经过多个协同注意力模块来完成交互，从而形成最终的多模态表征，提升了模型的检测效果。HMCAN[23] 使用层次化编码网络（Hierarchical Encoding Network）提取不同层次的文本特征，并通过复杂的多模态上下文注意力网络（Multimodal Contextual Attention Network）与图像特征融合，得到了更丰富的多模态表征。COSMOS[24] 则主要关注虚假信息中那些断章取义的图文对，使用自监督的训练策略让模型学会将图像中的对象与文本对齐，以便检测图文内容不一致的虚假信息。而 EM-FEND[25] 不仅仅比较图

文模态的一致性，还利用到了图片中嵌入的文本信息，并引入命名实体来捕获丰富多样的多模态线索用于虚假信息检测。与前人那些只在频域中处理图像特征的方法不同，FRSU[26] 认为不同模态特征在频域的单峰分布更具区分度，创新性地将文本特征转换到频域，在频域中进行多模态虚假信息检测。具体来说，它使用离散傅里叶变换（DFT, Discrete Fourier Transform）将图文的空间特征转化为频谱，然后压缩单峰频谱特征，再进行跨模态频谱协同选择，最后通过逆变换（IDFT, Inverse Discrete Fourier Transform）将特征还原到空间域完成样本分类。

除去基于注意力机制的多模态融合，CAFE[27] 还额外关注了单模态特征，采用跨模态对齐作为辅助任务并计算余弦损失，通过变分自编码器学习跨模态歧义，然后使用衡量不同模态数据分布之间的 KL 散度来引导最终的单模态和多模态融合。BRM[28] 提出了一种全新的自举多视图方法，分别从文本、图像模式和图像语义三个角度提取特征，然后使用改进后的混合专家网络来进行多模态特征融合，并采用自举的方式综合利用多个视图的信息。为了提升模型的分类效果，该算法还保留了单模态特征的视图并学习到了图文跨模态的一致性。考虑到相似的信息样本倾向有相同的样本标签，ALGM[29] 在深度神经网络的基础上创新地使用了马尔科夫随机场（MRF, Markov Random Fileds）来对样本之间的关系进行建模，将新闻作为随机无向图的结点，新闻样本之间的关系作为边，样本标签视作符合 Gibbs 分布的随机变量。在随机场中势能函数（Energy Function）的定义上，该方法使用样本预测标签的负对数似然估计来计算单变量的一元势函数，同时使用样本表征的余弦相似度来计算二元势从而反映相邻变量之间的关系。

2.1.2 基于社交上下文的虚假信息检测

基于社交上下文的检测算法通常将帖子放在社交网络中，而非视作一个孤立的样本个体，这类算法往往会利用用户个人资料信息、帖子的评论、信息传播路径等各种来自社交网络的信息特征进行检测。例如，当一个用户历史上发布的帖子中有很多虚假信息，那么该用户发布的新帖子也有可能是虚假信息。而相较于那些未经身份认证的个人用户，一个经过机构认证的官方媒体账号发布的帖子则具有较高的可信度。此外，当一则帖子评论数很多且评论区中有很多用户指出了其中内容的虚假性，那么这则帖子也有可能是虚假信息。

GCAN[30] 将用户的个人资料信息作为用户表征，通过卷积神经网络和门控循环单元提取其中的特征，还构建了以转发用户为结点的传播图，利用图卷积神经网络提取传播图的特征，最终与推文的特征融合以检测虚假信息。从谣言传播方向这一点出发，Bi-GCN[31] 分别构建自顶向下（Top-Down）和自底向上（Bottom-Up）的帖子传播图，以图卷积提取谣言传播和弥散的特征，最终将二者拼接送入分类器。考虑到用户偏好对鉴别虚假信息的作用，UPFD[32] 将待检测帖子的发布者的历史推文作为内因，同时构建了帖子传播图作为外因，把内因外因结合来判断帖子的可信度。

Fang[33] 构建了一个具有用户、新闻和信息源三类节点异质图，通过 GraphSage[34]

和 Bi-LSTM 提取特征，并引入无监督的近邻损失（Proximity Loss），自监督的立场损失（Stance Loss）和有监督的虚假信息分类损失（Fake News Loss）三个损失函数来共同优化模型。DUDEF[35] 将帖子的评论信息也纳入考虑，使用双重情感网络分别从帖子原文和评论中提取发布者情感和社会情感，然后计算二者之间的差距（Emotion Gap），以此作为虚假信息检测的依据。由于传播图中未直接相连的帖子节点也可能存在关联，EBGCN[36] 采用贝叶斯方法来建模帖子之间的潜在关系，使用节点表征不断更新图的邻接矩阵，进而补全潜在边，然后通过图卷积神经网络形成最终的帖子表征。

2.2 深度学习算法库

目前许多应用领域都有一些专门设计的开源算法库，这些算法库大多集成了各自领域常见的模型，覆盖了模型训练等一系列流程，并提供了便捷的调用接口，使得用户能够快速上手。

首先，在传统的机器学习和数据挖掘领域有着非常多经典且广泛使用的算法库，它们作为早期的开源工具，为研究者提供了大量的算法实现和数据集，极大地推动了机器学习领域的发展，同时也为后续出现的深度学习算法库提供了宝贵的经验。Scikit-learn[37] 是一个著名的的机器学习算法开源库，适用于中等规模的有监督和无监督问题，包含线性回归、决策树、K 近邻等算法，并提供了数据预处理、数据降维、网格搜索、交叉验证等常用功能。WEKA[38] 是一个 Java 编写的数据挖掘软件，提供了大量的数据预处理、分类、回归、聚类、关联规则挖掘等算法，内置常用的基础数据集并能解析多种数据文件格式，同时还支持实验设计、结果可视化等功能。更重要的是，它既提供了命令行接口，又支持外部第三方包导入，还能作为一个桌面端应用程序运行并提供了一个友好的图形界面，使得用户能够直观地查看数据集的特征和算法的执行过程，适用于数据挖掘等领域的研究和应用。

计算机视觉作为深度学习领域的一个重要分支和研究热点，学术界和工业界都有许多的开发者致力于为其构建专门的深度学习算法库，其中最具代表性的当属 OpenMMLab[1]开源社区推出的一系列计算机视觉库。该组织最初由香港中文大学与商汤的研究人员发起，旨在为计算机视觉提供一个统一的开源平台，自 2018 年成立以来陆续推出了 MMCV[39]、MMDetection[40]、MMSegmentation[41] 等几十余个知名开源算法库，涵盖了图像分类、目标检测、目标分割、超分辨率图像生成等 20 多种不同类型的计算机视觉研究任务，集成了 2400 多个模型并支持硬件部署，广泛应用于学术研究和工业应用。以 MMCV 为例，作为一个面向计算机视觉的基础库，它提供了图像和视频处理、图像和标注结果可视化、图像变换、多种 CNN 网络结构以及高质量实现的常见 CUDA 算子等功能，支持 Linux、Windows 和 macOS 等多种操作系统，同时也是 OpenMMLab 系列其他开源库的基础依赖。近年，开发团队也推出了新一代训练引擎 MMEngine[42] 来取代 MMCV 成为新的基础框架，新引擎提供了更高级且通用的训练器、接口更加统

[1]https://openmmlab.com/

一的开放架构以及可定制化程度更高的训练流程。更重要的是，MMEngine 还集成了 DeepSpeed 等主流的大模型训练框架，支持混合精度训练等丰富的训练策略，也兼容英伟达 CUDA、苹果 MPS 等主流的训练芯片，还可以用于非 OpenMMLab 项目中。到目前为止，共有 1400 多名贡献者参与了 OpenMMLab 的开源社区，并且他们在 GitHub 上开源的代码也已经获得了超过 83000 的 Star 数。

推荐系统作为一个十分成熟的应用领域，也有着许多专门的推荐系统算法库，它们集成了各种推荐算法模型。RecBole[43] 是一个开源的推荐系统算法库，它提供了一个统一的框架来复现和开发各种推荐算法，也有助于标准化推荐算法的评估协议。它在 28 个基准数据集上集成了 91 个推荐模型，涵盖了通用推荐、顺序推荐、上下文感知推荐和基于知识的推荐四个类别，同时提供了标准化的推荐算法评估指标，为已有算法的复现和新模型的开发提供了极大的便利。MMRec[44] 是一个专门面向多模态的推荐算法库，提供了覆盖数据预处理、多模态推荐模型、多模态信息融合、性能评估的全栈工具箱，简化了多模态推荐模型的实现和比较过程，并提供了一个统一可配置的平台，可以最大限度地减少实现和测试多模态推荐模型的工作量。它支持多种常用的多模态模型，包含从传统的矩阵分解到现代基于图的算法，能够同时融合来自多个模态的信息。

而在智慧教育领域，也存在一些专门的开源算法库。pyKT[45] 是一个用于训练基于深度学习的知识追踪模型的开源工具箱，它包括一套标准化的集成数据预处理程序，涉及 7 个以上不同领域的流行数据集、5 个详细的预测场景、十余种常用的基于深度学习的知识追踪算法，并提供了统一的模型评估方式，以便相关领域的研究者进行统一标准化的广泛实验。MWPToolkit[46] 是首个用于求解数学应用题的开源框架，它将现有的数学应用题求解器的程序分解为多个核心组件，并将模型解耦为高度可重用的模块，同时还提供了超参数搜索功能。它在 4 个广泛使用的单方程生成基准和 2 个多方程生成基准上实现并比较了 17 个求解器模型，适合研究人员复现各种求解器模型或快速开发新的数学应用题求解器。

除此之外，还有许多其他领域的优秀开源算法库。Transformers[47] 是一个由各种精心设计的基于 Transformer[48] 架构的最新模型组成的开源库，并提供了统一的应用程序接口。该开源库最初由 Hugging Face[2]公司开发，支持了大量的自然语言处理任务，包括文本分类、命名实体识别、问答、文本生成等，随着模型的不断更新和迭代，现在已经拓展到计算机视觉、音频、多模态等多个领域，为研究者提供了一个统一的深度学习模型库。Transformers 支持 PyTorch、TensorFlow、JAX 三种不同的深度学习框架，内置的模型也可以导出为 ONNX 和 TorchScript 格式以便部署。难能可贵的是，它不仅提供了大量开箱即用的 pipeline，还搭建了一个社区共建的模型 hub，支持用户分发和使用各种预训练模型，极大地减少了从头开始训练模型的开销，这也让它成为了 Transformer 架构流行以来的事实标准，在学术界和工业界都得到了广泛应用。

[2]https://huggingface.co/

LibCity[49] 是一个针对交通预测的算法库，它将与交通预测相关的所有必要步骤或组件包含到系统管道中，包含各种数据集、机制、模型和工具。它能适用于交通预测中的四大主流任务，包括交通速度预测、交通流量预测、按需服务预测和轨迹下一位置预测。OpenHGNN[50] 是一个开源的异质图神经网络工具包，内置 40 多种前沿的异质图神经网络模型，涉及节点分类、链接预测、推荐三类任务。它提供了易用的接口以便在给定的模型和数据集上运行实验，且集成了 optuna[3]进行超参数优化，用户可以自定义定制化的任务/模型/数据集来应对新的场景或应用新的模型，同时底层基于的 DGL[4]框架也保障了算法的高效性。

2.3 现有研究的问题

尽管目前已有许多针对虚假信息检测和深度学习算法库的研究，但是它们仍然存在一些问题。

基于社交上下文的虚假信息检测算法虽然充分利用了社交网络中的丰富信息，但是由于社交网络的复杂性，它们的计算复杂度通常较高，且模型参数量较大。此外，社交上下文信息的收集也并非易事。抓取用户的个人资料以及历史推文等数据较为棘手，往往受限于各大社交平台的隐私保护政策。而且构建帖子的传播路径也需要大量人工标注，难以推广到其他数据集。更重要的是，在帖子发布的初期，由于缺乏足够的评论、转发等社交上下文信息，这些方法在早期检测场景下的效果并不理想。

在基于内容的虚假信息检测方法中，单模态检测方法都只关注单一模态的特征，只利用到了其中的文本或图片，并未进行多模态融合。而多模态特征交互过程中恰好能提供许多补充信息（如跨模态一致性），单模态方法则忽略了这一点，用于最终判别的信息非常少，因此识别效果往往不佳。

虽然现有的多模态检测方法利用到了文本和图片的丰富信息，但是大多忽略虚假信息检测中不同模态所占的不同比重，而是一体化地简单处理，直接将图片和文本的特征简单拼接或者相乘，导致了模型的泛化能力不足，影响最终分类效果。尽管部分方法采用了复杂的多模态融合机制（如多模态注意力机制、多模态融合网络），但它们在进行加权融合的过程中都面临着模态惰性的问题，即某个权重小的模态会在训练和优化中被忽略，模型会更倾向学习权重大的模态特征，多模态特征难以得到充分利用。同时这些复杂的融合机制也导致了模型参数量的增加，计算复杂度急剧上升，需要更多的计算资源。并且这些方法在推理时还需要图文完备的样本，难以应对模态缺失问题。

此外，尽管在深度学习的不同研究方向都涌现出许多便捷易用的开源算法库，也有许多研究者不断献身于算法库的开发和研究，但是在虚假信息检测任务中一直缺乏这样的统一的开源算法库，为这个领域的研究工作带来了很大的不便，每次复现已有模型或者开发新模型都需要从头开始编码实现，浪费了大量的时间和精力。

[3]https://optuna.org/
[4]https://github.com/dmlc/dgl

针对前 3 点问题，本文提出了基于单模态交替的多模态虚假信息检测算法，该算法在利用到丰富的多模态信息的同时，又通过交替训练的方式有效避免了模态惰性的问题，并在推理阶段以熵加权综合文本和图片的预测结果，兼顾了不同模态的比重。同时该算法对算力资源要求低，还能适应模态缺失的样本，更适合实际应用。针对最后一点问题，本文设计并实现了虚假信息检测算法库，它集成了虚假信息检测领域常见的模型和评估指标，并提供了统一的应用程序接口，简化了虚假信息检测算法的编码开发流程。后续的第 3 章和第 4 章将分别详细介绍这两项工作。

2.4 本章小结

本章的主要内容为虚假信息检测算法和深度学习算法库两个方面的相关研究，分别从基于内容和基于社交上下文两大类别概括了虚假信息检测领域的已有工作，并介绍了其他领域的一些优秀开源算法库。前人的工作具有重要的参考价值，为本文的研究提供了重要的理论基础和技术支持；但更重要的是，本章还分析了现有研究中存在的问题与不足，并为后续的工作提出了解决方案。

第 3 章　基于单模态交替的多模态虚假信息检测算法

在基于图文多模态的虚假信息检测任务中，多模态特征融合的重要性不言而喻。然而，各模态对最终检测结果的贡献度各异，并且由于数据分布上的显著差异，简单的一体化处理方式并不适用。现有研究虽尝试借助注意力机制实现多模态融合，但仍面临模态惰性难题，即某一模态可能因权重较低而在训练过程中被忽视，进而导致其未能获得充分的学习。本章提出了一种基于单模态交替的多模态虚假信息算法。它在训练阶段分别从图片和文本提取单模态特征，并反复交替进行独立训练，避免了模态惰性问题，同时引入对比学习进一步提升了图片模态的判别能力。在推理阶段，此算法用熵衡量不同模态的不确定性，并将它们的预测加权综合作为最终结果。此外，这个算法模型更加轻量，并且能应对模态缺失问题，更适合实际场景。本章也通过在两个中英文真实世界数据集上开展的一系列实验证明了此方法的优越性。

3.1 多模态虚假信息检测的研究背景

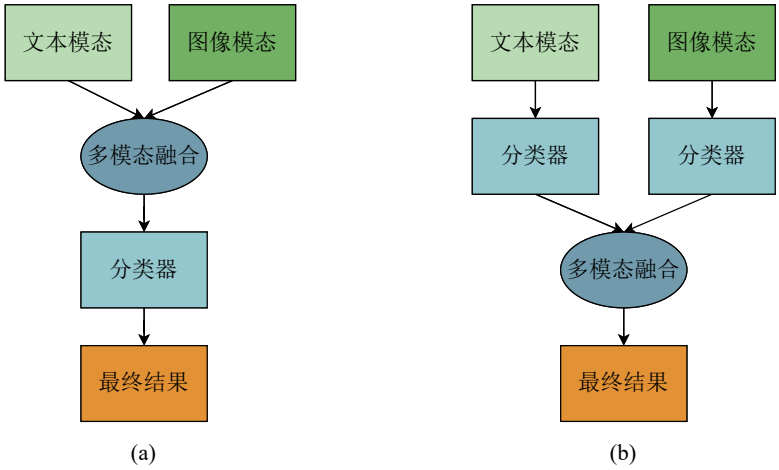


图 3-1　多模态融合的不同方式

近年来，许多研究分别提出了不同的基于多模态的虚假信息检测算法，这些方法能充分利用这些帖子中文字和图片中蕴含的信息，学习其中的特征以鉴别不实信息。尽管这些方法基于不同的创新思想，它们都可以被归为一类方案：先从不同的单模态中分别提取特征，然后再将它们融合获得多模态表征用于最终的分类任务（如图3-1(a) 所示）。但是虚假信息检测作为从自然语言处理衍生的任务，来自图片和文字等不同模态的特征对于最终结果的贡献和比重不一样，同时它们各自在数据分布上也存在显著差异。部分方法[15, 17, 51] 忽略了这一点，简单地采用拼接的方式将图文特征融合。还有些研究[22, 23, 27] 虽然使用注意力机制等方法对图文进行加权融合，但是存在模态惰性问题：某个模态被赋予较小的权重，导致在反向传播时其梯度也变小，对应模型中的部分参数

更新变慢，最终这个模态在训练中被模型忽略，没有得到充分的学习。此外，为了学习多模态表征，不少方法使用了注意力等复杂的融合机制，导致模型的参数变多，计算复杂度急剧上升。同时送入模型计算的样本也需要完备的图片和文字信息，不能应对模态缺失问题，难以应对真实世界中的场景。

为了解决上述问题，本章提出了一种基于单模态交替的多模态虚假信息算法 **AlLNet**（**Al**ternative **L**earning **Net**work），聚焦于多模态加权融合中的模态惰性问题。在训练阶段，模型分别从图片和文本提取单模态特征后，不将它们进行加权融合，直接分开依次交替训练，图片和文本特征分别对样本进行分类预测，然后计算损失更新参数，这样每个模态都能得到充分学习。另外，考虑到在对图片单独训练时，其判别能力有限。受 MoCo[52] 的启发，AlLNet 还额外引入了对比学习以提升图片的预测效果。而到了推理阶段，AlLNet 采用类似晚期融合的方式（如图3-1(b) 所示），图片和文本分别对样本进行预测并将它们加权求和作为最终结果，这样在图文分开学习的前提下又能兼顾到了不同模态的不同比重。考虑到当一种模态在预测中表现出更高的不确定性时，它更容易产生错误的预测，因此每个模态的预测结果的熵将会作为它在最终结果中的权重。此外，此算法模型并未使用任何诸如注意力机制之类的复杂度多模态融合方法，具有更少的参数量和更快的推理速度。并且当样本缺少某个模态时，可以直接使用剩下的另一个模态的预测结果作为最终样本分类结果。

3.2 单模态交替学习方法

3.2.1 符号定义及问题描述

表 3-1　本章符号定义

符号	含义
m	某个特定的模态
R_m	该模态的表征
d	该模态表征的维度
f_m	该模态的特征编码器
θ_{f_m}	该模态的特征编码器的参数
h	图文共享分类层
θ_h	图文共享分类层的参数
f_{v_k}	对比学习中的动量编码器
$\theta_{f_{v_k}}$	对比学习中的动量编码器的参数
$loss_m$	该模态的损失
e_m	该模态输出的熵
$\hat{y}_m$	该模态预测结果
λ_m	该模态的权重
n_c	TextCNN 的卷积核数量
n_s	TextCNN 的窗口数

表3-1列出了本文中用到的符号定义，其中 $m \in \{t, v\}$ 代表了多模态学习过程中图

片模态 v 和文本模态 t 中的任意一个模态。

给定一个社交媒体中的待检测多模态帖子 $P = \{W, I\}$，其中包含文本信息 W 和图片信息 I。对于文本模态 t 和图片模态 v，它们分别经过各自的编码器 f_t 和 f_v 得到的最终表征为 $R_t \in \mathbb{R}^d$ 和 $R_v \in \mathbb{R}^d$，其中 d 为特征的维度。样本的真实标签和预测标签分别为 $y \in \{0, 1\}$ 和 $\hat{y} \in \{0, 1\}$。而多模态虚假检测任务则是根据 P 中的图文特征来判断它是虚假信息（$\hat{y} = 0$）或真实信息（$\hat{y} = 1$）。从数学角度表达，本文的任务就是找到一个函数 f，使得 $\hat{y} = f(W, I)$ 与 y 尽可能接近。

3.2.2 算法流程

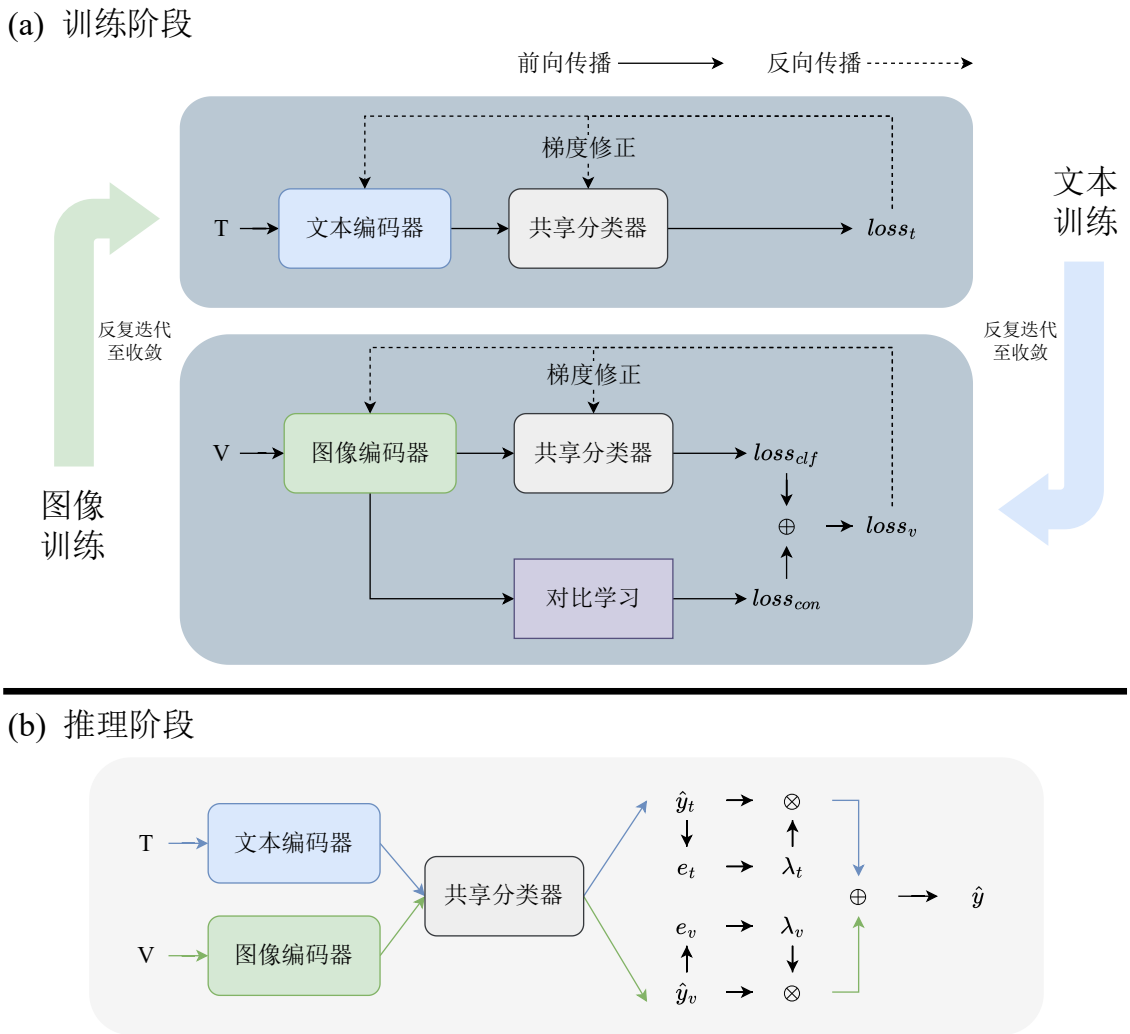


图 3-2　基于单模态交替学习的多模态虚假信息检测算法

本章提出的基于单模态交替学习的多模态虚假信息检测算法的框架如图3-2所示，该算法主要分为训练阶段和推理阶段两个部分，并在不同的部分采用了不同的策略来处理文本和图片的特征学习和融合。

在每一轮训练中，文本和图片的学习被分为两个独立的阶段。模型先对文本信息编

码获得其单模态特征，送入分类头得到文本预测结果，计算文本分类损失 $loss_t$ 并更新对应文本特征编码器和分类头的参数；再同样地对图片信息进行编码计算，送入分类头得到图片预测结果，计算图片损失 $loss_v$ 并更新对应图片特征编码器和分类头的参数。在整个模型训练过程中，程序会反复迭代上述步骤直至模型收敛，文本和图片的训练依次交替、循环往复，从而保证各自的特征都能得到充分学习，避免了模态惰性。同时文本模态和图片模态还共享同一个分类头来学习跨模态融合的特征，并且训练中还引入了梯度修正机制来修改反向传播中的梯度数值，从而避免每次更新参数时遗忘上一次学习过的模态特征。此外，为了提升图片的判别能力，此算法还额外引入了对比学习作为辅助任务，以 InfoNCE[53] 计算对比损失 $loss_{con}$，与图片分类损失 $loss_{clf}$ 一起作为最终图片模态损失 $loss_v$ 来更新图片编码器的参数。

而在推理过程中（图3-2(b) 所示），本方法则兼顾文本模态和图片模态的预测结果，并通过熵来衡量每个模态预测结果的不确定性，计算出每个模态的各自权重 λ_t 和 λ_v，将它们的预测输出 $\hat{y}_t$ 与 $\hat{y}_v$ 加权综合作为最终预测结果 $\hat{y}$，以此充分利用多模态信息。

下文将分别介绍此算法中的 4 个重要模块：文本单模态特征学习、图片单模态特征学习、带梯度修正的图文共享层、推理阶段的动态融合。

3.2.3 文本单模态特征学习

(1) 文本特征编码

帖子的文本模态 t 的内容经过分词操作后变为一个连续的单词列表 $W = \{w_1, w_2, \cdots w_l\}$，其中 l 为输入的文本序列长度。如公式3-1所示，本算法先使用预训练 BERT[9] 模型得到对应的词向量 T，其中 $token_i \in \mathbb{R}^{d_t}$ 代表 w_i 经过 BERT 最后一个隐含层的输出，d_t 则是输出向量的维度。

$$T = \{token_1, token_2, \cdots, token_l\} = \textbf{BERT}(W) \tag{3-1}$$

随后再使用 TextCNN 作为文本特征编码器 f_t，对文本特征进行进一步编码得到最终的文本表征。传统的 CNN 通常用于图像处理，但是在文本特征提取中，TextCNN 将文本视为一维的图像序列，通过卷积核的滑动窗口来提取文本的局部特征。它能充分利用带有不同窗口大小（即感受野）的多个卷积核来捕获不同层次的语义特征，特别适合虚假信息检测任务中的短文本信息提取。

以一个窗口大小为 s 的卷积核为例，该卷积核对文本中连续的 s 个单词 token 采用公式3-2中的卷积操作，得到第 i 个特征 c_i。

$$c_i = \sigma(W_c \cdot token_{i:i+s-1} + b_c) \tag{3-2}$$

其中 $W_c \in \mathbb{R}^{s \cdot d_t}$ 和 b_c 分别为这个卷积核的权重和常数偏置项，$token_{i:i+s-1}$ 代表第 i 个单词 token 到第 $i+s-1$ 个单词 token 的拼接，σ 为 ReLu 激活函数。

这个卷积核会对句子 $T = \{token_1, token_2, \cdots, token_l\}$ 中的所有词向量依次用此窗

口扫描并运算得到特征图 $c \in \mathbb{R}^{l-s+1}$。

$$c = [c_1, c_2, \cdots, c_{l-s+1}] \tag{3-3}$$

由于该特征图中的每个元素都是由一个卷积核对窗口大小为 s 的词向量计算得到的，因此它能捕获文本中的局部特征。但是这个特征图的特征维度仍然很高，存在信息冗余，因此还需要对其再使用最大池化操作获取其中最重要的信息，作为对应这个卷积核的特征，得到池化特征 $\hat{c} = \mathbf{max}\{c\}$。

为了获取不同层次的语义，本算法使用了 n_s 个不同大小的窗口，每个窗口都对应 n_c 个卷积核，将这些卷积核按上述公式3-2、3-3运算后的结果拼接作为最终的文本表征 $R_t \in \mathbb{R}^d$，其中 $d = n_s \cdot n_c$。图3-3展示了 TextCNN 的结构，图片最左侧的矩阵代表输入的词向量矩阵，句子中每个词元 token 对应矩阵中的一行词向量。图中黄、绿、蓝三种颜色的方块分别代表了三个不同窗口大小的卷积核，每个卷积核经过卷积操作得到的特征图以及最大池化操作得到的池化特征均用对应颜色的线表示。最后再将这三个不同大小窗口的特征拼接在一起，得到最终的文本表征 R_t。

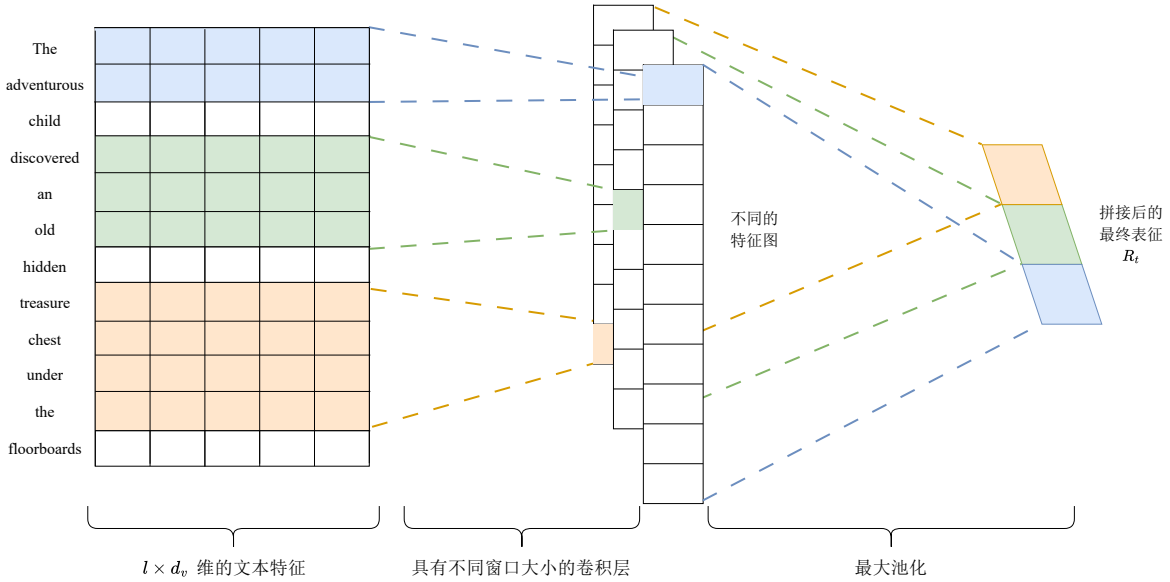


图 3-3　用于文本特征编码的 TextCNN 结构

(2) 文本单模态分类

对于经过编码器得到的文本表征 R_t，AlLNet 会使用一个基于多层感知机的图文共享的分类器 h（在3.2.5节将会对其展开详细讨论），按公式3-4计算得到文本的分类结果 $\hat{y}_t$，其中 W_h、b_h 为 h 的参数。

$$\hat{y}_t = \text{SoftMax}(W_h(\sigma(R_t)) + b_h) \tag{3-4}$$

随后按公式3-5使用交叉熵计算文本分类的损失 $loss_t$，并通过反向传播更新

TextCNN 编码器 f_t 和共享分类头 h 中的参数，其中 y 为样本的真实标签。

$$loss_t = -[y\log\hat{y}_t + (1-y)\log(1-\hat{y}_t)] \tag{3-5}$$

3.2.4 图片单模态特征学习

(1) 图片特征编码

与文本的特征处理方式类似，对于输入的图片模态 v 的特征 $I \in \mathbb{R}^{3\times width\times height}$，本算法采用预训练的 ResNet[54] 模型来对它进行初步编码，将 ResNet 模型中倒数第二层的输出作为为图像基本特征 $V \in \mathbb{R}^{d_v}$，其中 3、$width$、$height$、d_v 分别图片的通道数、宽度、高度和 ResNet 倒数第二层输出特征的维度。

$$V = \textbf{ResNet}(I) \tag{3-6}$$

随后再使用一个简单的线性层 f_v 作为图像特征编码器，得到最终的图像表征 $R_v \in \mathbb{R}^d$，其中 $W_v \in \mathbb{R}^{d\times d_v}$ 和 b_v 为可训练的参数。

$$R_v = f_v(V) = W_v V + b_v \tag{3-7}$$

(2) 图片对比学习

考虑到光使用图片特征对于虚假信息鉴别的帮助有限，此处引入对比学习作为除图像分类任务外的辅助任务来训练图像编码器，提升它对图像特征学习的能力，尤其是那些难以区分的困难样本的特征。

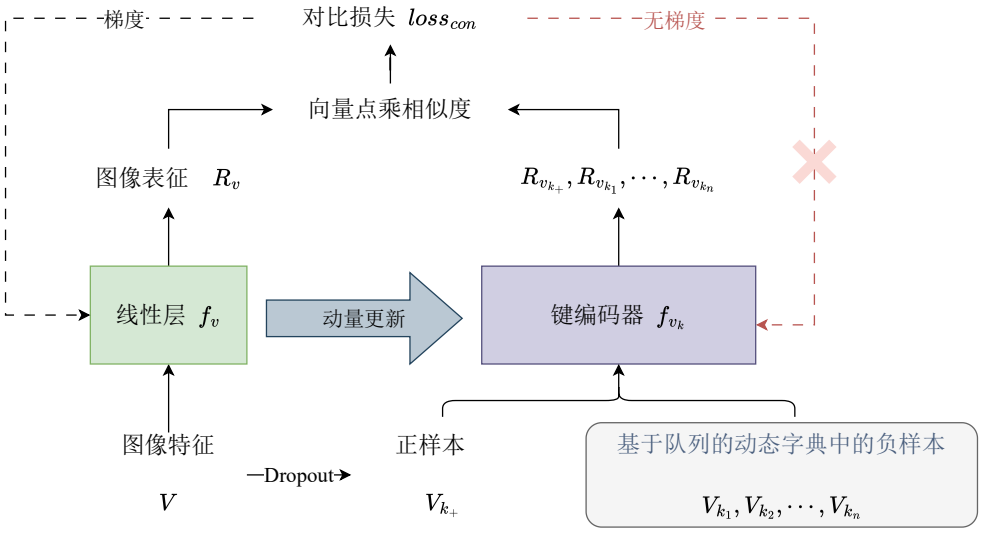


图 3-4　基于 MoCo 的图片对比学习

如图3-4所示，受 MoCo[52] 的启发，本算法采用动量对比学习的方式，以队列构建查询字典，解决用于对比的样本数量过少的问题，再使用动量来更新键编码器（key

encoder），来避免每次对比样本特征不一致的问题。由于此处主要论述对比学习，本文使用的术语与 MoCo 中的保持一致，将锚点样本（anchor）称为查询字典中的查询（query），而把正（positive）、负（negative）样本不加区分地统称为查询字典中的键（key），图像特征编码器 f_v 即为查询编码器（query encoder），而动量编码器 f_{v_k} 等价于键编码器（key encoder）。下文对于这些名称将不再赘述。本方法的对比学习有 4 个关键点：正负样本特征编码、对比损失计算、动态字典和动量更新，下文将分 4 段分别介绍。

对于每个图片样本的特征 V，将它自身作为锚点样本（即查询），以 f_v 作为查询编码器，得到锚点样本表征 $R_v \in \mathbb{R}^d$（在图像特征编码步骤中，模型已按公式3-7完成这一计算过程）。而将 V 经过随机 dropout 后的结果作为一个正样本 V_{k_+}，同时选取数据集中 n 个除自身外的其他样本作为负样本 V_{k_i}（$i = 1, 2, \cdots, n$），所有正负样本均作为查询字典中的键。为了保证正负样本的区别来自于它们自身的特征，而非由不同的编码器引起的差异，在对比学习中使用到的键编码器 f_{v_k} 与查询编码器 f_v 具有相同的结构，二者均为简单的线性层。随后按公式3-8使用 f_{v_k} 对正样本特征进行编码，得到结果 $R_{v_{k_+}} \in \mathbb{R}^d$，其中 $W_{v_k} \in \mathbb{R}^{d \times d_v}$ 和 b_{v_k} 为键编码器的参数。

$$R_{v_{k_+}} = f_{v_k}(V_{k_+}) = W_{v_k} V + b_{v_k} \tag{3-8}$$

在完成正负样本特征编码后，需要定义无监督的对比学习损失来训练图像编码器。对比学习的目标是在尽可能拉进锚点样本与正样本之间的距离同时拉远锚点样本与负样本的距离，故而本方法使用向量点乘来衡量样本之间的相似度，并根据公式3-9采用 InfoNCE[53] 计算对比损失 $loss_{con}$。

$$loss_{con} = -\log \frac{\exp(R_v \cdot R_{v_{k_+}}/\tau)}{\exp(R_v \cdot R_{v_{k_+}}/\tau) + \sum_{i=1}^{n} \exp(R_v \cdot R_{v_{k_i}})/\tau)} \tag{3-9}$$

上式中 τ 为温度系数，$R_{v_{k_i}}$ 是查询字典中存储的上一次计算过程中 f_{v_k} 对第 i 个负样本的编码输出，分母是所有正负样本与锚点样本的相似度之和。

以往端到端方式的对比学习方式通常采用一个批次内的除自身以外的其他样本作为负样本，而由于机器内存有限，批次大小往往比较小（通常不超过 256），导致用于对比的样本数有限，对比学习效果一般。本方法采用一个大小为 n 的队列（一种先进先出的数据结构）来构建查询字典以存储所有负样本的特征，n 可以远大于批次大小，并且独立出来作为一个超参数，使得对比样本数不再受限于批次大小。每批数据依次输入时，将其中存储的所有负样本的特征 $R_{v_{k_i}}$ 用于公式3-9的计算。此外，查询字典的内容也是动态替换更新的，完成每批的计算后，将当前批次的正样本特征 $R_{v_{k+}}$ 入队，作为下一个批次的负样本等待后续使用，同时将队列中最前面的一批数据（即最旧批次的特征向量）出队。这样字典中始终保存的是整个大数据集中采样的一小部分子集，并且保证是每个训练轮次中较新的数据。

如公式3-10所示，由于维护了一个非常大的队列，为了减小计算开销，键的编码器 f_{v_k} 不参与反向传播中的参数更新，因此队列中存储的每个键的特征 $R_{v_{k_i}}$ 都没有梯度。但是为了避免对比样本特征不一致的问题，每个批次完成反向传播后，采用动量更新的方式将完成更新后的查询编码器（即图像特征编码器）f_v 的参数复制到键编码器 f_{v_k} 中。

$$\theta_{f_{v_k}} \leftarrow (1-m)\theta_{f_v} + m\theta_{f_{v_k}} \tag{3-10}$$

其中 $m \in [0,1)$ 是动量系数，$\theta_{f_{v_k}}$ 和 θ_{f_v} 分别为 f_{v_k} 和 f_v 的参数，这意味着每次更新键编码器的参数时，它不会直接取更新后的值，而是兼顾上一次的参数，从而避免每次更新后的特征不一致问题。值得再次强调的是，在整个对比学习过程中只有 f_v 的参数才会参与反向传播，f_{v_k} 则通过动量 m 将 f_v 中的参数平滑缓慢地复制给自身。

(3) 图片单模态分类

在进行图片样本分类时，图片表征 R_v 使用与文本表征 R_t 共享同一个分类器 h，以期望 h 能在图文交替训练的过程中学习到多模态交互信息。R_v 经过线性变换得到图片分类结果 $\hat{y}_v$ 如公式3-11所示。

$$\hat{y}_v = \text{SoftMax}(W_h(\sigma(R_v)) + b_h) \tag{3-11}$$

根据分类结果再按公式3-12计算图片分类的交叉熵损失 $loss_{clf}$。

$$loss_{clf} = -[y\log\hat{y}_v + (1-y)\log(1-\hat{y}_v)] \tag{3-12}$$

最后按公式3-13将对比学习损失与分类损失相加，作为最终的图片模态损失 $loss_v$，从而通过反向传播更新编码器 f_v 和分类器 h 中的参数。

$$loss_v = loss_{clf} + loss_{con} \tag{3-13}$$

3.2.5 带梯度修正的图文共享层

在3.2.3节文本单模态分类和3.2.4节图片单模态分类中，为了避免模态惰性的问题，图片和文本被分开依次交替进行训练，在每一轮训练中图片和文本的特征编码器会各自完成一次反向传播。同时它们二者又共享了同一个分类器 h 来捕获跨模态融合的特征。但是这也带来了一个新的挑战：模态遗忘，即在每一轮训练过程中，分类器 h 学习新的模态特征时会遗忘上一个学习过的模态特征，从而严重影响跨模态学习的效果。因此本算法还引入了梯度修正机制，在每轮迭代学习反向传播计算梯度时，修改分类器参数的梯度值，让它能兼顾上一轮训练中的数据，从而避免遗忘上一个学习过的模态特征。

具体来说，第 n 次反向传播更新参数时，假定当前正在学习的模态为 $m \in \{t, v\}$，$\nabla_h loss_m$ 是分类器 h 的参数 θ_h 关于该模态损失 $loss_m$ 的梯度，γ 是学习率，θ_h 将按公

式3-14更新。

$$\theta_h^n \leftarrow \theta_h^n - \gamma \begin{cases} \nabla_h loss_m & \text{if } n = 0 \\ \mathbf{G}_n \nabla_h loss_m & \text{if } n > 0 \end{cases} \tag{3-14}$$

在初始阶段（即 $n = 0$），θ_h 以常规的梯度下降方式更新；但是在后续每一次反向传播时（即 $n > 0$），梯度修正矩阵 G_n 能够在更新参数前修改对应的梯度值。记第 n 次反向传播时，一个样本批次中所有样本经过对应的模态编码器 f_m 得到的最终表征 R_m^n 的平均值为 $\bar{R}_m^n$，则 G_n 可通过公式3-15和3-16的递归最小二乘法得出。

$$G_n = G_{n-1} - q_n [\bar{R}_m^n]^\top G_{n-1} \tag{3-15}$$

$$q_n = \frac{G_{n-1} \bar{R}_m^n}{[\bar{R}_m^n]^\top G_{n-1} \bar{R}_m^n + \alpha} \tag{3-16}$$

其中 α 是一个常数项，而梯度修正矩阵的初始值 G_0 则是一个单位矩阵。通过这种修改更新参数的梯度的方式，能有效解决在多模态共享学习的过程中模态遗忘的问题，保证每次学习的一致性。

模型训练阶段的单模态交替学习的流程如算法3.1所示。

算法 3.1 训练阶段的单模态交替学习

输入： 待检测的多模态帖子 $P = \{W, I\}$ 以及待训练的模型
输出： 文本特征编码器 f_t 的参数 θ_{f_t}，图片特征编码器 f_v 的参数，图文共享层 h 的参数 θ_h

```
随机初始化上述模型中的参数
while 模型未收敛 do
    begin 文本单模态学习 do
        使用 BERT 与文本特征编码器 f_t 计算文本表征 R_t
        送入 h 得到文本分类结果 ŷ_t，计算文本分类损失：loss_t = −[y log ŷ_t + (1−y) log(1−ŷ_t)]
        以梯度下降的方式更新 θ_{f_t}
        以梯度修正的方式，按公式3-14更新 θ_h
    end
    begin 图片单模态学习 do
        使用 ResNet 与图片特征编码器 f_v 计算图片表征 R_v
        送入 h 得到图片分类结果 ŷ_v，计算图片分类损失：loss_clf = −[y log ŷ_v + (1−y) log(1−ŷ_v)]
        进行对比学习，按公式3-9计算图片 InfoNCE 损失 loss_t
        累加得到最终图片模态损失：loss_v = loss_clf + loss_con
        以梯度下降的方式更新 θ_{f_v}
        以梯度修正的方式，按公式3-14更新 θ_h
        更新对比学习中的动量编码器 f_{v_k} 的参数：θ_{f_{v_k}} ← (1 − m)θ_{f_v} + mθ_{f_{v_k}}
    end
end while
```

3.2.6 推理阶段的多模态动态融合

如前文所述，为了避免模态惰性，在这项工作中图片和文字的特征并未融合，而是分别单独训练。但是虚假信息检测作为自然语言处理领域衍生的任务，图片模态和文本

模态又对最终结果的贡献不同，应该被赋予不同的权重，因此 AlLNet 在推理阶段使用了动态融合机制，根据模态的不确定性来动态调整模态的权重。在图3-2(b) 所示的推理阶段，模型会分别使用训练好的文本、图片编码器 f_t 和 f_v 得到样本的文本表征 R_t 和图片表征 R_v，再经过共享分类头 h 产生各自的预测结果 $\hat{y}_t$ 和 $\hat{y}_v$，然后按公式3-17加权综合作为最终分类结果 $\hat{y}$，其中 λ_v 和 λ_t 分别为图片和文字结果的权重。

$$\hat{y} = \text{SoftMax}(\lambda_v \hat{y}_v + \lambda_t \hat{y}_t) \tag{3-17}$$

对于某个特定的模态 $m \in \{t, v\}$，考虑到当它在推理过程中表现出更高的不确定性，那么它就更容易产生错误的预测，因此它对最终结果的共享权重 λ_m 就应该越小，而每个模态经过分类器 h 的输出的熵 e_m 则能够很好地反应这种不确定性。公式3-18定义了模态分类输出的熵的计算方式，其中 p_i 为样本被分到第 i 个类别的概率（$i = 0, 1$）。

$$e_m = -p_0 \log p_0 - p_1 \log p_1 \tag{3-18}$$

当模态 m 输出的熵越大时，意味着它的预测结果的可信度越低，对应的权重应该越小。公式3-19中用两个模态输出的熵的相反数，经过 SoftMax 操作得出其权重 λ_m。

$$\lambda_m = \text{SoftMax}(-e_m) \tag{3-19}$$

这种基于熵的计算方式能够在推理阶段考虑了文本和图片对于分类结果的贡献，能够兼顾来自图片和文本的信息并将二者进行动态地融合，灵活应对各种具有不同模态特性或者数据分布情况的样本。模型推理阶段的流程在算法3.2中以伪代码形式呈现。

算法 3.2 推理阶段的多模态动态融合

输入： 待检测的多模态帖子 $P = \{W, I\}$ 以及训练完毕的模型
输出： 样本分类预测结果 $\hat{y}$

```
使用 BERT 与文本特征编码器 f_t 计算文本表征 R_t
送入共享分类头 h 得到文本分类结果 ŷ_t
使用 ResNet 与图片特征编码器 f_v 计算图片表征 R_v
送入共享分类头 h 得到图片分类结果 ŷ_v
for 模态 m in {t, v} do
    计算此模态的熵：e_m = −p_0 log p_0 − p_1 log p_1
    计算此模态的权重：λ_m = SoftMax(−e_m)
end for
图文预测结果加权融合作为最终结果：ŷ = SoftMax(λ_v ŷ_v + λ_t ŷ_t)
```

3.2.7 复杂度分析

为了更好地探究本章提出的多模态虚假信息检测算法的计算复杂度，下文将分别对训练阶段和推理阶段的时间复杂度与空间复杂度进行分析，只分析前向传播的计算过程，而不考虑反向传播，并采用 O 表示法来表示分析结果。O 表示法会忽略所有高次幂项的系数、常数项和低次幂项，并且只与输入的数据规模有关，而与模型内部的具体参数量无

关。此外，算法中使用的 BERT、ResNet 等预训练模型由于模型参数和结构已经固定，故不在本文的讨论范围内。已知输入的词向量为 $T = \{token_1, token_2, \cdots, token_l\} \in \mathbb{R}^{l \times d_t}$，其序列长度为 l，词嵌入的维度为 d_t；输入的图片特征为 $V \in \mathbb{R}^{d_v}$；对比学习中使用的负样本个数为 n。

(1) 时间复杂度

首先考虑 TexCNN 结构的 f_t 进行文本特征编码的计算复杂度。当窗口大小为 s 时，每次需要对连续的 s 个词向量进行公式3-2中的卷积操作，其复杂度为 $O(d_t)$，而对文本序列长度为 l 的词向量进行卷积操作的次数为 $l - s + 1$，因此卷积的复杂度为 $O(l \cdot d_t)$。接下来对由卷积得到的 $l - s + 1$ 个局部特征 c_i 进行最大池化操作，其复杂度为 $O(l)$。即便 TextCNN 中使用了 n_s 个不同大小的窗口，每个窗口对应 n_c 个卷积核，由于其时间复杂度与模型内部的参数量无关，故而不应该被计入时间复杂度中。最终二者相加得到文本编码的时间复杂度 T_t。

$$T_t = O(l \cdot d_t) + O(l) = O(l \cdot d_t) \tag{3-20}$$

接下来考虑图片特征编码器 f_v 的计算复杂度。由于它只是一个普通的线性层，只需要对图片特征 V 进行一次矩阵乘法和加法操作，故其时间复杂度 T_v 为 $O(d_v)$。

然后计算对比学习的时间复杂度。由于键编码器 f_{v_k} 与 f_v 结构相同，同理它对正负样本进行特征编码的复杂度为 $O(d_v)$。在按公式3-9计算对比损失时，一对样本进行向量点乘相似度的复杂度为 $O(d)$，锚点样本需要逐个地与 n 个负样本和 1 个正样本进行相似度计算，因此计算对比损失的复杂度为 $O(n \cdot d)$。而查询字典中批量样本特征的出队与入队操作与样本批次大小相关，可将其复杂度视为常数项。f_{v_k} 中的参数量为 $d_v \cdot d$，每次动量更新需要复制其中的参数，时间复杂度为 $O(d_v \cdot d)$。最终累加得到对比学习的时间复杂度 T_{con}。

$$T_{con} = O(d_v) + O(n \cdot d) + O(d_v \cdot d) = O((n + d_v) \cdot d) \tag{3-21}$$

又由3.2.3节文本特征编码可知 $d = n_c \cdot n_s$，同样为模型内部参数，可视作常数项，故对比学习的时间复杂度可再简化为 $O(n + d_v)$。

最后考虑图文共享分类层 h 的时间复杂度。作为一个线性层，它同样只需要对文本 $R_t \in \mathbb{R}^d$ 和图片表征 $R_v \in \mathbb{R}^d$ 分别进行一次矩阵乘法和加法操作，故其时间复杂度 T_h 与 d 相关，同样为常数项 $O(1)$。

综上所述，将文本特征编码、图片特征编码、对比学习、图文共享分类层的时间复杂度相加，可得到整个模型在训练阶段的时间复杂度。由公式3-22可知，该时间复杂度与输入的文本序列长度 l、词向量维度 d_t、图片特征维度 d_v 和负样本个数 n 有关，并且

受 $l \cdot d_t$ 影响更大。

$$\begin{aligned} T_{train} &= T_t + T_v + T_{con} + T_h \\ &= O(l \cdot d_t) + O(d_v) + O(n + d_v) + O(1) \\ &= O(l \cdot d_t + d_v + n) \end{aligned} \tag{3-22}$$

而到了推理阶段，由于无需进行对比学习，其时间复杂度仅仅与输入的文本序列长度 l、词向量维度 d_t、图片特征维度 d_v 相关。

$$T_{infer} = T_t + T_v + T_h = O(l \cdot d_t) + O(d_v) + O(1) = O(l \cdot d_t + d_v) \tag{3-23}$$

(2) 空间复杂度

在计算空间复杂度过程中，本文只考虑模型中的参数量，而不考虑特征计算时的中间变量的空间占用。

对于 TextCNN 结构的 f_t 而言，其中每个卷积核的权重和偏置项的参数量均为 $d_t \cdot s$。与时间复杂度类似，即便 TextCNN 中使用了 n_s 个不同大小的窗口，每个窗口对应 n_c 个卷积核，这些参数仍不应该被计入空间复杂度中，故文本特征编码的空间复杂度 S_t 为 $O(d_t)$。图片特征编码器 f_v 的参数量为 $d \cdot d_v$，其空间复杂度 S_v 为 $O(d_v)$。同样地，对比学习中的键编码器 $f_{v_{k_+}}$ 的空间复杂度也是 $O(d_v)$，而以队列构建的查询字典的空间复杂度为 $O(n)$，最终对比学习的空间复杂度 S_{con} 为 $O(d_v + n)$。图文共享分类层 h 的参数量为 $2 \cdot d$，故其空间复杂度 S_h 为 $O(1)$。

综上所述，AlLNet 在训练阶段的空间复杂度 S_{train} 计算结果如下，与词向量维度 d_t、图片特征维度 d_v 和负样本个数 n 有关，并且呈线性增长。

$$\begin{aligned} S_{train} &= S_t + S_v + S_{con} + S_h \\ &= O(d_t) + O(d_v) + O(d_v + n) + O(1) \\ &= O(d_t + d_v + n) \end{aligned} \tag{3-24}$$

而到了推理阶段，其空间复杂度 S_{infer} 为 $O(d_t + d_v)$。

3.3 实验结果及分析

为了探究 AlLNet 模型的实际分类效果，本节在两个真实世界的数据集上进行了一系列实验，并对实验结果进行了详细的分析。下文首先将介绍实验设置，包括使用到的数据集、基线方法以及超参数等实现细节，然后从下面几个步骤验证 AlLNet 的有效性：

- **对比实验**：将 AlLNet 与其他基线方法在两个数据集上进行对比实验，比较各个方法的分类效果，并分析它们之间产生差异的原因。
- **消融实验**：通过去除 AlLNet 的不同模块，探究在缺失部分模块的情况下，模型的分类效果会如何变化，以此验证设计的各模块的有效性。

- **案例分析**：选取一些典型的虚假信息与真实信息样本作为案例进行具体分析，展示 AlLNet 对这些样本的分类结果，以便更好地说明模型在推理阶段对于文本模态和图片模态的加权综合方式。
- **轻量性比较**：从实际程序运行情况的角度出发，对比 AlLNet 与其他基线方法的模型参数量、推理速度等方面的性能差异，来突出本方法在轻量性方面的优势。
- **模态缺失实验**：分析 AlLNet 在样本缺失图片或文本模态的情况下，模型的推理效果受影响的程度，以此验证模型对模态缺失的鲁棒性。

3.3.1 实验设置

(1) 数据集

为了保证样本的丰富多样性，实验中分别使用中文短文本的数据集 Weibo21[13] 和英文长文本的数据集 GossipCop[55]，它们各自的统计信息如表3-2所示。由于本文主要探讨的是多模态的虚假信息检测算法，因此实验中移除了那些没有图片的纯文本样本，而对于有多张图片的样本则随机选取其中一张。每个数据集都严格按照 7:1:2 的比例随机划分训练集、验证集和测试集。

- Weibo21 是一个近年新发布的中文虚假信息检测数据集，它包含了从微博社区管理中心爬取的从 2014 年 12 月至 2021 年 3 月的短文本微博，覆盖科技、军事、教育、事故、政治、健康、财经、娱乐、社会 9 个不同领域的内容，共有 9128 个样本，其中包括 4640 条真实信息和 4488 条虚假信息。
- GossipCop 的数据都来自于 GossipCop 网站中从各种媒体渠道收集的娱乐故事，并且内容大多数为长文本，共有 22140 个样本，包含 16817 条真实信息和 5323 条虚假信息。

表 3-2　数据集的数据汇总情况

数据集	真实信息数量	虚假信息数量	样本总数
Weibo21	4640	4488	9128
GossipCop	16817	5323	22140

(2) 基线方法

本实验中使用了近年发表在顶级会议或期刊上的五种具有代表性的多模态虚假信息检测算法作为基线方法，它们的简要介绍如下。

- **SpotFake**[15]：作为早期经典的多模态虚假信息检测算法，SpotFake 采用预训练的 VGG 模型来提取图片特征，并使用 BERT 模型代替传统的 Word2Vec 方式来获得词嵌入向量，然后将图片表征与文本表征拼接形成最终的样本多模态表征送入分类器进行虚假信息检测。
- **EANN**[17]：EANN 同样地使用拼接图文特征的方式来获取多模态特征，并在此基础上针对新闻领域信息使用了对抗训练，从而避免训练得到的模型在某个领域的

新闻上过拟合，提升了模型的泛化能力。

- **MCAN**[22]：MCAN 从那些被篡改或伪造的图片样本着手，在预训练 VGG19 模型提取图片像素级的空间域特征的同时，再使用离散余弦变换提取图片频率级的频域特征。然后堆叠多个协同注意力层，不断地对文本特征、空间域特征和频域特征进行多模态融合，最终得到样本的多模态表征以完成分类任务。
- **HMCAN**[23]：HMCAN 为了获得更加丰富的层次化语义特征，不仅仅使用 BERT 最后一层隐含层输出文本特征，而是创新性地利用到了 12 个隐含层的输出信息，将其中相邻的 4 层平均分为 3 组，分别累加每组内的输出形成 3 种不同层次的文本表征。然后使用复杂的上下文 Transformer[48] 将这些文本表征分别与图片表征进行多模态融合，最后拼接融合结果送入分类器。
- **CAFE**[27]：CAFE 不仅仅考虑多模态特征融合，还兼顾了单模态特征中的信息。为了平衡单模态与多模态特征，该模型额外引入了跨模态对齐这一辅助任务，以余弦距离作为该任务的损失函数，使用变分自编码器学习跨模态歧义，然后计算图片模态与文本模态数据分布的 KL 散度作为单模态与多模态特征融合的权重。

(3) 实现细节

对于中英文文本，实验中分别使用了来自 HuggingFace[1]的预训练模型 bert-base-chinese 和 bert-base-uncased 以获得词向量 T，它们隐含层输出的词嵌入维度 d_t 均为 768。同时实验中还使用了 PyTorch 提供的在 ImageNet1K[56] 数据集上预训练的 ResNet50 模型[2]来提取图像特征 V，其输出的图像特征维度 d_v 为 2048。上述所有预训练模型的参数在训练中都保持固定，不参与梯度计算和参数更新。文本特征编码器 TextCNN（即 f_t）中使用的窗口个数 n_s 为 3，窗口大小分别为 3、4、5。Weibo21 和 GossipCop 数据集的文本序列长度 l 分别设置为 170 和 512。

而在 AlLNet 中对比学习模块进行编码实现时，还可以对其进行一定程度的优化，省去部分重复计算。具体来说，每一次进行对比学习时，模型会在当前批次完成对比后，将对应的正样本特征入队，同时将队列最前端的一批负样本特征出队，以维持动态更新。但是实际上在这个过程中，动态字典中那些未出队的负样本特征会在不同批次的对比中被重复使用，如果每次对比时都将负样本特征 V_{k_i} 送入键编码器 f_{v_k} 会导致大量的重复计算，因此在实现过程中可以对这一步改进。优化后的动态字典中存储的不再是原始图像特征 V_{k_i}，而是经过 f_{v_k} 编码后的图像特征 $R_{v_{k_i}}$。在每批样本对比时，f_{v_k} 仅对正样本 V_{k_+} 进行编码得到 $R_{v_{k_+}}$，然后将字典中所有负样本特征 $R_{v_{k_i}}$ 直接送入对比损失计算，完成计算后再将当前批次的正样本特征 $R_{v_{k_+}}$ 入队，作为下一批次的负样本特征使用。这样在每批次对比时，只需要计算一次正样本特征，而负样本特征则直接使用缓存的编码结果，大大减少了计算量，提升了模型的训练速度。

[1]https://huggingface.co/
[2]https://pytorch.org/vision/stable/models/resnet.html

此外，InfoNCE 损失在具体编码实现可借助交叉熵函数来计算，将锚点样本分别与正、负样本的点乘相似度拼接形成一个大矩阵，保证正样本的相似度值在矩阵的第一列上，并且生成一个全 0 的伪标签向量（代表正样本在矩阵中的位置）。这样就可以将其转化为预测正样本位置的多分类问题，从而方便地使用交叉熵函数实现损失计算。代码片段3.1展示了优化后的对比学习伪代码实现，为了与普通英文字母和数学符号区分，下文对于这些程序代码变量统一使用Consolas等宽字体书写。x_q和x_k分别代表锚点样本 V 和正样本 V_{k_+} 的基本特征，f_q和f_k分别代表查询编码器 f_v（即图像特征编码器）和键编码器 f_{v_k}，而b、d、n分别代表批次大小、特征维度和队列大小（即负样本特征数），queue代表动态字典中存储的所有负样本特征，bmm代表批量矩阵乘法，mm代表矩阵乘法。

代码片段 3.1　优化后的对比学习伪代码实现

```
q = f_q.forward(x_q)  # 计算查询/锚点样本的特征: b*d
k = f_k.forward(x_k)  # 计算键/正样本的特征: b*d
k = k.detach()  # 键特征不参与梯度计算
l_pos = bmm(q.view(b,1,d), k.view(b,d,1)) # 锚点样本与正样本的相似度: b*1
l_neg = mm(q.view(b,d), queue.view(d,n)) # 锚点样本与负样本的相似度: b*n
logits = concat([l_pos, l_neg], dim=1) # 拼接正负样本相似度
labels = zeros(b)  # 全0伪标签
loss = CrossEntropyLoss(logits/t, labels) # 通过交叉熵计算InfoNCE损失
loss.backward() # 反向传播
update(f_q.params) # 更新查询编码器的参数
f_k.params = m * f_k.params + (1 - m) * f_q.params # 动量更新键编码器
enqueue(queue, k)  # 将当前批次的正样本特征入队
dequeue(queue)  # 将最早一批的负样本出队
```

模型训练时采用 Adam 优化算法作为优化器。实验中使用的服务器内存为 90GB，CPU 为 12 核的 Xeon(R) Platinum 8352V，GPU 则是一张 Nvidia GeForce RTX 4090(24GB)。

3.3.2 对比实验结果及分析

表3-3展示了本文提出的 AlLNet 方法与其他基线方法在 Weibo21 和 GossipCop 这两个真实数据集上的实验结果，采用准确率 accuracy、精确率 precision、召回率 recall 和 f1 分数作为评估指标，并分别对虚假信息和真实信息这两个样本类别进行了评估，每项指标的最优结果均采用加粗标出。

不难发现，所有方法在 GossipCop 数据集的虚假信息这一类别上都表现不佳，这是因为这个数据集的样本数量非常多，文本序列长度也较长，样本特征复杂，学习起来较为困难，并且还存在类别不平衡的情况，导致模型容易在某一类别的样本上过拟合，这也符合真实世界场景。而 Weibo21 数据集由于样本数量不多，文本序列长度较短，样本特征较简单，因此各个方法在这个数据集上的表现都相对较好。

在所有的基线方法中，SpotFake 的效果最差，在两个数据集上的准确率居所有方法最低，在 GossipCop 数据集上的虚假信息召回率甚至只有 0.274。这是因为它只采用了最简单方式来拼接图文，无法充分学习多模态特征。

而同样使用拼接方法的 EANN，由于使用了事件对抗训练，一定程度上提升了模型

表 3-3　本文方法 AlLNet 与其他基线方法的对比实验结果

数据集	模型	accuracy	虚假信息			真实信息		
			precision	recall	f1 score	precision	recall	f1 score
Weibo21	SpotFake	0.857	0.873	0.825	0.847	0.835	0.883	0.854
	EANN	0.870	0.902	0.825	0.862	0.841	0.912	0.875
	MCAN	0.906	0.872	0.935	0.904	0.939	0.886	0.907
	HMCAN	0.896	0.861	0.934	0.896	**0.943**	0.862	0.906
	CAFE	0.908	0.885	0.932	0.909	0.933	0.883	0.907
	AlLNet	**0.933**	**0.922**	**0.939**	**0.930**	**0.943**	**0.927**	**0.935**
GossipCop	SpotFake	0.819	0.686	0.274	0.397	0.821	0.962	0.883
	EANN	0.864	0.702	0.518	0.594	0.887	0.956	0.920
	MCAN	0.833	0.646	0.505	0.561	0.876	0.920	0.907
	HMCAN	0.851	0.818	0.428	0.562	0.855	0.972	0.910
	CAFE	0.859	**0.859**	0.468	0.606	0.859	**0.977**	0.914
	AlLNet	**0.884**	0.848	**0.618**	**0.714**	**0.892**	0.966	**0.928**

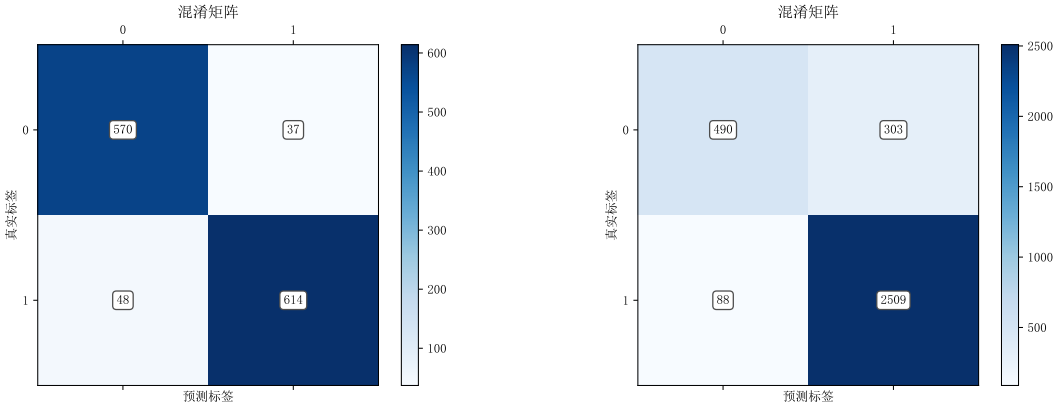
的泛化能力。它在样本类别不均衡的 GossipCop 数据集上取得了不错的效果，在虚假信息上的召回率达到了 0.518，而在真实信息的精确率上达到了 0.887，在除本文 AlLNet 外的所有基线方法中均达到了单项指标的最优。

MCAN、HMCAN 和 CAFE 都采用注意力机制学习到了更丰富的图文多模态特征，均取得了不错的效果。MCAN 考虑到虚假信息中存在被篡改或伪造的图片，从频域角度出发，利用离散余弦变换提取图片特征，因此在 Weibo21 虚假信息的指标上达到了 0.935，在除本文方法外的所有基线模型中最高。而 HMCAN 它通过聚合相邻的 BERT 文本编码器的信息，学习到了层次化的文本语义特征，在 Weibo21 真实信息的指标上达到了 0.943，与文本方法并列单项第一。CAFE 使用 KL 散度平衡单模态特征和多模态特征，兼顾单模态信息与多模态信息对虚假信息检测的贡献，从多项指标的综合结果来看，它较其他基线方法在 Weibo21 和 GossipCop 数据集上均取得了不错的效果，在 GossipCop 虚假信息的指标和真实信息的指标上分别达到了 0.859 和 0.977，是所有方法中最高的。

但是上述这些基线模型在训练过程中都无法避免模态惰性的问题，还是无法充分学习图片和文本的特征。而 AlLNet 采用图文分别交替训练的方式，避免在训练过程中进行多模态特征加权融合。在每一次反向传播中，模型学习的只是单一模态的特征，该模态未被加权，反向传播的梯度值并未减小，对应的特征编码器参数更新过程也未被延缓，而是正常执行，因此模型在训练过程中不会出现模态惰性的问题，在一定程度上提升了模型的分类效果。在所有方法的结果中，AlLNet 在 Weibo21 和 GossipCop 上均达到了最高的准确率，分别为 0.933 和 0.884，在 precision、recall 和 f1 score 这三个指标上也取得了不错的结果。除 GossipCop 虚假信息的 precision 指标和真实信息的 recall 指标外，AlLNet 在其他指标上均取得了最优结果，这充分证明了它的优越性。与此同时，

AlLNet 在 GossipCop 数据集上的虚假信息指标和 f1 分数仍然只有 0.618 和 0.714，这代表它仍然受 GossipCop 的类别不均衡问题困扰。这是数据集本身带来的偏差影响，是不可避免的问题。

为了更直观地展示 AlLNet 在 Weibo21 和 GossipCop 数据集上的分类结果，下文将从混淆矩阵、ROC 曲线和 PR 曲线三个方面对实验结果进行分析。

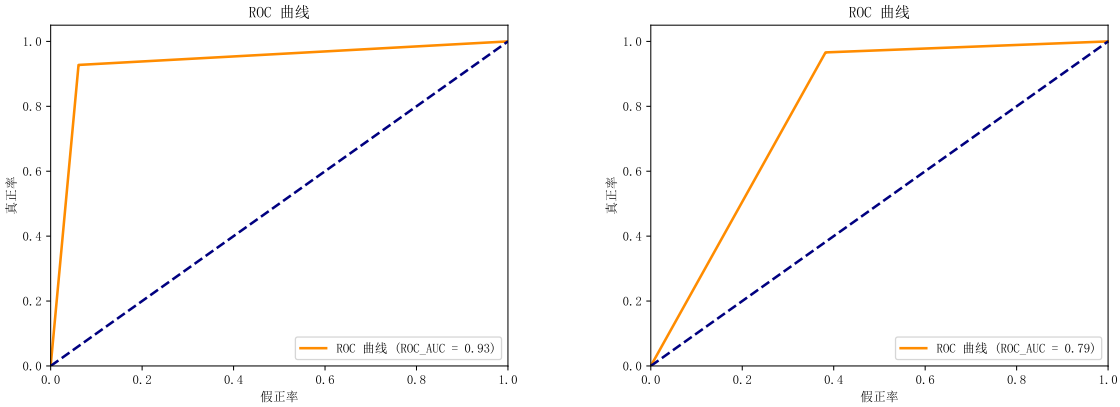


(a) Weibo21 数据集　　(b) GossipCop 数据集

图 3-5　混淆矩阵

图3-5展示 AlLNet 在两个数据集上分类结果的混淆矩阵。混淆矩阵是一种常见的分类算法性能评估工具，它以表格形式展示了分类模型的预测结果和真实标签之间的关系，包括真正例（True Positive, TP）、假正例（False Positive, FP）、真负例（True Negative, TN）和假负例（False Negative, FN）的数量。从图中可以看出，AlLNet 在 Weibo21 测试集中正确识别了 570 个虚假信息样本（样本标签为 0）和 614 个真实信息样本（样本标签为 1），同时错误地将 37 个虚假信息样本识别为真实信息样本（假正例），将 48 个真实信息样本识别为虚假信息样本（假负例）。在 GossipCop 测试集分类结果中假正例的数量为 303，而假负例的数量仅为 88，在四个类别中假负例的数量最少，这代表 AlLNet 倾向于将该数据集中的样本分类为真实信息。但同时它也正确的识别了 2509 个真实信息样本，因此该算法在此数据集上还是取得了较高的准确率。

图3-6则展示了分类结果的 ROC 曲线，它是一种展示二分类器性能的图表，它通过绘制真正例率（TPR）与假正例率（FPR）来展示模型在不同阈值下的表现，其中横坐标为假正例率（FPR），纵坐标为真正例率（TPR），对角线（即基准线）上的点表示随机猜测的结果，而曲线越靠近左上角，说明模型的效果越好。从图中可以看出，Weibo21 数据集上的 ROC 曲线几乎完全覆盖了整个图像，并且在假正例率较低的情况下，真正例率已经较高，同时曲线下方面积 AUC（Area Under Curve）也达到了 0.93，这代表模型在该数据集上的分类效果较好。由于 ROC 曲线对类别不平衡不敏感，AlLNet 在 GossipCop 数据集上的 ROC 曲线同样也在远在对角线上方，但是在 FPR 较低时，TPR 的增长较慢，直到 FPR 增长到 0.4 以后，TPR 才开始达到较高的值，同时 AUC 也只有

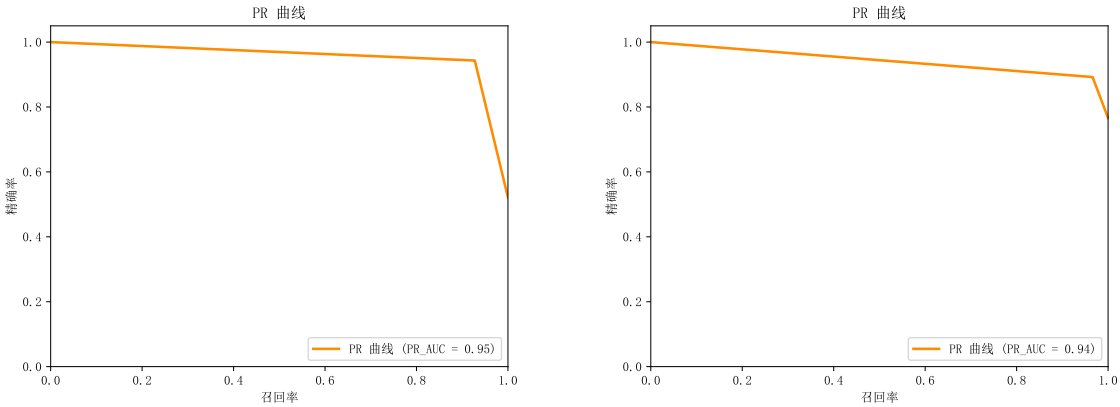


(a) Weibo21 数据集　　(b) GossipCop 数据集

图 3-6　ROC 曲线

0.79，这代表虽然模型也能正确分类样本，但是误报率较高。

PR 曲线是另一种展示二分类器性能的图表，它关注的是模型在不同阈值下的精确率（precision）和召回率（recall），更能体现类别不平衡问题下的分类效果。从图3-7中可以看出，虽然模型在 GossipCop 数据集上分类效果不如在 Weibo21 数据集上的效果，但是二者的 PR 曲线下方面积分别为 0.95 和 0.94，均取到了较高的值。这表明本文模型 AlLNet 在保持高召回率的同时，也能保持高精确率，是一个较为理想的分类模型。



(a) Weibo21 数据集　　(b) GossipCop 数据集

图 3-7　PR 曲线

3.3.3 消融实验结果及分析

为了验证 AlLNet 中各个关键模块的作用，实验中还额外设计了下列 AlLNet 模型的不同变种来进行消融实验。

- AlLNet w/o T：去除文本模态学习模块，只使用图片模态进行分类训练并完成推理。
- AlLNet w/o I：去除图片模态学习模块，只使用文本模态进行分类训练并完成推理。
- AlLNet w/o C：去除图片模态学习中的对比学习模块进行训练。

- AlLNet w/o S：图文交替训练中不使用共享的分类头 h，而是各自单独使用一个线性层结构的分类器输出单模态预测结果。
- AlLNet w/o G：图文共享分类头中不进行梯度修正，只采用普通的梯度下降算法进行参数更新。
- AlLNet w/o E：预测推理时，不使用熵作为权重的动态融合机制，而是直接按照 $\hat{y} = \hat{y}_v + \hat{y}_t$ 的方式将图片和文本输出的分类结果相加作为最终结果。

表 3-4　AlLNet 不同变种的消融实验结果

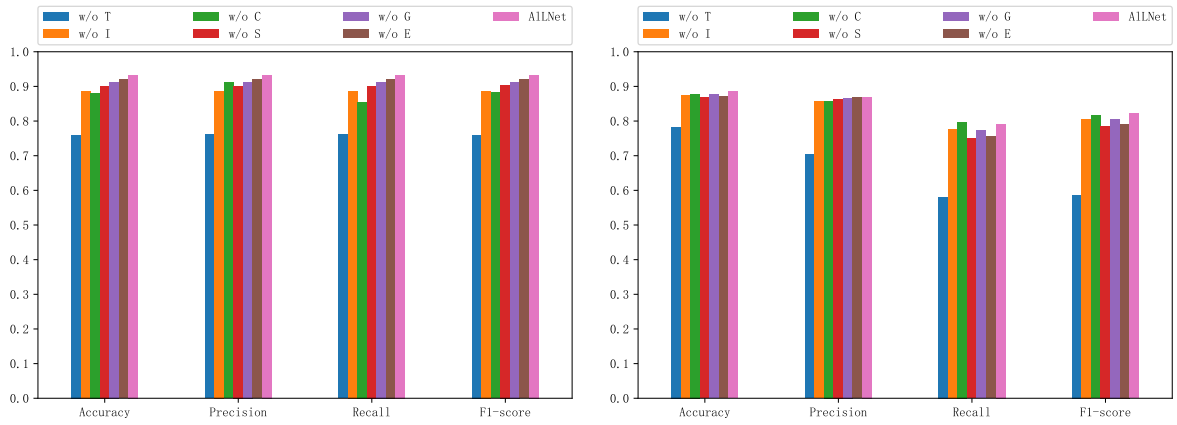
数据集	模型	accuracy	虚假信息			真实信息		
			precision	recall	f1 score	precision	recall	f1 score
Weibo21	AlLNet w/o T	0.760	0.737	0.775	0.756	0.784	0.746	0.764
	AlLNet w/o I	0.887	0.875	0.891	0.883	0.898	0.883	0.891
	AlLNet w/o C	0.881	0.911	0.854	0.882	0.903	0.826	0.897
	AlLNet w/o S	0.908	0.890	0.907	0.898	0.910	0.895	0.902
	AlLNet w/o G	0.913	0.907	0.912	0.910	0.917	0.913	0.915
	AlLNet w/o E	0.920	0.908	0.927	0.917	0.932	0.913	0.923
	AlLNet	**0.933**	**0.922**	**0.939**	**0.930**	**0.943**	**0.927**	**0.935**
GossipCop	AlLNet w/o T	0.782	0.609	0.200	0.301	0.797	0.960	0.871
	AlLNet w/o I	0.875	0.827	0.591	0.689	0.856	0.776	0.805
	AlLNet w/o C	0.878	0.834	**0.698**	**0.791**	0.891	0.956	0.923
	AlLNet w/o S	0.868	0.857	0.524	0.651	0.870	0.963	0.918
	AlLNet w/o G	0.877	0.852	0.575	0.686	0.882	0.875	0.923
	AlLNet w/o E	0.871	**0.862**	0.535	0.661	0.873	0.963	0.920
	AlLNet	**0.884**	0.848	0.618	0.714	**0.892**	**0.966**	**0.928**

表3-4展示了这些变种在 Weibo21 和 GossipCop 数据集上的分类效果。不难看出，在各自去掉不同的关键模块后，这些变种的效果都不如 AlLNet 好。去除文本模块的 AlLNet w/o T 和去除图片模块的 AlLNet w/o I 在两个数据集上的效果均出现了大幅度下降，尤其是 AlLNet w/o T 的准确率甚至不足 80%，这说明文本信息对于多模态虚假信息检测任务更重要。去掉对比学习模块的 AlLNet w/o C 在 Weibo21 数据集上的准确率下降了约 5 个百分点，同时在 GossipCop 数据集上的准确率下降了约 1 个百分点，这证明了对比学习模块在学习图片特征时的重要性。但是 AlLNet w/o C 在 GossipCop 数据集上的虚假信息召回率和 f1 分数却分别达到了 0.698 和 0.791，位列所有方法之首，这表明了对比学习模块在 GossipCop 数据集上的一定局限性，可能是因为 GossipCop 数据集中的图片数据较为简单且类别不平衡，对比学习模块的作用不明显。

去掉共享头后，模型的准确率也下降了约 2 个百分点，这是因为它丢失了重要的图文跨模态交互的信息，而单模态特征能提供的信息相当有限。同时共享头中的梯度修正机制在解决模态遗忘这一问题上也起到了很大作用，没有梯度修正的 AlLNet w/o G 在 Weibo21 和 GossipCop 数据集上的准确率分别只有 0.913 和 0.877，这表明 AlLNet w/o G

在学习过程中会遗忘掉一部分重要的信息。在推理阶段，AlLNet 以每个模态输出的熵作为模态的权重对图文进行动态融合。而 AlLNet w/o E 直接将图文各自的输出相加，简单地将二者完全一视同仁，忽略了图片和文本对于虚假信息检测任务的贡献不同，因此在 Weibo21 和 GossipCop 数据集上的准确率分别只有 0.920 和 0.871，其余三项指标也出现了轻微下降。

图3-8则更直观地展示了这些变种在 Weibo21 和 GossipCop 数据集上的分类结果。这幅图更明显突出了去除文本模态学习模块的 AlLNet w/o T 在各项指标上的表现最差，而其他的变种在各项指标上的表现也都不如 AlLNet，只有在 GossipCop 数据集上的召回率和 f1 分数上 AlLNet w/o C 取得了较好的效果。



(a) Weibo21 数据集　　(b) GossipCop 数据集

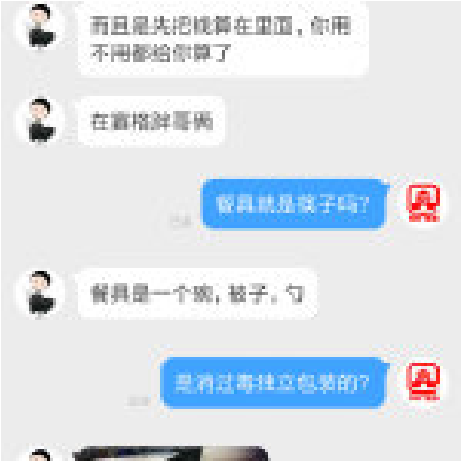
图 3-8　消融实验的分类结果柱状图

3.3.4 案例分析

为了更好地展示 AlLNet 在多模态虚假信息检测任务中的效果，探讨模型在推理阶段对于图片模态和文本模态的预测结果的加权综合情况，下文将从 Weibo21 数据集中选取一些真实信息样本和虚假信息样本进行案例分析。

图3-9给出了两条真实信息样本的案例，包括它们的文本内容、图片内容以及 AlLNet 的预测结果。第一条真实信息样本中的文本内容是关于《食品卫生法》中对于餐饮企业提供餐具的相关规定，图片内容则是商家与顾客关于是否提供餐具而发生争执的对话截图。对于该样本，AlLNet 文本模态预测分支经过 SoftMax 操作后，得出样本被分到虚假信息（$y=0$）和真实信息（$y=1$）两个类别的概率分布为 $[0.2389, 0.7611]$，而图片模态预测分支的分类概率分布为 $[0.4237, 0.5763]$，这代表这两个模态都倾向于将该样本分类为真实信息（即标签为 1）。但不同的是，文本模态将其分类为真实信息的概率（0.7611）比图片模态的概率（0.5763）更高。换言之，文本模态将样本判定为真实信息的置信度更高，更加地坚定确信，而图片模态在进行预测时则稍显犹豫，这也符合人类在判断真实信息时更多地依赖文本信息的认知特点。根据文本和图片模态的各自预测结果，按公

微博原文：@胖哥俩肉蟹煲 我国《食品卫生法》第八条“餐具、饮具和盛放直接入口食品的容器，使用前必须洗净、消毒，炊具、用具用后必须洗净，保持清洁”所以，向消费者提供合格餐具是餐饮企业的义务，配套服务项目，也是经营者法定义务和附随义务，不能因为餐具密封包装，就将消毒产生的费用转嫁到消费者身上



分析结果：

$\hat{y}_t = [0.2389, 0.7611]$　$\hat{y}_v = [0.4237, 0.5763]$

$e_t = 0.54981$　$e_v = 0.6815$

$\lambda_t = 0.5329$　$\lambda_v = 0.4671$

$\hat{y} = [0.3184, 0.6816]$

(a)

微博原文：#微博辟谣#近日网传一则视频称，江苏四名妇女轮流掌掴一名小保姆，小保姆来自农村，被打原因是给老人喂饭时老人嫌烫。经查证，视频所反映事件于今年3月发生在江苏盐城市，被打者系一名未成年少女，打人者为被打者的前男友的现女友（有点绕），对该事件的具体处理结果可咨询盐城警方。

分析结果：

$\hat{y}_t = [0.000001, 0.999999]$　$\hat{y}_v = [0.3119, 0.6881]$

$e_t = 0.000016$　$e_v = 0.6206$

$\lambda_t = 0.6503$　$\lambda_v = 0.3497$

$\hat{y} = [0.0001, 0.9999]$

(b)

图 3-9　Weibo21 数据集中的真实信息案例

式3-18可以得出二者的输出的熵 e_t 和 e_v 分别为 0.54981 和 0.6815。如3.2.6节所述，熵反映了信息的无序度，而一个模态的熵越大，代表其对于该样本的预测结果越不确定，越容易产生错误的预测。因此 AlLNet 在推理阶段会将文本模态的预测结果更多地考虑在内，根据公式3-19计算模态权重 λ_t 和 λ_v 分别为 0.5329 和 0.4671，这表明文本模态对于该样本的最终分类的影响（或者说贡献度）更大。最终按公式3-17得出该样本的分类概率分布为 $\hat{y} = [0.3184, 0.6816]$，即 AlLNet 将其判定为真实信息，与该样本的真实标签一致，分类正确。

图3-9(b) 则展示了第二条真实信息样本，其文本内容是江苏网警对于一则关于“江苏四名妇女轮流掌掴一名小保姆”网络谣言的辟谣通报，而图片内容则是相关视频的截图。该案例反映了一种更为极端的预测情况，文本模态的预测概率分布为 $[0.000001, 0.999999]$，这表明文本模态对于该样本的分类结果非常确定，几乎可以肯定其为真实信息，故而文本模态的熵 e_t 为 0.000016，对应的权重 λ_t 为 0.6503，几乎是图片模态权重的两倍。模型最终的预测结果也变得极端，$\hat{y} = [0.0001, 0.9999]$，即 AlLNet 将其判定为真实信息，分类正确。

图3-10给出了两条虚假信息样本的案例，其中第一个案例展示了当图片和文本模态预测产生分歧，二者的分类结果相互矛盾时，AlLNet 如何进行综合预测。该样本的微博原文是关于“民众上街维权”的内容，而图片内容则是一张人群聚集的照片，以此来混淆视听，塑造游行示威的假象。不难发现，文本模态的预测概率分布为 $[0.45667, 0.54333]$，意味着文本模态将其分类为真实信息。但同时该样本分为虚假信息与真实信息的概率差异并不大，因此文本模态的熵 e_t 为 0.68939，表明文本模态对于该样本的预测结果非常不确定，十分容易预测错误。而事实也确实如此，该样本实际上其实是虚假信息。与此同时，图片模态的预测概率分布为 $[0.9795, 0.0205]$，对应的熵为 0.0998。这意味着图片模态的置信度非常高，几乎可以肯定地将其分类为虚假信息。因此文本和图片的权重分别为 0.3567 和 0.6433，最终加权得到的结果为 $\hat{y} = [0.91883, 0.08117]$，模型将其识别为虚假信息，分类正确。这个案例很好地展示了 AlLNet 对多模态预测结果进行加权综合的推理过程，如何在图文模态预测结果相互矛盾时，通过熵和权重的计算来进行最终的分类决策。当文本模态在分类时表现得犹豫不定且将样本分类错误时，图片模态的高置信度可以避免模型产生错误的预测，将最终分类结果拉回正确方向。

微博原文：河南获嘉县官与抗议维权民众对话 警察持枪撑伞护卫 据网友15日发帖，河南获嘉县群众抗议污染企业，民众上街维权，警察为保护出来与民众对话领导，一手执枪，一手为领导撑着伞。

分析结果：

$\hat{y}_t = [0.45667, 0.54333]$ $\hat{y}_v = [0.9795, 0.0205]$

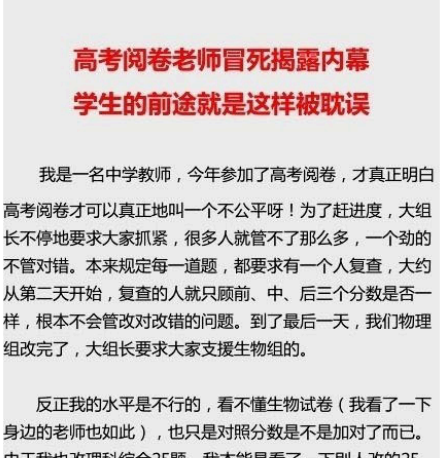
$e_t = 0.68939$ $e_v = 0.0998$

$\lambda_t = 0.3567$ $\lambda_v = 0.6433$

$\hat{y} = [0.91883, 0.08117]$

(a)

微博原文：#直通车爆料#【高考阅卷老师冒死揭露内幕】一位参加过高考阅卷的教师，为我们揭露了阅卷中不为人知的内幕真相！让我们惊讶于一个如此重要的、决定一个学生前途命运的考试，其分数竟然是在这样难以想象的粗放流程中产生的！！！我们想问一句，到底有多少学生，是这样被耽误的？



分析结果：

$\hat{y}_t = [0.99988, 0.00012]$ $\hat{y}_v = [0.973, 0.027]$

$e_t = 0.001178$ $e_v = 0.1240$

$\lambda_t = 0.5307$ $\lambda_v = 0.4693$

$\hat{y} = [0.99848, 0.00152]$

(b)

图 3-10 Weibo21 数据集中的虚假信息案例

图3-10(b) 则展示了第二条虚假信息样本的分类情况。在这个案例中，图片模态和文

本模态在预测时达成了高度一致。文本模态的分类概率分布为 $[0.99988, 0.00012]$，对应的熵为 0.001178，而图片模态的分类概率分布为 $[0.973, 0.027]$，对应的熵为 0.1240。二者都十分确信地将该样本分类为虚假信息，而分类为真实信息的概率几乎为 0。因此文本模态权重 λ_t 和图片模态权重 λ_v 相差无几，分别为 0.5307 和 0.4693。最终模型也成功地将该样本判定为虚假信息，分类正确。

图 3-11　Weibo21 数据集中的模型分类错误案例

此外，为了更好地探究 AlLNet 进行推理预测时的局限性，本文还对 AlLNet 分类错误的样本进行了分析。如图3-11所示，Weibo21 数据集中有两条真实信息被误判为虚假信息。第一个样本的文本模态的分类概率分布为 $[0.03962, 0.96037]$，对应的熵为 0.16676，权重为 0.4779，分类正确；图片模态的分类概率分布为 $[0.9849, 0.0151]$，对应的熵为 0.0783，权重为 0.5221，分类错误；最终二者加权的结果为 $\hat{y} = [0.65879, 0.34121]$，模型分类错误。第二个样本的文本模态的分类概率分布为 $[0.74551, 0.25449]$，对应的熵为 0.56722，权重为 0.5311，分类错误；图片模态的分类概率分布为 $[0.4728, 0.5272]$，对应的熵为 0.6917，权重为 0.4689，分类正确；最终二者加权的结果为 $\hat{y} = [0.65879, 0.34121]$，模型分类错误。

不难发现，这两个样本的共同特点都是图文两个模态的预测结果互相矛盾，其中一者分类正确，而另一者分类错误。并且两个模态的权重也较为接近，这就导致了 AlLNet 会同等地采纳它们的预测结果，并受到其中某个预测错误的模态的干扰，最终做出了错

误的决策。这表明 AlLNet 在处理图文模态结果有分歧且二者权重接近的情况时，仍然存在一定的不足，有待进一步改进。

3.3.5 模型轻量性比较

除去准确率等常用分类指标，模型的轻量性也是评估算法时需要考虑的重要因素。尤其是在真实场景中，计算资源往往有限，并且希望能得到快速响应。这就要求部署的模型不能过于庞大，在不大幅度地牺牲准确率等效果的前提下，参数量应尽可能少，推理速度尽可能快，不占用过多的内存、CPU、GPU 等计算资源。本章在3.2.7节中已对 AlLNet 模型的复杂度进行了理论分析，下文的实验将从参数数量和推理速度两个方面来比较 AlLNet 以及各基线方法的轻量性。

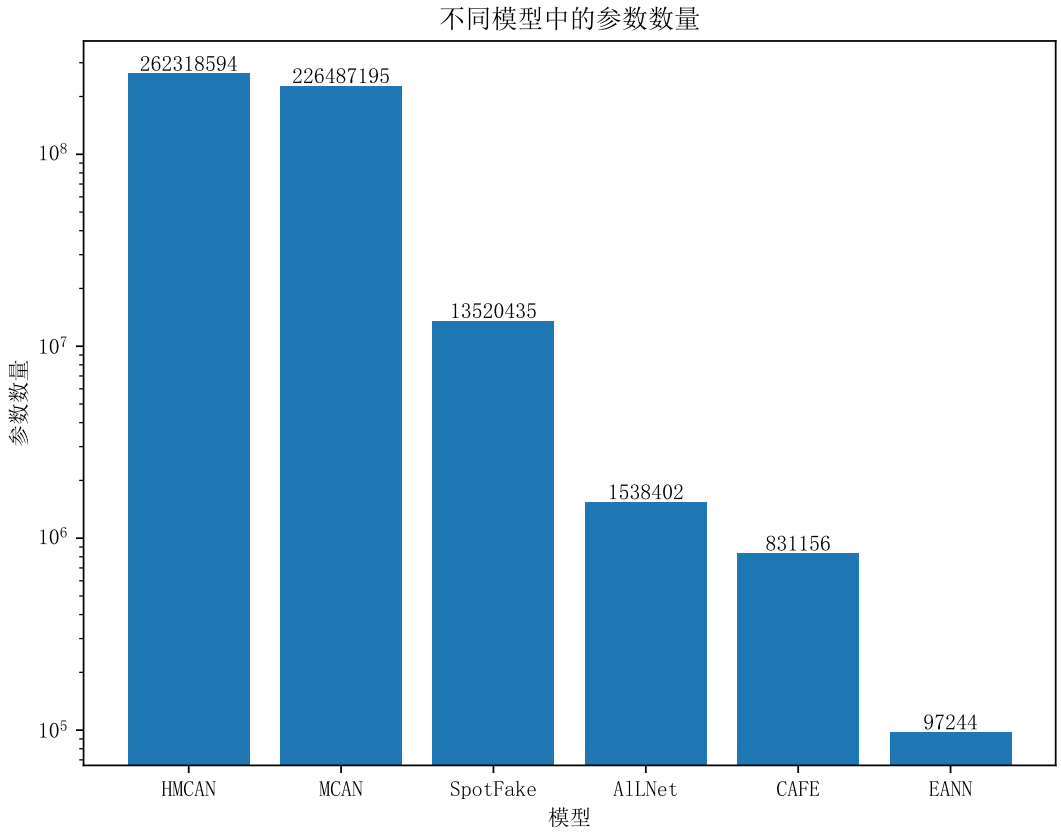


图 3-12　不同模型中的参数数量

图3-12展示了所有模型中包含的可训练参数的数量，其中 y 轴使用了对数坐标，各方法中使用的 BERT、Word2Vec、VGG、ResNet 等用于模态编码的预训练模型参数并未计入其中。HMCAN 和 MCAN 中包含了多个复杂的注意力机制，参数量在所有方法中分居第一、二名。而包含了多个多层感知机的 SpotFake 和只在最后阶段进行注意力多模态融合的 CAFE 模型的参数量则相对较小。EANN 中的 TextCNN 的卷积核数量以及窗口数较小，因此参数量最少。而本文的 AlLNet 模型中待训练的模块仅为文本特征编码器 f_t（TextCNN 结构）、图片特征编码器 f_v（线性结构）以及共享的分类头 h（线性

结构），参数量为 1538402，在所有模型中居倒数第二。

除去模型参数的统计分析外，本章实验中还比较了这些方法在真实世界数据集上的计算速度。值得再次强调的是，下列 benchmark 实验中的所有模型均在相同的硬件环境下进行运算，以保证实验结果的公平性。图3-13展示了所有方法在 Weibo21 的测试集上完成推理预测所需的运算时间。HMCAN 由于使用了多个复杂的 transformer 模块提取单模态和多模态特征，计算所需时间最长。MCAN 提取三种模态的特征，并堆叠多个注意力机制完成多模态融合，推理耗时也比较大。而只使用普通的注意力机制的 CAFE 模型的耗时则次之。SpotFake 和 EANN 都只使用最简单的拼接方式，计算耗时有所降低，均在 13 秒左右。而本文提出的 AlLNet 模型对于图文特征只使用最简单的 TextCNN 和线性变换来提取，计算复杂度非常低，同时也并未进行复杂的多模态融合，而是直接用熵来加权求和，因此耗时最短，只有 11 秒。

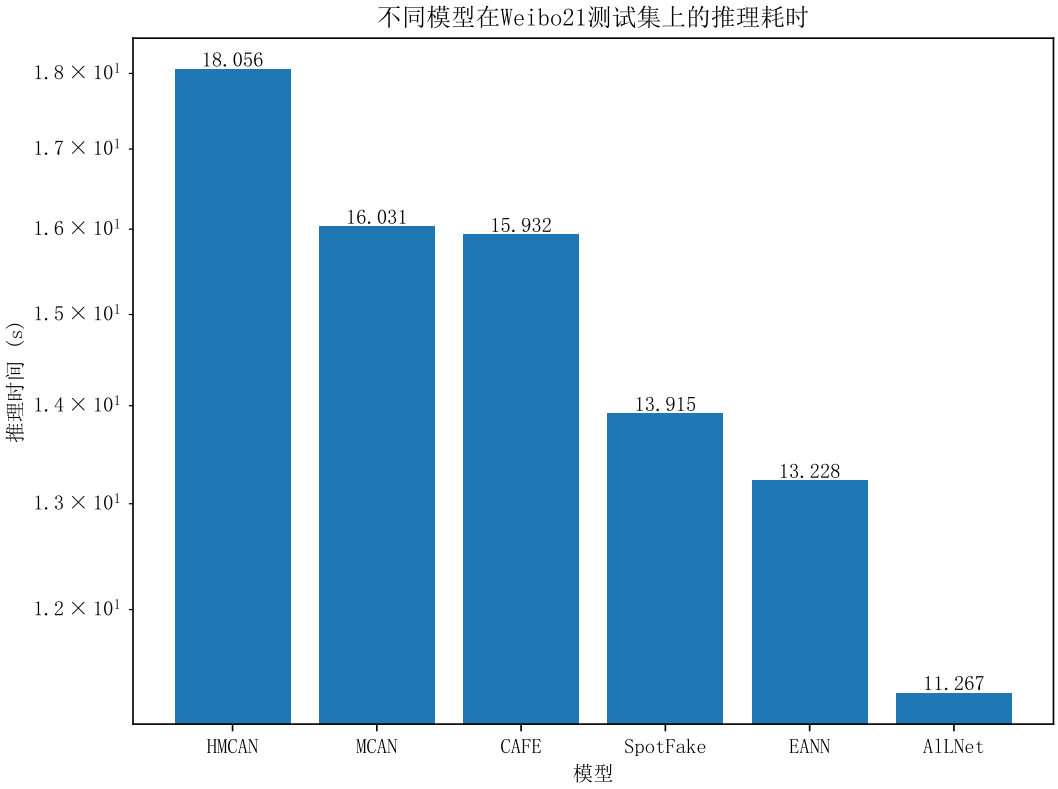


图 3-13　不同模型在 Weibo21 测试集上的推理耗时

3.3.6 模态缺失情况下的推理结果及分析

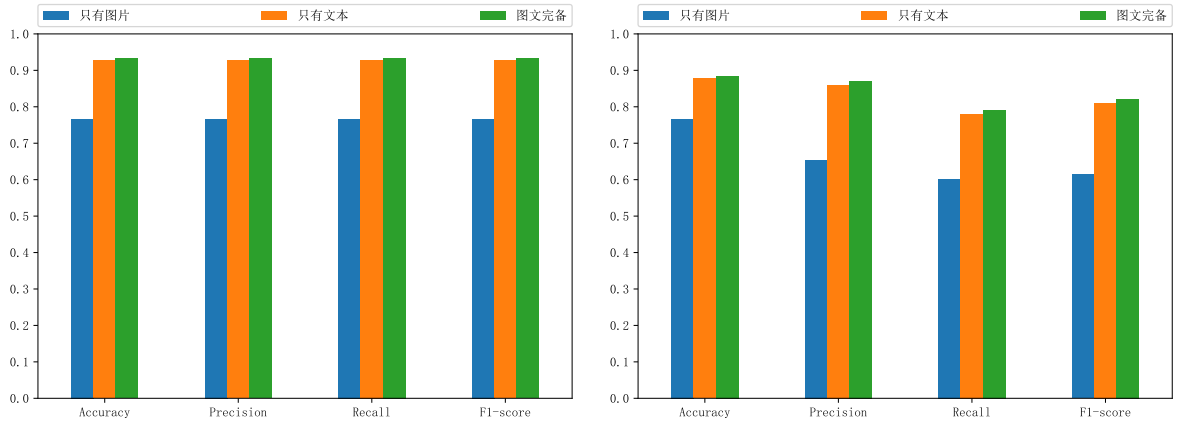
除去模型性能以及轻量性，多模态虚假信息检测算法在真实世界场景中还面临着一个重要挑战：模态缺失。通常在离线训练中使用到的样本均包含了一段文本和一张图片，但是在真实世界进行推理的过程中，模型会经常碰到只有文本或者只有图片的样本。传统的多模态虚假信息检测算法需要图文完备的样本作为模型输入，不适用于模态缺失的场景。但是本文提出的 AlLNet 模型在推理时，图片预测和文本预测是两条彼此

独立的分支，二者的计算过程互不干扰，然后使用熵来衡量二者的权重，将图文的输出进行动态加权融合作为最终结果。因此在缺少图片或者文本时，AlLNet 可以不计算这个模态，直接取另一个模态的计算结果作为模型的最终输出。

表 3-5　模态缺失时的推理结果

数据集	模态	accuracy	虚假信息			真实信息		
			precision	recall	f1 score	precision	recall	f1 score
Weibo21	只用图片	0.766	0.748	0.772	0.760	0.785	0.761	0.773
	只用文本	0.927	0.912	**0.939**	0.925	0.942	0.916	0.929
	图文完备	**0.933**	**0.922**	**0.939**	**0.930**	**0.943**	**0.927**	**0.935**
GossipCop	只用图片	0.766	0.500	0.295	0.371	0.808	0.909	0.856
	只用文本	0.865	**0.933**	0.487	0.616	0.830	0.963	0.913
	图文完备	**0.884**	0.848	**0.618**	**0.714**	**0.892**	**0.966**	**0.928**

表3-5展示了在图文完备的多模态训练集上完成训练后的 AlLNet 模型，在测试集中模态缺失样本上进行推理的结果。不难发现，在缺少图片或者文本模态时，AlLNet 依然能正常识别那些待分类样本。尤其是只有文本模态时，AlLNet 在 Weibo21 和 GossipCop 数据集上依然达到了 0.927 和 0.865 的准确率，比图文完备的正常情况只下降了 1 到 2 个百分点，精确率、召回率和 f1 分数这三项指标也都保持在较高水平。这说明文本中可用于判别的特征非常丰富，对于虚假信息检测任务的贡献非常大，即便在只有文本信息的情况下，模型也能取得不错的分类效果。而只有图片模态时，AlLNet 在两个数据集上的准确率均仅为 76%，相较于正常情况低了许多。尤其是在 GossipCop 数据集中虚假信息类别的样本上，precision、recall 和 f1 score 均出现了暴跌。这反过来也再次证明了，在多模态虚假信息检测任务中，图文不同模态对于最终结果的贡献不同，不应该对它们不加区分地简单一体化处理。



(a) Weibo21 数据集　　(b) GossipCop 数据集

图 3-14　缺失不同模态时的推理结果柱状图

图3-14则以柱状图的方式更直观地展示了 AlLNet 在 Weibo21 数据集和 GossipCop

数据集上缺失不同模态时的推理结果。相较于表格数据，这张图更能明显地展示出在只有文本模态时，AlLNet 的分类效果依然较好，与图文完备情况下的结果相差不大。而只有图片模态时，AlLNet 的分类效果则出现了明显的下滑，尤其是 precision、recall 和 f1 score 这三项指标均出现了较大幅度的降低。

3.4 本章小结

本章提出了一种基于单模态交替的多模态虚假信息检测算法 AlLNet，该算法在训练阶段使图片和文本特征分开交替训练，而推理时则以熵衡量不同模态的比重，加权融合得到最终结果。本章还设计了一系列实验，充分证明了文本和图片特征在多模态虚假信息检测任务中的确占据不同的比重，而 AlLNet 的交替训练方式能有效避免模态惰性，提升了模型的准确率。并且 AlLNet 也兼顾轻量性，推理速度快，还能应对模态缺失的情况，在真实世界场景中具有很大的应用潜力。

第 4 章　基于深度学习的虚假信息检测算法库

上一章介绍了一种有效的基于单模态交替的多模态虚假信息检测方法 AlLNet，但是这个方法仍然只是一个独立的算法模型，缺乏通用性和可扩展性。此外在上章节的实验中，每次复现已有的基线算法模型都需要重新编写代码，在变更参数等情况下需要反复手动调整，这其中存在大量重复性工作，对于研究者较为费时费力，并且在比较这些算法模型的运行时间和性能时也缺乏一个统一的基准。而且在实际应用中，虚假信息检测任务的数据集和场景多种多样，不同的数据集和场景可能需要不同的算法模型。因此，本文首次提出了一个基于深度学习的虚假信息检测开源算法库 **FaKnow**（**Fake Know**）[1]，其中包含多种虚假信息检测算法，方便研究者快速复现和比较不同的模型。此外，这个算法库还提供了一种通用的数据处理和模型训练框架，使得用户可以方便地将新的算法模型加入到这个算法库中。本章节首先介绍算法库这项工作的背景和意义，并对它的项目框架和主要功能模块展开叙述，然后通过复现性实验验证了此算法库的有效性，最后还给出了此算法库的使用示例。

4.1 虚假信息检测算法库的研究背景

目前，虚假信息检测这一研究领域发展较快，新的检测算法层出不穷，能够充分利用虚假信息的各种特点进行检测。但是，各类检测算法的关注点各不相同，所采用的技术手段有较大差别，适用的场景也有所区别。此外，这些虚假信息检测算法的编码实现对于这一领域内的研究者也是一大难题，尽管它们大多有开源的代码，但这些代码通常是在不同的平台或框架下开发的，各自采用的深度学习第三方依赖包不同，构建模型时没有统一规范，缺乏统一的调用接口，使用方式也大相径庭。而且这些开源的代码有不少只是算法原型 demo，并非一个完整的工程系统，缺乏通用性和可扩展性。同时，这些检测算法在神经网络模型的训练、验证和评估等方面又有不少相似之处，所用到的部分组件和模块也较为接近，存在着大量重复性工作。这些问题无疑给那些想要复现研究这些算法的学者带来了很多困扰。

为了解决上述痛点，本文提出了一个基于 PyTorch[2]实现的统一、全面、易用的虚假信息检测算法库 FaKnow，以便于研究者对虚假信息检测算法的复现和开发。作为虚假信息检测研究领域的首个算法库，FaKnow 集成了近年发表在顶会顶刊上、有代表性的算法模型，给使用者提供了丰富多样的选择，而且还包含了模型训练所需的一系列工作流程，包括便捷的数据集格式，完善的模型训练和评估功能，用户友好的结果可视化和日志保存，以及最终模型参数的保存，能够帮助开发者完成一些算法代码编写中的琐碎工作。下面将从 5 个方面介绍它的功能特点。

[1]https://github.com/NPURG/FaKnow
[2]https://pytorch.org/

- **丰富的算法模型**：FaKnow 内置了几十余种流行且具有代表性的虚假信息检测算法，包含了基于内容和基于社交上下文等多种检测手段的算法模型，给研究者提供了丰富多样的选择，以便他们进行论文结果的复现，或者将其作为自己算法研究的基线方法，又或者在这些算法的基础上改进。内置的模型中不止包含了一些早期经典算法，更多地是集中于近年新颖流行的算法，涵盖了文本内容、多模态、图神经网络和领域自适应等多个角度。
- **统一的功能框架**：FaKnow 将现阶段各种虚假信息检测算法集成为一个系统性的工程，提供了统一规范的调用接口，能够覆盖数据处理、模型编写、训练与评估以及最终结果保存等一系列算法开发的流程。它以数据模块、模型模块、评估模块和训练模块作为框架的核心组件，并封装了许多虚假信息检测任务中常用的算法功能组件，有效地减少了重复性任务并简化了算法开发过程。
- **通用的数据结构**：针对不同类型的任务和场景，FaKnow 中包含不同的数据格式以应对各种需要。对于基于内容的算法模型（包括基于纯文本和基于多模态），由于目前虚假信息检测的数据集大多是从微博等社交平台网页上爬虫抓取的数据，本算法库以 JSON（JavaScript Object Notation）作为输入文件格式，并允许用户自定义不同字段的处理方式。在框架内部，代码之间则采用 Python 的字典[3]数据结构进行数据交互，用户可以轻松地根据特征名来获取对应的数据值。而对于基于社交上下文的算法模型，则采用了业界通用的图神经网络库 PyG[4]中的图数据集格式。
- **便捷的使用方式**：FaKnow 整体框架均基于著名的深度学习库 PyTorch 实现，并在多个方面进行了封装，省去了日常模型训练和评估中的繁琐工作，提供了结果可视化、日志记录、参数保存等辅助功能，避免了大量代码冗余，还支持从配置文件和命令行读取超参数进行模型训练。此外，虽然框架有自己封装的类和函数，但是整体依然是 PyTorch 训练模型时常用的行为逻辑，学习成本低，任何熟悉 PyTorch 的研究者都可以快速上手。
- **良好的可拓展性**：FaKnow 中数据集、模型、训练、评估的相关类在设计时都屏蔽了内部细节代码逻辑，对外提供调用接口，使得整个框架的可拓展性较好。除去运行框架中已有的内置算法，用户如果想对已有算法进行微调，或者是开发全新的虚假信息检测算法模型，只需关注库中暴露的 API（Application Programming Interface，应用编程接口），按照规范继承它们就可以复用绝大部分功能，编写少量代码就可以满足自己的需要。

4.2 算法库框架及主要功能模块

FaKnow 算法库的总体框架如图4-1所示。FaKnow 以 PyTorch 作为后端底层实现，共由六个模块组成。数据模块负责输入模型的数据，包括统一的数据集格式以及数据处

[3]https://docs.python.org/zh-cn/3.13/library/stdtypes.html#dict
[4]https://www.pyg.org/

理等；模型模块集成了几十余种主流的虚假信息检测模型以及一些公共通用的神经网络模型组件；评估模块包含了常用的评估指标和评估器，用于评估模型在验证集和测试集上的分类表现；训练器模块建立在前面三个模块的基础上，将它们的功能有机地组织起来形成一个完整的训练学习流程，包括模型的训练、验证和评估；执行模块则在训练器模块的接口上进行了再次封装，用于直接运行库中的算法程序，它为用户提供了便捷的方式来快速启动内置的算法模型，允许用户从配置文件或者参数字典中传入所需的参数；工具模块主要提供程序运行中所需的一些便捷辅助功能，包括数据可视化，日志输出以及早停等。其中数据、模型、训练器和评估模块是本算法库中的核心功能模块，下文将对它们的设计思路和实现细节进行详细介绍。再次强调，本文将使用Consolas等宽字体对章节中出现的程序代码变量进行书写，以便与普通英文字母和数学符号区分。

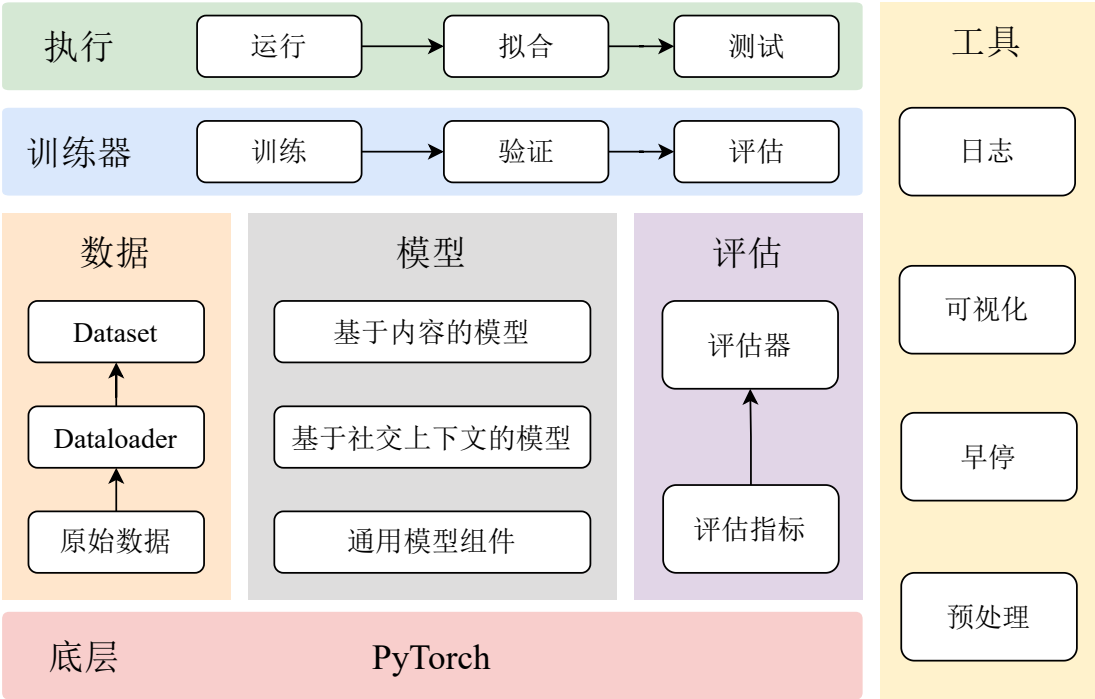


图 4-1　算法库框架示意图

4.2.1 数据模块

数据模块中主要包含了 FaKnow 库中集成的所有模型需要用到的PyTorch.Dataset数据集类以及一些常用的数据处理函数，也允许用户在此基础上拓展或者添加新的数据集类来应对不同情况。

(1) 基于字典的批量数据

为了使得模型训练过程和调用方式能够统一，FaKnow 在模型内部以 Python 中的Dict字典（一种键值对形式的数据结构）作为批量输入的数据的格式，并将输入数据的特征名作为字典中的键，输入数据的PyTorch.Tensor形式的向量作为字典中的值，这使得用户能够轻松地引用它们的名字来获取对应的特征值。数据模块中包含的Dataset类

均继承PyTorch.Dataset，可以通过其getitem方法[5] 来迭代遍历Dataset获得上述Dict数据。

(2) 适用不同类型模型的数据集类

如上文所述，FaKnow 使用Dict字典作为内部数据读取格式，但是用户在自己编写的程序代码中通常并不直接操纵字典数据，而是使用 PyTorch 中的Dataset数据集类。因此在字典数据外部还需要再包裹一层Dataset来作为代码程序的交互数据结构。针对算法库中集成的不同类型的模型，数据模块中也提供了不同的Dataset数据集类，以适应不同模型的输入数据格式。

对于那些使用社交网络传播信息的基于社交上下文的检测算法，FaKnow 直接采用 PyG（一个著名的基于 PyTorch 的图神经网络库）中的数据集格式。针对基于内容的检测算法，FaKnow 中设计了专门的数据结构用于存储这些来自内容的特征。由于现有的虚假信息数据集大多是从推特、微博等社交平台爬取的帖子数据，同时为了契合上文中提到的字典这一通过特征名引用数据的方式，FaKnow 采用了基于键值对的 JSON 作为原始数据文件的格式。每个样本作为一个 JSON 对象，其中包含它的原始字段特征作为键值对，所有的样本作为一个数组保存在 JSON 文件中。

FaKnow 中内置的TextDataset和MultiModalDataset类，分别适用于文本数据集和多模态数据集，其中MultiModalDataset类继承自TextDataset。这两个数据集类能够读取上述 JSON 数据文件中的样本，包括文字、图片路径、样本标签等字段，并允许用户自定义对不同字段特征的处理方式。

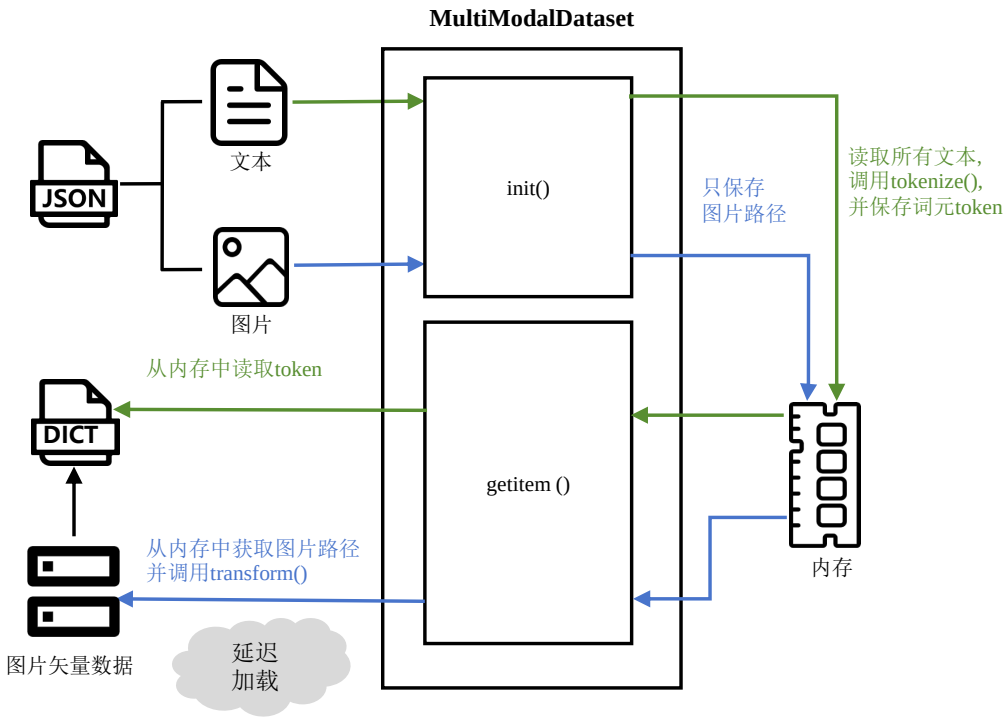


图 4-2 MultimodalDataset对多模态数据的处理流程

[5]为表区分，本文按面向对象程序设计的术语规范，将类中的函数称为方法

如图4-2所示，以MultiModalDataset为例，它能够处理同时包含文字和图片的多模态数据，用户只需要分别指定 JSON 文件中文字和图片的字段名称，并传入文本数据的处理函数tokenize和图片数据的处理函数transform，就能使MultiModalDataset的getitem方法同时返回包含文字和图片数据的Dict。为了保证程序的可拓展性，用户可以分别在tokenize函数中自定义文本的处理逻辑代码（包括分词等操作）和transform函数中自定义图片的处理逻辑代码（包括读取像素值等操作）。

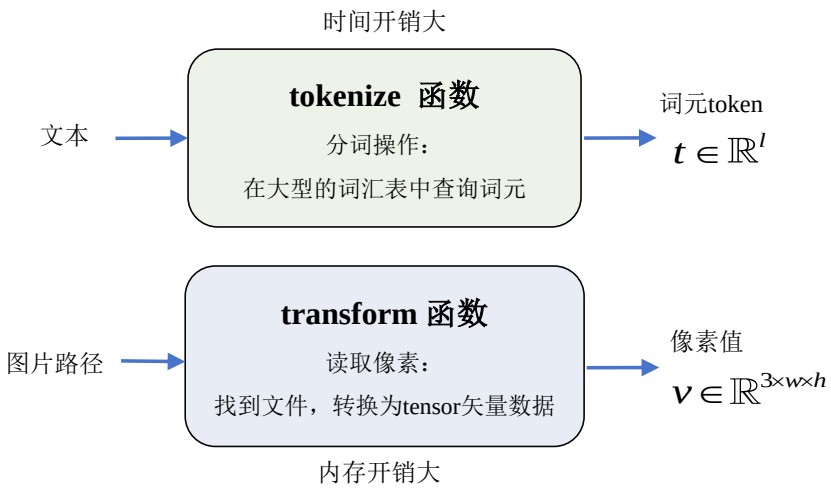


图 4-3　tokenize函数与transform函数的不同处理方式

图4-3展示了文本数据处理和图片数据处理的不同特性。对于文本数据，由于文字在分词时需要在庞大的词汇表中查找，这一过程较为耗时，而一段文本经过分词后得到的词元序列通常是一个不太长的向量 $t \in \mathbb{R}^l$（其中 l 是文本序列的长度），并不会占用过多内存。因此对于文本数据，MultiModalDataset在执行init方法进行初始化时就调用用户传入的tokenize函数，完成包括分词在内的所有处理流程，并存储分词后的词元 t。从数学的角度来看，tokenize函数可以表示为 $t = tokenize(text)$，其中 $text$ 是原始文本数据。在遍历MultiModalDataset中的数据时，程序将直接根据索引下标获取返回的词元。

与文字处理的特性相反，读取图片像素值的操作耗时较短，但是存储一张 RGB 格式的图像数据通常是一个占用内存空间较大的三维数组 $v \in \mathbb{R}^{3\times w\times h}$（其中 3、$w$ 和 h 分别为图片的像素通道数量、宽度和高度），因此 FaKnow 采用了时间换取空间的策略来处理图片。MultiModalDataset在init方法中进行初始化时只存储图片的文件路径，在遍历访问时才会调用transform函数，根据文件路径来读取图片数据到内存中。从数学的角度来看，transform函数可以表示为 $v = transform(image_path)$，其中 $image_path$ 代表原始图片文件的路径。这种延迟加载（lazy loading）的方式只需付出些许程序执行时间的代价就能极大地节省内存空间。

而作为MultimodalDataset父类的TextDataset，只适用于文本数据而不能处理图片数据，这两个Dataset类处理文本的方式相同，故此处不再赘述。

通过 JSON 数据文件、内置的数据集类以及基于字典的批量数据格式这三种数据结构，用户能够便捷地将原始数据转化为 FaKnow 库所需的格式并用于算法内部的交互。

4.2.2 模型模块

基于上述的数据模块，FaKnow 将内部集成的所有虚假信息检测算法的模型组织成模型模块并提供了统一的调用接口。

(1) 集成的模型

表 4-1　FaKnow 中集成的算法模型

种类	模型	出版来源	年份
基于内容	TextCNN[18]	EMNLP	2014
	EANN[17]	KDD	2018
	SpotFake[15]	BigMM	2019
	SAFE[19]	PAKDD	2020
	MDFEND[13]	CIKM	2021
	MCAN[22]	ACL	2021
	HMCAN[23]	SIGIR	2021
	MFAN[57]	IJCAI	2022
	ENDEF[58]	SIGIR	2022
	M3FEND[59]	TKDE	2022
	CAFE[27]	WWW	2022
	ALGM[29]	SIGIR	2023
基于社交上下文	GCN[60]	ICLR	2017
	GraphSAGE[34]	NeurIPS	2017
	GAT[61]	ICLR	2018
	GCNFN[62]	arXiv	2019
	BIGCN[31]	AAAI	2020
	FANG[33]	CIKM	2020
	UPFD[32]	SIGIR	2021
	GNNCL[63]	ICANN	2021
	DUDEF[35]	WWW	2021
	EBGCN[36]	ACL	2021
	TrustRD[64]	CIKM	2023

表4-1展示了本算法库中集成的不同模型，它们大多来自近五年发表在 AAAI、SIGIR、IJCAI、ACL、KDD 等会议和期刊上的具有代表性的论文。为了使库中的模型尽可能多元化，给用户提供丰富的选择来应对不同类型的任务，FaKnow 中选取的这些论文覆盖了基于内容和基于社交上下文两大类别，涉及到多模态、域适应、图神经网络等多种技术手段。此外，FaKnow 中也包含了一些早期经典的通用分类模型，包括用于文本分类的 TextCNN、用于图分类的 GCN、GAT 和 GraphSAGE 等。

同时，为了减少重复性工作，FaKnow 中也将那些虚假信息检测算法中常用的神经网络组件抽取出来，作为独立的一部分，形成通用模型组件，比如用于提取文本特征的

TextCNN 层，用于提取图片频域特征的离散余弦变换层，用于多模态信息融合的协同注意力层等。这些组件既能供算法库自身内置的模型复用，又可以被用户用于新模型的开发。

(2) 统一的接口

为了使得模型被调用的接口能够统一，FaKnow 中设计了一个名为`AbstractModel`的抽象类，它继承自 PyTorch 中的`nn.Module`并且是 FaKnow 中所有模型的父类。因此，这些内置模型实现的大致逻辑与使用 PyTorch 时并无差别，同样需要重写`init`和`forward`方法分别用于模型中成员变量的初始化和模型的前向传播计算。但是`AbstractModel`还具备两个新的成员方法`calculate_loss`和`predict`，库中所有的模型都必须重写这两个方法。图4-4通过 UML 类图说明了算法库中设计的模型的继承关系。

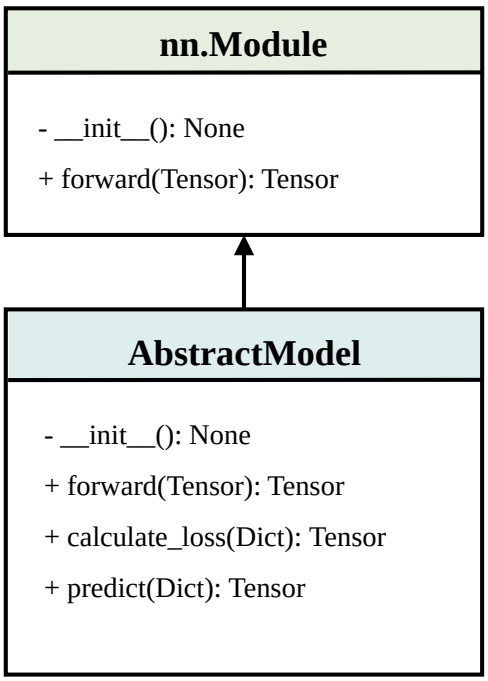


图 4-4　`AbstractModel`的 UML 类图

新增的`calculate_loss`和`predict`都接收上文数据模块中提到的`Dict`字典数据作为批量样本的输入，并调用`forward`方法获取模型最后一层的输出。前者根据定义好的损失函数计算并返回损失值，用于模型训练中通过反向传播更新模型的参数；后者则返回模型最终虚假信息的分类结果，用于模型的推理和评估。

通过这个两个新增的方法，FaKnow 库可以适应各种类型的模型。无论是模型最后一层有多个输出，或者由多个损失累加形成模型最终损失的情况，模型在训练和推理阶段的调用方式都能够统一起来。用户在使用 FaKnow 库开发新的模型时，也只需要重写这些接口方法，无需关心在算法库内部模型如何被调用的代码细节（实现新模型的具体方式详见后文4.4.2节）。图4-5展示了`calculate_loss`方法和`predict`方法的调用过程。

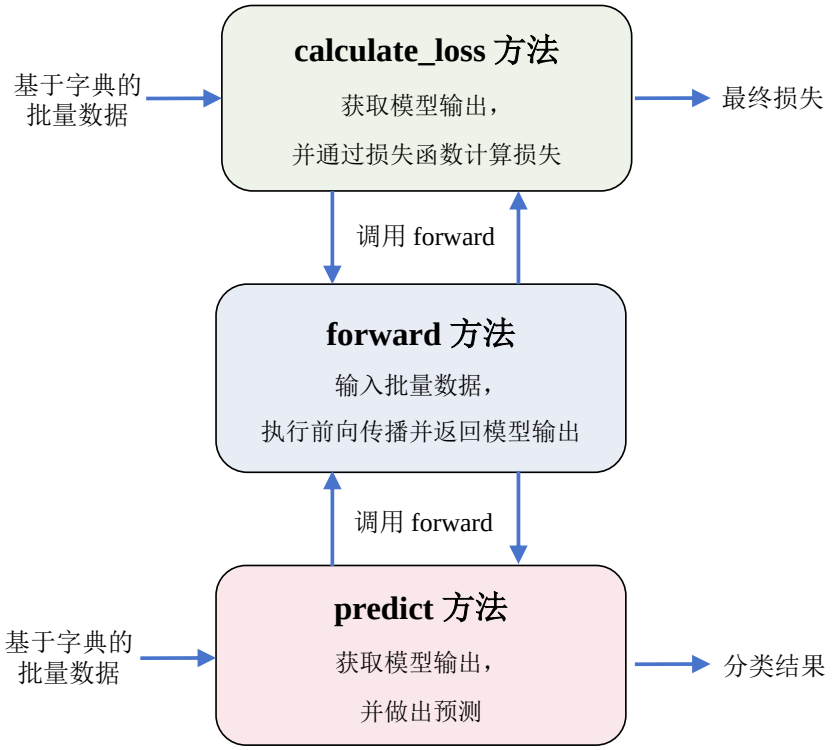


图 4-5　calculate_loss方法和predict方法的调用过程

4.2.3 评估模块

在模型训练与测试过程中通常需要对模型的分类效果进行评估，以便于选择最优的模型和超参数，或者对模型进行调优，以提高模型的泛化能力。因此，FaKnow 中设计了评估模块，其中包含了一系列评估指标和评估器，用于评估模型在数据集上的分类表现。

(1) 评估指标

由于虚假信息检测通常为一个二分类问题，因此评估模块中主要内置了一些常用的二分类评估指标，包括准确率、精确率、召回率、f1 分数等。为了保证算法库的性能以及程序执行的效率，FaKnow 并未借助诸如 Scikit-learn 之类的外部第三方库，而是使用 PyTorch 中的Tensor矢量，从底层编写代码来实现上述评估指标的计算，以便能充分利用 GPU 显卡加速计算，提升模型评估的执行速度。这些评估指标均以功能函数的形式集成在评估模块中，可供用户直接调用，但更多的情况下，它们是作为评估器（见下文）的一部分被调用。

除去上述常用的评估指标，在某些任务场景下，用户可能还需要一些特定的评估指标。例如，在 NEP[65] 这篇论文中，作者为了评估模型在真实世界数据不平衡的情况下的性能，使用了标准化部分面积[66]（spAUC，standard partial Area Under Curve）作为评估指标。因此，为了满足用户的不同需求和算法库的可拓展性，FaKnow 中的评估模块也支持用户自定义评估指标。用户需要在程序中编写自定义评估指标的计算函数，并将

其传入评估器中，评估器将在评估过程中调用这些函数来计算用户自定义的评估指标。

如图4-6所示，用户自定义的评估指标计算函数需要接收模型的预测概率 $pred \in \mathbb{R}^{b\times 2}$ 和样本的真实标签 $label \in \mathbb{R}^{b}$ 共两个参数，分别代表样本被分到正/负类的概率和样本的类别下标索引，其中 b 为批量样本的数量（batch size）。用户需要在函数中根据这两个参数计算出自定义的评估指标值，然后将其作为函数输出返回给评估器，评估器将会在评估过程中统一计算所有评估指标的值。从数学角度上来看，用户自定义的评估指标计算函数可以表示为 $metric_value = metric_function(pred, label)$。

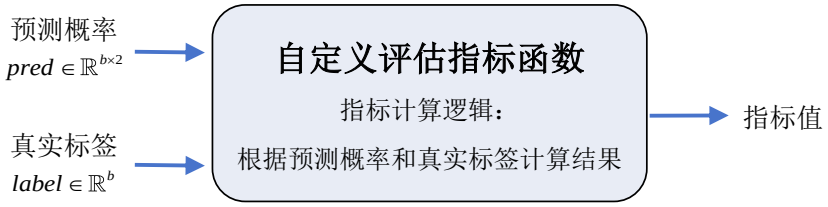


图 4-6　自定义评估指标的计算函数

(2) 评估器

无论是 FaKnow 内置的评估指标还是用户自定义的评估指标，它们都只是作为函数的形式存在，而要想在模型训练和测试过程中得到模型评估的结果，就需要一种方式来统一管理这些评估指标的计算过程。为此，评估模块中额外设计了评估器类`Evaluator`，用于统一调用评估指标的计算函数。

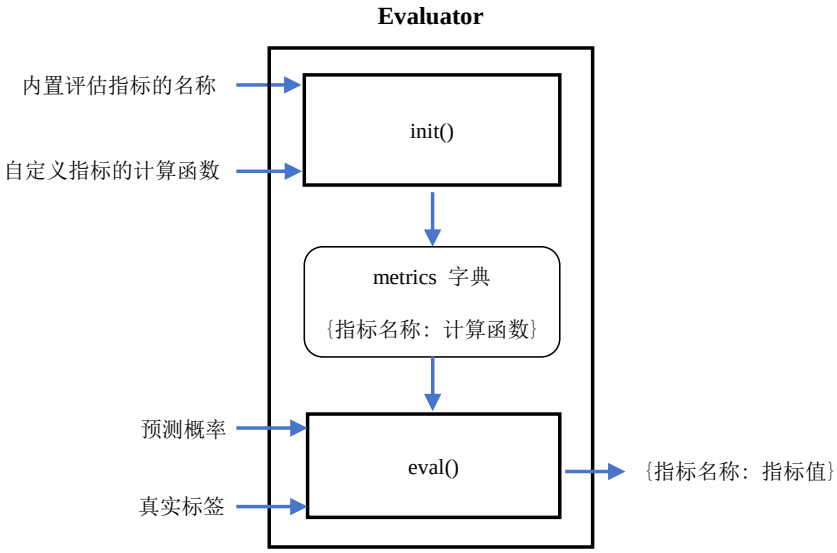


图 4-7　评估器类`Evaluator`

如图4-7所示，`Evaluator`类初始化时，用户需要向`init`方法中传入 accuracy 等内置评估指标的名称以及自定义指标的计算函数（如无需自定义指标则不传入）。评估器内部维护了一个字典数据结构的成员变量`metrics`来存储所有评估指标，其中键为指标的名称，值为指标的计算函数。在评估过程中，用户只需调用`eval`方法并传入模型的预测

概率和样本的真实标签，评估器内部会遍历metrics中的所有评估指标，调用它们的计算函数来计算各个评估指标的值，再将这些值以键值对的形式返回输出。

从整体来看，评估器作为评估模块对外的接口，统一管理了所有评估指标并在内部隐藏了它们的具体计算细节，使得用户能够在模型训练和测试过程中方便地调用内置的评估指标和自定义的评估指标。

4.2.4 训练器模块

为了适配统一库中不同模型的训练过程，减少重复性工作，FaKnow 中还设计了训练器模块。训练器模块紧承上述数据、模型和评估模块，负责将数据集分批次地送入模型进行运算，其中包含了模型训练、验证和评估，模型参数保存，训练日志输出，训练过程可视化等功能，并将它们统一封装成到一个名为Trainer的训练器类中。训练器的训练过程遵守常见的深度学习模型训练规则。用户在自行划分数据集后，将训练集和验证集传入训练器中，训练器将对样本开展多轮训练学习。在每一轮中训练器会将训练集分批次送入模型进行训练，计算损失值并进行反向传播更新模型参数，并在每一轮训练结束后将验证集送入模型进行验证，计算模型在验证集上的评估指标。更重要的是，Trainer屏蔽了内部细节，用户不用关心它的执行流程，只需指定待训练的模型、损失函数优化算法等基本参数就能快速训练模型。同时它也支持梯度裁剪（gradient clipping）、学习率调度器（learning rate scheduler）以及早停（early stopping）等高级功能来适应不同的情况和需求。

（1）工作流程

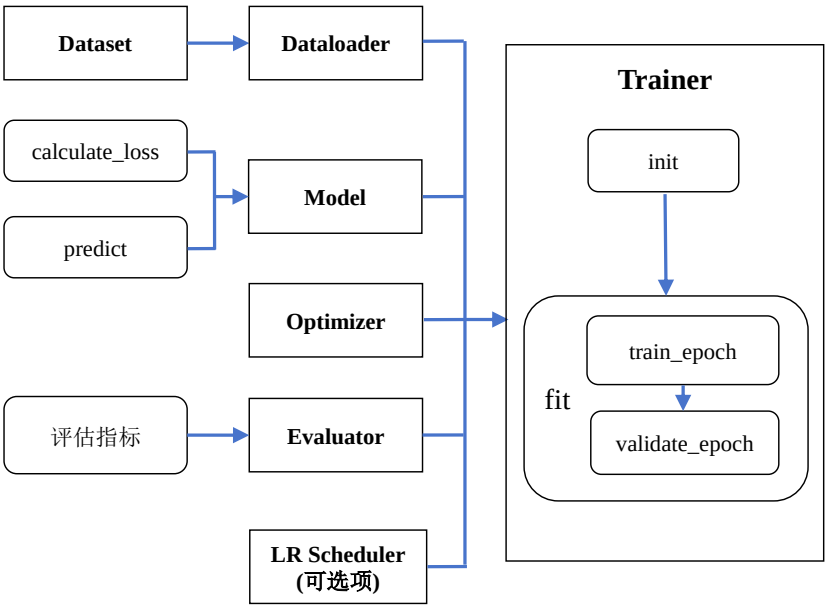


图 4-8　Trainer的工作流程

图4-8展示了Trainer的工作流程。训练器初始化时，需要用户向init方法中传入数据加载器Dataloader、待训练的模型Model、用于参数优化的优化器Optimzer以及评估

器Evaluator等基本参数。随后，训练器就可以调用fit方法让模型对数据集进行拟合学习，该方法接收训练数据、验证数据和训练轮数等参数。在每一个训练轮次中，fit将调用自身的train_epoch和validate_epoch方法来分别训练和验证模型，同时也会记录训练损失和模型在验证集上的分类表现，以便于用户可以随时跟踪查看模型的训练结果。

具体来说，train_epoch方法将会遍历训练数据集，将数据分批次送入模型执行前向传播并计算损失值（即调用AbstractModel类的统一接口calculate_loss），然后根据用户选择的优化算法进行反向传播更新模型参数。而validate_epoch则同样地分批次遍历验证数据集，将数据送入模型执行前向传播并获取模型的预测概率（即调用AbstractModel类的predict方法），然后调用评估器的eval方法计算在验证集上的评估指标值。同时，为了便于进行模型和超参数的选择，validate_epoch还会记录分类表现最优的训练轮次以及对应的最优指标结果。此外，为了尽可能给用户提供自由发挥的空间，当未传入验证集数据时，fit方法只会对模型进行训练而不会调用validate_epoch方法进行验证。

(2) 高级功能

为了给用户提供更多便捷的功能，训练器模块除了支持基本的模型训练，还提供了很多额外的辅助功能。针对模型训练，训练器支持许多高级配置。

- **梯度裁剪**：梯度裁剪是一种用于防止梯度爆炸的技术，能够使得梯度的范数不超过一个给定的阈值，避免模型训练过程中产生数值不稳定的情况。FaKnow 支持用户自定义梯度裁剪的阈值，当梯度的范数超过这个阈值时，梯度将被裁剪。训练器将在训练阶段的每一次反向传播中梯度计算完毕后、参数更新前执行裁剪操作。
- **学习率调度器**：学习率调度器是一种用于在训练过程中动态调整学习率的技术，能保证模型在训练后期能够更快地收敛。FaKnow 支持用户使用不同的学习率调度器，用户可以选择使用 PyTorch 中自带的调度器，也可以自定义调度器。训练器将在每轮训练和验证执行结束后按照用户指定的方式动态调整优化器的学习率。
- **早停**：早停是一种用于防止模型过拟合的技术，当模型在验证集上的表现不再提升时，训练器将停止训练，避免模型学习到训练集的噪声。FaKnow 支持用户自定义早停的参数，包括忍耐轮数（patience）、最小提升值（delta）等。训练器将根据每轮模型在验证集上的表现来决定是否要提前结束训练，并保存所有轮次中在验证集上表现最优的模型以及对应的最优指标结果。
- **日志**：训练器支持将训练过程中每一轮的损失值、评估指标（包括准确率以及其他指标）、日期时间等信息输出到控制台，并同步保存到本地日志文件中，方便用户随时查看训练过程中的细节，以及后续的分析和调试。
- **可视化**：训练器支持将训练过程中的损失值、评估指标等信息可视化输出到 TensorBoard （一个由 Google 开源的深度学习可视化工具）[6]中形成折线图，用户可以通过 TensorBoard 的控制面板来查看训练过程中的变化趋势，以便于更直观地

[6]https://tensorflow.google.cn/tensorboard?hl=zh-cn

了解模型的训练情况。

如果上述功能仍不能满足用户需要，那么用户可以选择继承本模块中的`Trainer`，在尽可能复用已有代码的条件下去制订新功能。

4.3 可复现性实验

为了保证 FaKnow 中集成的虚假信息检测模型的正确性和实验结果的可复现性，本文对所有模型在不同的数据集上进行了实验，并将实验结果与原论文中的实验结果进行了对比。而对于如 TextCNN、GCN 等模型，由于其在诞生之初就只作为通用的分类算法，并非专门用于虚假信息检测任务，故未对其在虚假信息数据集上进行复现性实验。

4.3.1 实现细节

为了确保实验严谨和结果准确，在本节的复现性实验中，学习率、训练轮数、批量大小等所有超参数以及所使用到的数据集以及评估指标（准确率，精确率，召回率，f1 分数，auc 面积）均与原论文的代码中严格保持一致。而对于那些原论文实验结果中并未进行评估的指标，则使用空白符在结果表格中显示。

特别地，为了与原论文保持一致，对于 ENDFN、DUDEF 和 UPFD 算法框架，实验中分别使用了 MDFEND、BiGRU 和 GraphSage 模型作为其中的主干模型（backbone）与原文的实验结果进行对比。而对于 GCNFN 和 GNNCL 这两个模型，由于它们的原作者并未提供开源代码，本实验中使用了在 UPFD 论文中作为基线方法一起发布的这两个模型的代码和数据集，并按照该论文中的步骤进行复现性实验。对于 UPFD-GraphSage、GNNCL、GCNFN 三个模型，它们的输入特征按原论文分别为经 BERT 编码后的词向量、用户个人资料和帖子内容。另外，由于 HMCAN 的作者并未提供数据处理的代码或已处理好的数据文件，本实验只能按照论文中的说明，重新编写代码来处理数据集中的原始文本和图片。此外，由于 SAFE 论文作者上传的数据集中部分图片文件已损坏，本试验中已将这些损坏的图片剔除，只使用剩下完好的数据集来训练模型。

4.3.2 数据集

本实验中对每个模型进行复现时，均使用了原论文中所使用的数据集，包括 Weibo17、Weibo21、CED、WeiboNEP、Weibo20、FakeNewsNet、MM-COVID、MediaEval15&16、Twitter15&16、TwitterMa 和 PHEME，以保证实验结果的可比性。它们均为虚假信息检测任务中的常用数据集，涉及微博、推特等多种社交媒体平台，其中包含了大量的真实新闻和虚假新闻样本，以及丰富的多模态信息和社交上下文信息。这些数据集的详细信息如下所述。

- **Weibo17**：该数据集由中国科学院计算技术研究所的曹娟等人在 2017 年 ACM MM 会议上发布，是虚假信息检测领域中较早的中文多模态数据集，其中共有 9528 个帖子，包含 4749 条虚假信息和 4779 条真实信息，并按 8:2 的比例划分为训练集和测试集。作者在新浪微博官方辟谣系统中抓取了 2012 年 5 月至 2016 年 1 月期间

所有经过验证的虚假谣言帖子作为其中的正样本，并使用经过中国权威通讯社新华社认证的推文作为数据集的负样本。与传统给数据集不同的是，Weibo17 除了包含帖子的文本信息外，还包含了帖子的图片以及可用的社交网络信息。同时作者还使用了基于局部敏感哈希（LSH）的近重复图像检测算法从原始集中删除重复图像，并也删除非常小或长的图片，以保持图像样本的质量。数据集中的每个样本有帖子 id、帖子的 URL、帖子文本、图片 URL、发布用户信息（id、个人主页 URL、账号认证类型、粉丝数、关注数、历史发帖数）、发布时间、转发数、评论数、点赞数等丰富的多模态以及社交上下文信息。该数据集质量之高，使得它成为了虚假信息检测领域的一个重要基准数据集 [67]。

- **Weibo21**：该数据集由南琼等人在 2021 年发布，是首个中文多领域虚假信息检测数据集，其中共有 9128 个帖子，其中包括 4640 条真实信息和 4488 条虚假信息。作者从微博社区管理中心爬取了从 2014 年 12 月至 2021 年 3 月的数据，对每个样本都收集了文本内容、图片、时间戳、评论等多个维度的数据，并使用 one-pass 聚类的方法对数据集中的重复帖子进行了去重处理。然后由 10 名标注人员对其进行领域标注，最终将样本分为科技、军事、教育、事故、政治、健康、财经、娱乐、社会共 9 个不同的领域[13]。
- **CED**：该数据集为作者从新浪微博不实信息举报平台抓取的中文谣言数据，其中共有 3387 条帖子，包含 1538 条谣言和 1849 条非谣言。每条微博提供了文字内容、发布的用户信息以及发布的时间戳三个维度的信息。同时还包含了与微博原文相关的转发与评论信息，包括转发/评论的用户 id、转发/评论的文字内容以及转发/评论的发布时间[68]。
- **WeiboNEP**：该数据集综合了多个前人开源的微博数据集，覆盖了多个年份和不同的新闻话题环境，并移除了其中的重复样本。同时作者还对合并后的数据集进行了重新采样，以保证真假样本的数量平衡。最终处理后的数据集包含了从 2010 年到 2021 年的 39066 条微博[65]。
- **Weibo20**：该数据集由作者在马晶等人发布的中文虚假信息检测基准数据集 Weibo16[10] 的基础上进行了部分拓展。作者保留了 Weibo16 的 1355 条虚假信息，并进一步收集了 2014 年 4 月至 2018 年 11 月期间被微博社区管理中心官方认定为错误信息的微博，同时还过滤掉高度相似的假样本以保证没有重复数据。对于真实信息，作者则保留了 Weibo16 的 2351 条微博样本，并收集了 850 条与虚假信息同时期的无重复的真实样本。此数据集最终共包含 3161 条虚假信息和 3201 条真实信息，并按照 6:2:2 的比例分割训练集、验证集和测试集[35]。
- **FakeNewsNet**：该数据集包含了英文新闻以及从推特上抓取的与之相关的推文，并提供了丰富的样本特征信息，主要分为新闻内容（推文文本、图片），社交上下文（粉丝数、关注数、点赞数等推文用户信息，转发数、评论数等转发评论信

息），时空信息（用户地理位置、发文位置等空间信息，发布时间、回复时间等时间信息）三大特征属性。该数据集分为 Politifact 和 GossipCop 两个子集，分别为从 politifact[7]和 gossipcop[8]这两个事实核查网站上收集的政治时事新闻和娱乐新闻。FakeNewsNet 的庞大样本数以及丰富的样本特征信息使得它成为了虚假信息检测任务中常用的英文基准数据集[55]。

- **MM-COVID**：该数据集为新冠疫情 COVID-19 相关的虚假信息检测数据集，由作者从推特上抓取的新冠疫情相关的推文数据构成，共包括英语、西班牙语、葡萄牙语、印地语、法语、意大利语共 6 种语言的 3981 条虚假新闻内容和 7192 条可信信息。与 FakeNewsNet 类似，MM-COVID 同样包含了新闻内容、社交上下文、时空信息等基本特征。更难能可贵的是，MM-COVID 不单局限于传统的中英文的文本数据，还提供了多种不同语言的样本，使得其成为了一个多语言虚假信息检测数据集的典范[69]。
- **MediaEval15&16**：该数据集最早源自 MediaEval 2015 和 2016 Workshop 这两年的 Verifying Multimedia Use 比赛任务，主要涉及网络多媒体内容中篡改和滥用的自动化检测。MediaEval15 中由 17 个事件相关的推文组成，其中包含 193 个真实图像案例、218 个滥用图像案和 2 个滥用视频案，这些推文来自于 5895 和 9025 个独立用户分别发布的 6225 条真实信息和 9404 条虚假信息。而 MediaEval16 则是在 MediaEval15 的基础上拓展而来，包含了更多的事件和更多的数据[70]。
- **Twitter15&16**：该数据集是在前人发布的两个推特数据集[10, 71] 的基础上发展而来。原始数据集主要用于针对包含相关推文的给定事件进行谣言和非谣言的二分类。而作者则收集了推文发布源的相关信息，以及它们的传播路径信息（包含转发和回复），以供使用者可以方便地构建谣言传播树或传播图。同时作者也将其中每个事件的标签拓展到四个类别，分别为 Non-Rumor、Unverified Rumor、False Rumor 和 True Rumor。整个数据集分为 Twitter15 和 Twitter16 两个子集，分别为 2015 年和 2016 年的推文数据。前者包含了 1490 条源推文（分为 374 个 Non-Rumor 样本、374 个 Unverified Rumor 样本、370 个 False Rumor 和 372 个 True Rumor 样本）和 331612 条转发/回复推文，共涉及 276663 个推特用户。后者则提供了 173487 条用户信息，818 条源推文（四类样本数分别为 205、203、205、205）以及 204820 条转发/回复的推文线索[72]。
- **TwitterMa**：作者从 Snopes[9]（一个在线辟谣网站）收集了 2015 年 3 月至 12 月期间报道的新闻事件（其中 64% 为谣言），并根据每个事件的关键字从推特抓取了相关的推文信息，从而构建了此数据集。整个数据集包含了 992 个新闻事件，分为 498 条谣言和 494 条非谣言样本，共涉及 1101985 条推文和 491229 个推特用户，平均

[7]https://www.politifact.com/
[8]https://www.gossipcop.com/
[9]http://www.snopes.com

每篇新闻事件对应着 1111 条推特帖子，平均传播时长为 1582.6 小时[73]。

- **PHEME**：该数据集由突发新闻期间发布的推特上出现的谣言推文与非谣言推文组成，也是一个四分类数据集。其中包括 Charlie Hebdo、Sydney siege、Ferguson、Ottawa shooting、Germanwings-crash、Putin missing、Prince Toronto、Gurlitt、Ebola Essien 共 9 个新闻事件，涉及 6425 条线索样本（分为 4023 个 Non-rumor、697 个 Unverified Rumor、1067 个 True Rumor 和 638 个 False Rumor）[74]。

4.3.3 实验结果及分析

表4-2展示了 FaKnow 中集成模型的进行复现性实验的结果，以及与原论文中给出的评估指标的比较结果。由于使用的 Python、PyTorch 等工具和环境的版本差异、主机硬件设备的不同以及随机数初始化等不可控因素的影响，任何实验结果都不可能完全与原论文中的结果完全一致，但是本文尽可能地保证了实验条件的一致性，以确保实验结果的可比性。

从表中可以看出，对绝大多数模型，FaKnow 的复现结果均与原文给出的指标较为接近，只相差 1 到 2 个百分点，均在可接受的误差接受范围内。对基于内容的检测算法，以 MFAN 为例，FaKnow 在 CED 数据集上复现的结果与原文差距最小的指标为准确率 acc，差距仅为 0.001，而在差距最大的召回率 rec 这一项上，两者相差也只有 0.11。同样地，ALGM 算法在 MediaEval16 数据集上的复现结果与原文中给出的实验结果也非常接近，在准确率、精确率、召回率和 f1 分数四项指标上的差距分别为 0.006、0.004、0.006 和 0.006，几乎可以认为是一致的。

而对于基于社交上下文的模型，如 UPFD，在 Politifact 数据集上复现的准确率和 f1 分数分别为 0.833 和 0.817，同样地与原论文中给出的实验结果非常接近。DUDEF 模型在 Weibo20 数据集上复现的准确率和 f1 分数分别为 0.865 和 0.893，与原文分别相差 0.01 和 0.038，也可视为复现成功。剩下的 EANN、SpotFake、MDFEND、MCAN、M3FEND、GCNFN、BIGCN、FANG、TrustRD 等模型复现得到的各项指标也与原文中的实验结果相差不大，此外，ENDEF、CAFE 和 GNNCL 这三个模型的复现准确率甚至比原文的结果还要高出大 4 个百分点。这些结果都充分证明了 FaKnow 中集成的模型的有效性。

然而，从实验表格中也不难观察到部分模型的复现结果与原论文存在轻微差异，这可能是复现时实验条件和实现细节稍有不同所引起的。这些差异为我们提供了进一步深入研究的契机，以便更好地理解模型行为，并可能改进现有的方法。

对 HMCAN，复现出的准确率 acc、精确率 pre、召回率 rec 和 f1 分数分别只有 0.816、0.8、0.841、0.82，较原论文中给出的指标分别低了 4 到 8 个百分点不等，这可能是由实验中数据预处理的步骤与原作者实际处理方式有所出入而导致的。同时该模型在 MediaEval15 数据集上的复现结果也比原论文略低几个百分点，但是本文复现的结果与 CSFND[75] 这篇论文中对 HMCAN 模型在这两个数据集上的复现结果几乎一致。

表 4-2　FaKnow 内置模型的复现性实验结果

模型	数据集	结果	评估指标				
			acc	pre	rec	f1	auc
EANN	Weibo17	原文	0.827	0.847	0.812	0.829	-
		复现	0.800	0.800	0.790	0.790	-
SpotFake	MediaEval16	原文	0.777	0.791	0.753	0.760	-
		复现	0.769	0.765	0.866	0.812	-
SAFE	FakeNewsNet-Politifact	原文	0.874	0.889	0.903	0.896	-
		复现	0.791	0.836	0.796	0.816	-
MDFEND	Weibo21	原文	-	-	-	0.913	-
		复现	-	-	-	0.912	-
MCAN	Weibo17	原文	0.899	0.898	0.899	0.899	-
		复现	0.873	0.919	0.832	0.873	-
HMCAN	Weibo17	原文	0.885	0.888	0.885	0.885	-
		复现	0.816	0.800	0.841	0.820	-
MFAN	CED	原文	0.889	0.889	0.881	0.883	-
		复现	0.888	0.896	0.870	0.879	-
ENDEF	WeiboNEP	原文	0.806	-	-	0.731	0.849
		复现	0.846	-	-	0.802	0.877
M3FEND	FakeNewsNet&MM-COVID	原文	0.897	-	-	0.851	0.934
		复现	0.921	-	-	0.920	0.976
CAFE	MediaEval15	原文	0.806	0.806	0.806	0.806	-
		复现	0.840	0.868	0.812	0.839	-
ALGM	MediaEval16	原文	0.891	0.909	0.893	0.890	-
		复现	0.897	0.913	0.899	0.896	-
GCNFN	Politifact	原文	0.832	-	-	0.836	-
		复现	0.850	-	-	0.889	-
BIGCN	Twitter16	原文	0.880	-	-	0.879	-
		复现	0.868	-	-	0.854	-
FANG	FakeNewsNet&PHEME&TwitterMa	原文	-	-	-	-	0.751
		复现	-	-	-	-	0.776
UPFD	FakeNewsNet-Politifact	原文	0.846	-	-	0.846	-
		复现	0.833	-	-	0.817	-
GNNCL	FakeNewsNet-Politifact	原文	0.629	-	-	0.622	-
		复现	0.660	-	-	0.714	-
DUDEF	Weibo20	原文	0.855	-	-	0.855	-
		复现	0.865	-	-	0.893	-
EBGCN	Twitter16	原文	0.915	-	-	0.910	-
		复现	0.837	-	-	0.820	-
TrustRD	Twitter15	原文	0.931	-	-	0.927	-
		复现	0.924	-	-	0.846	-

对 SAFE 模型，由于在实验中使用了残缺不全的数据集来训练，故而在各项指标上其复现结果均低于原论文中的表现。此外，该算法中需要为图片生成一段文字描述，用来比较图片和文本模态的一致性，因而生成的文字描述在很大程度上决定了模型的效

果。但是用于图片内容文本生成的预训练模型 ShowAndTell 又往往受其使用的训练数据集影响，迁移到新的任务或数据集上时会有一定偏差。

同样地，EBGCN 的复现结果也比原论文低了大约 8 个百分点，这可能是由于其中用于预测无标签潜在边的无监督 Edge-wise Consistency 模块，并没有很好地学习到结点之间的关系，从而影响了模型最终的训练效果。

无论如何，除去上述几个例外，复现性实验的结果在准确率等各项评估指标上基本与原作者给出的数据接近，这充分证明了 FaKnow 中集成的模型的有效性。

4.4 使用示例

本节将介绍如何使用 FaKnow 算法库进行虚假信息检测任务的实验，并给出简单的示例代码进行说明，主要包含两部分：运行 FaKnow 中内置的模型和基于 FaKnow 开发新的模型。值得注意的是，为简单起见，本节的示例代码中省略了那些导入必要的包和模块的 import 代码，用户在实际使用时需要自行添加。有关 FaKnow 的更多使用方法和详细说明，请参考 FaKnow 的官方文档[10]。

4.4.1 运行内置模型

(1) 快速启动

为了方便用户快速上手使用，FaKnow 提供了`run`和`run_from_yaml`两个函数。这两个函数封装了模型训练和评估的所有流程，只需要用户指定模型的名称以及相关参数就可以快速启动程序。`run`接收关键字形式的参数（包括数据集，模型初始化的参数，训练的超参数等）作为函数的输入，而`run_from_yaml`则从用户提供的 YAML（一种方便人类阅读的数据序列化格式，通常用于使用标记语言的配置文件）配置文件中读取程序运行所需的参数。

此外，若用户不愿意详细地设定模型训练的各种细节，如训练轮次、学习率、学习率调度器等，只想快速地运行内置的模型程序，FaKnow 也提供了更简单的方式。用户只需传入数据集等少量必要参数，对于模型初始化的参数和学习率等超参数，FaKnow 将会采用论文开源代码中的相关取值作为默认值。

另外，`run`函数的源代码本身对用户而言也是一个很好的运行库中各种集成模型的示例。其中的代码特意呈若璞玉，未经额外抽象封装，以便研究人员快速迭代魔改而不致溺于各种抽象接口和文件跳转之中。

下文的三个代码片段展示了如何使用`run`和`run_from_yaml`函数运行 FaKnow 中的模型。如数据模块4.2.1节所述，FaKnow 以 JSON 格式的文件作为数据集的输入。代码片段4.1展示了一个示例数据集，其中仅包含一个样本。数据集文件中的每个样本作为一个 JSON 对象，包含三个字段：`text`表示帖子的文本内容，`image`表示帖子图片的文件路径，`label`表示样本的标签。

[10]https://npurg.github.io/FaKnowDoc/

代码片段 4.1　JSON 格式的数据集文件

```
[
    {
        ”text”: ”this is a sentence.”,
        ”image”: ”news.jpg”,
        ”label”: 0
    }
]
```

代码片段4.2展示了一个 YAML 格式的配置文件，用户可以在其中自行编写程序所需参数的配置信息，供`run_from_yaml`函数读取并运行 FaKnow 中的模型。示例配置文件中包含 3 个字段：`train_path`、`validate_path`和`test_path`，分别表示训练集、验证集和测试集的文件路径。

代码片段 4.2　YAML 格式的配置文件 spotfake.yaml

```
train_path: train.json
validate_path: val.json
test_path: test.json
```

代码片段4.3展示了`run`和`run_from_yaml`函数的运行方式。在示例代码中，这两个函数都指定使用的模型名称为 spotfake（该模型已在 2.1.1 节研究现状中论述过）。`run`函数通过关键字参数（kargs, keyword arguments）指定要使用的数据集 JSON 文件的路径，它们分别为`./train.json`、`./val.json`和`./test.json`。而`run_from_yaml`函数则需要指定 YAML 配置文件路径，即`spotfake.yaml`，YAML 文件中同样指明了 JSON 数据文件的路径。在此例中，两个函数均并未指定验证集数据文件路径以及诸如学习率之类的其他超参数，因此 FaKnow 库将会默认采用 SpotFake 论文发布的代码中的超参数来训练和测试此模型。

代码片段 4.3　分别使用`run`和`run_from_yaml`函数运行模型

```
# run函数：通过指定的数据路径和默认超参数运行spotfake模型
kargs = {’train_path’: ’./train.json’, ’validate_path’: ’./val.json’, ’
    test_path’: ’./test.json’}
run(model=’spotfake’, **kargs)
# run_from_yaml函数：通过指定的YAML配置文件运行spotfake模型
run_from_yaml(model=’spotfake’, config_path=’spotfake.yaml’)
```

图4-9展示了代码片段4.3运行后得到的 SpotFake 前两轮训练日志记录，该日志信息在输出到控制台时会一并同步保存到本地文件中。每条日志信息都包含了记录时间戳、日志级别、日志内容等信息，方便用户在训练过程中随时查看。可以发现该日志首先输出了 TensorBoard 可视化文件路径与日志文件路径，再显示了训练集和验证集中的样本数量分别为 11715 和 1009，然后逐步输出了每一轮训练的耗时（training time）、损失值（training loss）、在验证集上的准确率（accuracy）等指标信息。不难发现，程序还会比较每轮训练的模型在验证集上的性能。当第 1 轮（epoch=[0/19]）比第 2 轮（epoch=[1/19]）在验证集上的准确率更高时，程序会记录下最佳的训练轮次（best epoch）和最高指标

值（best score）。同时该模型的参数也会被保存至本地，以便用户在训练结束后选择最优的模型参数进行推理。

```
2024-01-08 22:12:11,154 - INFO - Tensorboard log is saved in tb_logs/SpotFake-2024-01-08-22_12_11
2024-01-08 22:12:11,154 - INFO - log file is saved in logs/tb_logs/SpotFake-2024-01-08-22_12_11.log

2024-01-08 22:12:11,154 - INFO - training data size=11715
2024-01-08 22:12:11,154 - INFO - validation data size=1009
2024-01-08 22:12:11,154 - INFO - ----start training-----

2024-01-08 22:12:11,154 - INFO - epoch=[0/19]
2024-01-08 22:13:42,909 - INFO - training time=1m31s
2024-01-08 22:13:42,909 - INFO - training loss : loss=0.109055
2024-01-08 22:13:51,843 - INFO - model is saved in /root/autodl-tmp/project/save/SpotFake-2024-01-08-22_12_11.pth
2024-01-08 22:13:51,844 - INFO - validation result : accuracy=0.699703    precision=0.797872    recall=0.643225    f1=0.712251
2024-01-08 22:13:51,844 - INFO - current score : 0.699703, best score : 0.699703, best epoch : 0
2024-01-08 22:13:51,844 - INFO - epoch=[1/19]
2024-01-08 22:15:24,290 - INFO - training time=1m32s
2024-01-08 22:15:24,290 - INFO - training loss : loss=0.028001
2024-01-08 22:15:31,818 - INFO - validation result : accuracy=0.637265    precision=0.737418    recall=0.578045    f1=0.648077
2024-01-08 22:15:31,818 - INFO - current score : 0.637265, best score : 0.699703, best epoch : 0
```

图 4-9　SpotFake 训练日志记录

图4-10则展示了代码片段4.3运行后得到的 SpotFake 模型的 TensorBoard 训练曲线记录，包括训练损失，验证集的准确率、精确率、召回率和 f1 分数四项指标随训练轮次的变化情况。从图中可以看出，随着训练轮次的增加，训练损失逐渐降低，而验证集上的准确率、精确率、召回率和 f1 分数逐渐提高，这代表着模型在训练过程中逐渐收敛，性能逐渐提升。但是由于 SpotFake 只是简单地拼接了图片模态和文本模态的特征，故而即便在损失已经降到很低时，模型各项指标也不高，准确率甚至不足 80%，这也是这个算法自身局限性导致的。

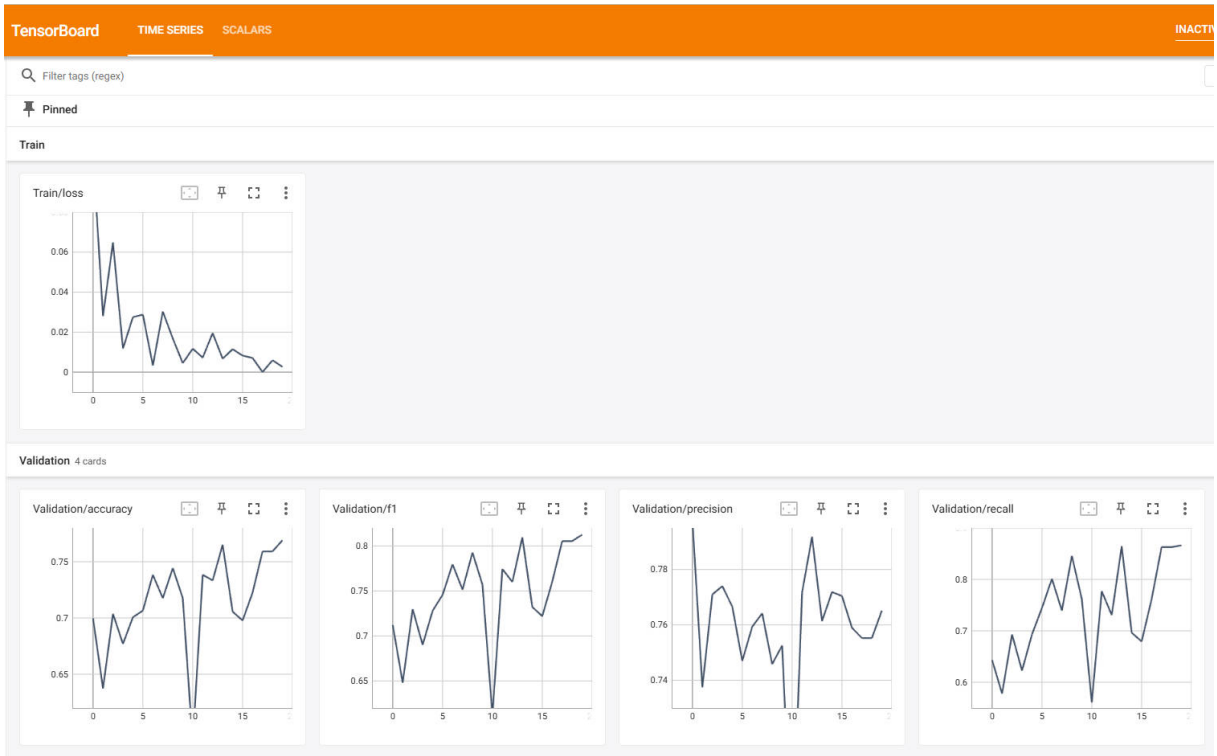


图 4-10　TensorBoard 训练曲线记录

(2) 自行训练

代码片段 4.4　从底层自行构建训练模型的脚手架

```
# 1.1 分割数据集
tokenizer = TokenizerForBert(max_len=170, bert='bert-base-uncased')
ratio = [0.7, 0.3]
train_set, test_set = split_data('data.json', tokenizer, ['text'],
    transform_spotfake, ['image'], ratio)
# 1.2 生成对应的数据加载器
train_loader = DataLoader(train_set, batch_size=64, shuffle=True)
test_loader = DataLoader(test_set, batch_size=64, shuffle=False)
# 2. 加载模型
model = SpotFake(bert='bert-base-uncased')
# 3. 指定评估指标
evaluator = Evaluator(metrics=['accuracy', 'precision', 'recall', 'f1'])
# 4. 初始化训练器
optimizer = torch.optim.Adam(params=model.parameters(), lr=5e-5)
trainer = BaseTrainer(model, evaluator, optimizer)
# 5. 训练
trainer.fit(train_loader, num_epochs=50)
# 6. 测试
test_result = trainer.evaluate(test_loader)
```

如代码片段4.4所示，如果用户想自己动手设置模型训练的每一处细节，可以使用 FaKnow 库从头开始编写完整的模型训练和评估的代码。具体操作包括以下几个步骤：

a. **准备数据**：用户需要为数据集准备好 PyTorch 中的数据加载器`Dataloader`。FaKnow 为内置的模型提供了所需的全部`Dataset`数据集类以及丰富多样的数据处理函数（比如文本分词，图片转换等），用户也可以使用自定义的数据集类来满足特定需求。此例使用了4.2.1节数据模块中提到的多模态数据集`MultiModalDataset`，并通过 FaKnow 中内置的`TokenizerForBert`生成了一个指定文本最大长度为 170，按不区分大小写的英文 BERT（bert-based-uncased）进行分词操作的分词器。该分词器将作为`tokenize`函数用于 JSON 数据文件中的`text`字段处理。而对于图片字段`image`，代码中则调用 FaKnow 内置的`transform_spotfake`函数来读取图片像素值。随后`split_data`函数对`data.json`文件中的数据按 7:3 的比例划分为训练集和测试集，并指定数据批量大小为 64，为它们分别生成对应的数据加载器`train_loader`和`test_loader`。

b. **加载模型**：这一步需要加载待训练的模型，FaKnow 提供了多种内置模型供用户选择。本例中实例化了一个 SpotFake 模型，`bert-base-uncased`是其中使用的预训练 BERT 模型的名称。

c. **指定评估指标**：用户需要在这一步中指定用于模型评估的指标，并生成对应的评估器。示例代码中使用了 FaKnow 内置的准确率 accuracy、精确率 precision、召回率 recall 和 f1 分数作为评估指标。

d. **初始化训练器**：在这一步中，用户需要初始化用于模型训练的训练器，选择合适的优化算法，设置学习率等超参数。用户在这一步还可以为训练器配置学习

率调度器、梯度裁剪、早停等高级设置。本例中使用的训练器为 FaKnow 内置的`BaseTrainer`，功能齐全，适用于大多数训练场景，用户也可以根据需要自定义训练器。同时代码中还指定了训练的优化器为学习率为 0.00005 的 Adam 算法。

e. **训练**：这一步中训练器会调用自身的`fit`方法来执行模型的训练和验证，并将训练结果通过日志文件和 Tensorboard 可视化的形式保存至本地。如果用户在准备数据时还额外指定了验证集数据加载器，训练器同样会在每轮训练结束后在验证集上评估模型性能。在本例中，训练器将会在步骤 a 生成的训练集数据加载器上持续训练 50 轮。

f. **测试**：这一步中训练器会调用自身的`evaluate`方法完成模型推理，并根据步骤 c 设置的指标来评估模型在测试集上的表现。本例中，之前通过评估器指定的 accuracy、precision、recall、f1 score 四项指标结果将作为方法的返回值存储在`test_result`变量中。

4.4.2 开发新模型

代码片段 4.5　基于 FaKnow 开发新模型

```
class AlLNet(AbstractModel):
    def __init__(self):
        # 1 定义模型结构
        self.bert_encoder = BertEncoder()
        self.text_cnn_layer = TextCNNLayer()
        self.resnet_encoder = ResNetEncoder()
        self.image_moco = MoCo()
        self.shared_head = nn.Linear()
        ...
    def forward(self, token, mask, image):
        # 2 前向传播，返回模型最后的输出
        ...
        return text_out, image_out, logits
    def calculate_loss(self, batch_data):
        # 3 损失计算，包括分类损失和对比学习损失
        text_out, image_out, logits = self.forward(token, mask, image)
        ...
        total = clf_loss + info_nce
        losses = {'total': total, 'class': clf_loss, 'contrast': info_nce}
        return losses
    def predict(self, batch_data) -> Tensor:
        # 4 推理预测，加权综合文本和图片预测结果
        text_out, image_out, _ = self.forward(token, mask, image)
        ...
        return text_out * text_weight + image_out * image_weight
```

用户在开发新的算法时，可以直接在 FaKnow 的基础上快速搭建出一个可用的模型，该模型的代码结构符合 FaKnow 的 API 接口规范，可以方便地与 FaKnow 的数据处理和训练框架进行对接，以便更快地实现新模型的训练和评估。如4.2.2节所述，用户开发的新模型都需要继承 FaKnow 内置的`AbstractModel`父类，然后依次重写类中相应的方法。为了更好地说明 FaKnow 算法库对于开发新模型的支持，以及它与本文的第一项工

作（基于单模态交替的多模态虚假信息方法 AlLNet）之间的关联，代码片段4.5展示了基于 FaKnow 的算法框架来开发 AlLNet 模型的示例代码，下文将结合此示例代码对开发新模型的步骤进行详细说明。值得注意的是，为了方便说明，下文的示例并非完整的 AlLNet 模型代码，而是一个简化版本，仅包含了模型的主要结构和逻辑（包含每个方法的输入与输出），省去了一些具体的实现细节（如模型的初始化、前向传播执行过程以及损失计算等）。完整具体的 AlLNet 模型代码详见本文附录。

a. **重写`init`方法：**FaKnow 中所有的模型都间接继承自 PyTorch 中的`nn.Module`，因此重写`init`和`forward`方法的逻辑与直接使用 PyTorch 时的方式一样。示例代码在`init`方法定义了模型结构，初始化了`bert_encoder`（用于获得词向量 T 的 BERT 模型）、`text_cnn_layer`（TextCNN 结构的文本特征编码器 f_t）、`resnet_encoder`（用于获得图像基本特征 V 的 ResNet 模型）、`image_moco`（对比学习模块，包含图片特征编码器 f_v 和键编码器 $f_{v_{k_+}}$ 两部分结构）、以及`shared_head`（图文共享分类层 h）共四个成员变量，它们分别对应 AlLNet 模型的各个组件。其中`BertEncoder`、`TextCNNLayer`、`ResNetEncoder` 和`MoCo`均为 FaKnow 中内置的常用神经网络组件（4.2.2节模型模块已作说明）。

b. **重写`forward`方法：**该方法定义了模型的前向传播逻辑，即输入数据经过模型的计算后得到输出的过程。AlLNet 模型接收文本特征 W（对应代码中的`token`和`mask`，二者分别为经过分词函数`tokenize`得到的词元序列和掩码序列）、图像特征 I（对应代码中的`img`，是由`transform`函数读取得到的图片像素值）作为输入。在前向传播过程中，输入的数据会经过上一步中定义的各个组件，得到模型最终的输出（此处省略，详见附录）。最后，`forward`方法会返回`text_out`（文本模态的预测结果 $\hat{y}_t$）、`image_out`（图片模态的预测结果 $\hat{y}_v$）以及`logits`（对比学习中拼接后的正负样本相似度矩阵，详见 3.3.1 节中关于 InfoNCE 损失的实现细节）

c. **重写`calculate_loss`方法：**用户需要在这个方法中编写计算损失的逻辑代码，调用`forward`获取模型最后一层输出，并根据样本的真实标签计算损失，用于反向传播中的参数更新。如果最终损失由多个损失组成，也可以将它们组成字典`Dict`一并返回。在示例代码中，方法的输入为基于字典的批量数据`batch_data`（4.2.1节已作详细说明），其中包含词元序列`token`、掩码序列`mask`、图片像素值`image`以及样本标签。程序在完成前向传播后，会计算样本分类损失`clf_loss`和对比学习损失`info_nce`（此处省略，详见附录），并将二者相加得到总损失`total`，最后将损失结果组成字典`losses`返回。

d. **重写`predict`方法：**用户需要在这个方法中编写模型推理的代码逻辑，根据`forward`的输出，返回样本分别被分为真、假信息的概率。在示例中，程序同样地接收输入的字典批量数据，然后获得模型的最终输出`text_out`和 `image_out`（推理时无需对比学习的输出结果`logits`）。随后，模型会根据文本模态和图片模态输出的熵得出各自

权重`text_weight`（文本权重 λ_t）和`image_weight`（图片权重 λ_v）。最后，`predict`方法会返回这两个模态预测结果的加权综合，作为最终分类结果 $\hat{y}$。

4.5 本章小结

本章提出了首个虚假信息检测算法库 FaKnow，其中集成了几十余种虚假信息检测模型，包含了基于内容和基于社交上下文两个主流领域的常见算法。同时，FaKnow 也为这些虚假信息检测算法组织了统一的框架，覆盖了数据处理、模型训练和评估等一系列流程，并提供了如可视化等丰富的辅助功能。这些功能和基于 PyTorch 的逻辑行为使得 FaKnow 非常简单易上手，给用户带来了友好的交互体验。此外，统一设计规范好的 API 也保障了这个库的良好的拓展性，允许用户只需编写少量代码就可以自定义各种功能来应对不同情况。目前 FaKnow 已作为 Python 软件包发布至 PyPI（Python Package Index）[11]，供研究者使用。FaKnow 的发布对于虚假信息检测领域的研究有着重要的意义，有利于虚假信息检测相关研究的大一统，使得后续的研究者能够方便地复现已有的算法或者开发新的模型。

[11]https://pypi.org/project/faknow/

第 5 章　总结与展望

5.1 本文总结

随着移动互联网的快速发展，社交媒体平台在人们的生活中扮演着越来越重要的角色。网民们喜欢在自己的社交帐号上发布各种帖子，包括文字、图片、视频等信息，内容涉及日常生活、时事政治、体育报道、娱乐八卦等不同领域。但是这些在互联网中泛滥的帖子内容质量参差不齐，其中充斥着不少虚假信息。虚假信息的存在与传播不仅会误导网民，还会对社会稳定和国家安全造成威胁。虽然微博、微信等社交媒体平台已经采取了一系列措施来打击虚假信息，但是虚假信息的传播速度仍然很快，很多虚假信息在短时间内就会被大量转发，给社会带来不小的危害。人工审核的方式虽然可以一定程度上识别虚假信息，但是效率低下，无法满足社交媒体平台海量信息的审核需求。因此，如何准确快速地自动化识别社交媒体中的虚假信息成为了一个重要的研究课题。本文针对社交媒体中的虚假信息检测问题，提出了多模态检测算法和开源算法库这两项工作。

本文首先提出了一种基于单模态交替学习的多模态虚假信息检测方法 AlLNet，从基于帖子内容的角度出发，主要用于社交媒体中那些包含图片和文字的多模态帖子的虚假信息检测。相较于传统的多模态虚假信息检测算法，AlLNet 在模型训练和推理阶段分别采用交替学习和加权融合这两种不同的策略，既避免了模态惰性又能兼顾不同模态的贡献比重。在每一轮训练过程中，模型将图片和文本模态分开，依次单独进行单模态分类，从而完成两种模态的交替式学习。此外模型中还使用了图文共享的分类头，以期望能学到跨模态交互的信息，并引入梯度修正机制来避免每轮交替学习过程中遗忘上一次学习到的模态特征。而在推理阶段，模型则通过每个模态输出的熵值计算权重，将不同模态的输出进行多模态加权融合，从而得到最终的分类预测结果。后续在不同数据集上的一系列实验也证明了 AlLNet 相较于其他基线模型的优越性，具有更好的分类性能和更快的推理速度，并且能应对真实世界中的模态缺失问题。

而针对虚假信息检测任务中复现模型困难、存在重复性工作、缺乏统一基准等问题，本文设计并实现了一个基于 PyTorch 实现的虚假信息检测算法库 FaKnow。该算法库中收集了几十余种自深度学习流行以来发表在顶级学术会议或期刊上的虚假信息检测算法模型，包含了基于内容和基于社交上下文两种类型的检测方法，覆盖领域适应、多模态、图神经网络等多种技术手段。更重要的是，FaKnow 还提供了一个统一的模型框架，能覆盖模型训练和推理的全过程，既规范了内部接口，又保证整个系统能具有良好的拓展性。此外，本文也通过一系列与原文对比的复现性实验验证了 FaKnow 中模型的有效性，并提供了使用示例来说明 FaKnow 使用的便捷性。FaKnow 作为为虚假信息检测领域的首个开源算法库，简化了代码开发流程，省去了大量重复性工作，能帮助研究者快速复现和比较不同的算法模型。同时它还为虚假信息检测领域的研究提供一个统

一的基准，有助于推动虚假信息检测领域的研究进展。

5.2 未来展望

尽管本文提出的多模态虚假信息检测方法 AlLNet 和虚假信息检测算法库 FaKnow 在虚假信息检测领域取得了一定的成果，但是社交平台中的虚假信息检测仍然是一个复杂的应用问题，还有很多工作有待进一步研究和探索。图文的多模态交互仍然面临着许多挑战：如何更好地学习到不同模态之间的关联信息，如何更好地处理模态缺失问题等等，同时这一研究领域也在飞速发展和进步。未来可以从以下几个方面继续优化和完善本文的工作。

- **多模态虚假信息检测算法的进一步研究**：本文提出的 AlLNet 方法虽然通过单模态交替学习的方式取得了不错的效果，但是它对多模态特征交互的处理方式较为简单，仅采用了共享的分类头，学习到的跨模态交互信息有限。作为自然语言处理和计算机视觉的交叉领域，多模态虚假信息检测还有很多未被挖掘的研究空间。一方面，可以将该算法的文本预测分支和图片预测分支中的模型更换为其他更前沿、更好的单模态预测算法；另一方面，也可以考虑引入更多的跨模态交互机制，如图文对抗学习等，以期望能更好地学习到不同模态之间的关联信息。同时，随着短视频等新型媒体形式的兴起，未来还可以考虑将 AlLNet 方法扩展到更多的模态，如音频、视频等，以期望能更好地处理多模态虚假信息检测问题。此外，大语言模型（Large Language Model）也正逐步成为研究热点，虚假信息检测也可以与之结合，既可以利用大语言模型作为预训练模型来提取文本特征，也可以利用其生成能力来生成虚假信息，增加数据集的多样性，以提升模型的泛化能力。
- **虚假信息检测算法库的持续开发**：为了使 FaKnow 能够与时俱进，适应不断变化更新的虚假信息检测的研究，我们还需要对它进行长期维护和更新。在算法丰富性上，未来可以不断向其中加入新的虚假信息检测模型，尤其是近年发表的具有高引用数、比较有代表性的工作，并增加其中算法类型和技术手段的多样性。在工程技术上，也可以不断向其中加入新的功能来提升框架的便捷可用性，如分布式训练、超参数搜索、训练断点恢复等，并将算法库与 wandb[1]这类记录实验超参数和结果的工具进行集成。我们在鼓励用户提出宝贵的意见和建议的同时，也欢迎世界各地其他开发者对开源在 GitHub 上的代码进行维护和功能开发，共同完善 FaKnow，以期望能够更好地服务虚假信息检测领域的研究者。此外，还可以考虑将它交由如 Apache 软件基金会等组织孵化，以便广大的开源社区更好地参与进来。

[1]https://wandb.ai/site

参考文献

[1] 中国互联网络信息中心. 第 54 次《中国互联网络发展状况统计报告》[Z]. 2024.

[2] 张志勇, 荆军昌, 李斐, 等. 人工智能视角下的在线社交网络虚假信息检测、传播与控制研究综述[J]. 计算机学报, 2021, 44(11): 2261-2282.

[3] Vosoughi S, Roy D, Aral S. The spread of true and false news online[J]. Science, 2018, 359(6380): 1146-1151. https://www.science.org/doi/abs/10.1126/science.aap9559.

[4] Bessi A, Ferrara E. Social bots distort the 2016 u.s. presidential election online discussion [J]. First Monday, 2016, 21(11). https://firstmonday.org/ojs/index.php/fm/article/view/7090. DOI: 10.5210/fm.v21i11.7090.

[5] Ferrara E. Disinformation and social bot operations in the run up to the 2017 french presidential election[J]. First Monday, 2017, 22(8). https://firstmonday.org/ojs/index.php/fm/article/view/8005. DOI: 10.5210/fm.v22i8.8005.

[6] 新华社. 世卫组织：关于新冠病毒的传言与事实[Z]. 2020.

[7] 新华网. 中央网信办部署开展 2024 年“清朗”系列专项行动[Z]. 2024.

[8] Mikolov T, Chen K, Corrado G, et al. Efficient estimation of word representations in vector space[C]. Bengio Y, LeCun Y. 1st International Conference on Learning Representations, ICLR 2013, Scottsdale, Arizona, USA, May 2-4, 2013, Workshop Track Proceedings. 2013. http://arxiv.org/abs/1301.3781.

[9] Devlin J, Chang M W, Lee K, et al. Bert: Pre-training of deep bidirectional transformers for language understanding[C]. Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers). Minneapolis, Minnesota: Association for Computational Linguistics, 2019: 4171-4186. https://aclanthology.org/N19-1423. DOI: 10.18653/v1/N19-1423.

[10] Ma J, Gao W, Mitra P, et al. Detecting rumors from microblogs with recurrent neural networks[C]. IJCAI’16: Proceedings of the Twenty-Fifth International Joint Conference on Artificial Intelligence. New York, New York, USA: AAAI Press, 2016: 3818-3824.

[11] Yu F, Liu Q, Wu S, et al. A convolutional approach for misinformation identification[C]. Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence. Melbourne, Australia: International Joint Conferences on Artificial Intelligence Organization, 2017: 3901-3907. DOI: 10.24963/ijcai.2017/545.

[12] Zhu Y, Sheng Q, Cao J, et al. Generalizing to the future: Mitigating entity bias in fake news

detection[C]. SIGIR ’22: Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval. New York, NY, USA: Association for Computing Machinery, 2022: 2120-2125. DOI: 10.1145/3477495.3531816.

[13] Nan Q, Cao J, Zhu Y, et al. Mdfend: Multi-domain fake news detection[C]. CIKM ’21: Proceedings of the 30th ACM International Conference on Information & Knowledge Management. New York, NY, USA: Association for Computing Machinery, 2021: 3343-3347. DOI: 10.1145/3459637.3482139.

[14] Qi P, Cao J, Yang T, et al. Exploiting multi-domain visual information for fake news detection[C]. 2019 IEEE International Conference on Data Mining (ICDM). 2019: 518-527. DOI: 10.1109/ICDM.2019.00062.

[15] Singhal S, Shah R R, Chakraborty T, et al. Spotfake: A multi-modal framework for fake news detection[C]. 2019 IEEE Fifth International Conference on Multimedia Big Data (BigMM). 2019: 39-47. DOI: 10.1109/BigMM.2019.00-44.

[16] Simonyan K, Zisserman A. Very deep convolutional networks for large-scale image recognition[C]. Bengio Y, LeCun Y. 3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings. 2015. http://arxiv.org/abs/1409.1556.

[17] Wang Y, Ma F, Jin Z, et al. Eann: Event adversarial neural networks for multi-modal fake news detection[C]. Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining. 2018.

[18] Kim Y. Convolutional neural networks for sentence classification[C]. Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP). Doha, Qatar: Association for Computational Linguistics, 2014: 1746-1751. https://aclanthology.org/D14-1181. DOI: 10.3115/v1/D14-1181.

[19] Zhou X, Wu J, Zafarani R. : Similarity-aware multi-modal fake news detection[C]. Advances in Knowledge Discovery and Data Mining: 24th Pacific-Asia Conference, PAKDD 2020, Singapore, May 11-14, 2020, Proceedings, Part II. Berlin, Heidelberg: Springer-Verlag, 2020: 354-367. https://doi.org/10.1007/978-3-030-47436-2_27.

[20] Vinyals O, Toshev A, Bengio S, et al. Show and tell: Lessons learned from the 2015 mscoco image captioning challenge[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2017, 39(4): 652-663. DOI: 10.1109/TPAMI.2016.2587640.

[21] Khattar D, Goud J S, Gupta M, et al. Mvae: Multimodal variational autoencoder for fake news detection[C]. WWW ’19: The World Wide Web Conference. New York, NY, USA: Association for Computing Machinery, 2019: 2915–2921. https://doi.org/10.1145/

3308558.3313552.

[22] Wu Y, Zhan P, Zhang Y, et al. Multimodal fusion with co-attention networks for fake news detection[C]. Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021. Online: Association for Computational Linguistics, 2021: 2560-2569. DOI: 10.18653/v1/2021.findings-acl.226.

[23] Qian S, Wang J, Hu J, et al. Hierarchical multi-modal contextual attention network for fake news detection[C]. SIGIR '21: Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval. New York, NY, USA: Association for Computing Machinery, 2021: 153-162. DOI: 10.1145/3404835.3462871.

[24] Aneja S, Bregler C, Niessner M. Cosmos: Catching out-of-context image misuse using self-supervised learning[J]. Proceedings of the AAAI Conference on Artificial Intelligence, 2023, 37(12): 14084-14092. DOI: 10.1609/aaai.v37i12.26648.

[25] Qi P, Cao J, Li X, et al. Improving fake news detection by using an entity-enhanced framework to fuse diverse multimodal clues[C]. MM '21: Proceedings of the 29th ACM International Conference on Multimedia. New York, NY, USA: Association for Computing Machinery, 2021: 1212-1220. DOI: 10.1145/3474085.3481548.

[26] Lao A, Zhang Q, Shi C, et al. Frequency spectrum is more effective for multimodal representation and fusion: A multimodal spectrum rumor detector[J]. Proceedings of the AAAI Conference on Artificial Intelligence, 2024, 38(16): 18426-18434. DOI: 10.1609/aaai.v38i16.29803.

[27] Chen Y, Li D, Zhang P, et al. Cross-modal ambiguity learning for multimodal fake news detection[C]. Proceedings of the ACM Web Conference 2022. Virtual Event, Lyon France: ACM, 2022: 2897-2905. DOI: 10.1145/3485447.3511968.

[28] Ying Q, Hu X, Zhou Y, et al. Bootstrapping multi-view representations for fake news detection[C]. AAAI'23/IAAI'23/EAAI'23: Proceedings of the Thirty-Seventh AAAI Conference on Artificial Intelligence and Thirty-Fifth Conference on Innovative Applications of Artificial Intelligence and Thirteenth Symposium on Educational Advances in Artificial Intelligence. AAAI Press, 2023. https://doi.org/10.1609/aaai.v37i4.25670.

[29] Dong Y, He D, Wang X, et al. A generalized deep markov random fields framework for fake news detection[C]. Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence, IJCAI-23. International Joint Conferences on Artificial Intelligence Organization, 2023: 4758-4765. https://doi.org/10.24963/ijcai.2023/529.

[30] Lu Y J, Li C T. Gcan: Graph-aware co-attention networks for explainable fake news detection on social media[C]. Proceedings of the 58th Annual Meeting of the Association

for Computational Linguistics. Online: Association for Computational Linguistics, 2020: 505-514. https://aclanthology.org/2020.acl-main.48. DOI: 10.18653/v1/2020.acl-main.48.

[31] Bian T, Xiao X, Xu T, et al. Rumor detection on social media with bi-directional graph convolutional networks[C]. Proceedings of the AAAI Conference on Artificial Intelligence: volume 34. 2020: 549-556. https://ojs.aaai.org/index.php/AAAI/article/view/5393. DOI: 10.1609/aaai.v34i01.5393.

[32] Dou Y, Shu K, Xia C, et al. User preference-aware fake news detection[C]. SIGIR '21: Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval. New York, NY, USA: Association for Computing Machinery, 2021: 2051-2055. https://doi.org/10.1145/3404835.3462990.

[33] Nguyen V H, Sugiyama K, Nakov P, et al. Fang: Leveraging social context for fake news detection using graph representation[C]. CIKM '20: Proceedings of the 29th ACM International Conference on Information & Knowledge Management. New York, NY, USA: Association for Computing Machinery, 2020: 1165-1174. https://dl.acm.org/doi/10.1145/3340531.3412046.

[34] Hamilton W L, Ying R, Leskovec J. Inductive representation learning on large graphs [C]. NIPS'17: Proceedings of the 31st International Conference on Neural Information Processing Systems. Red Hook, NY, USA: Curran Associates Inc., 2017: 1025-1035.

[35] Zhang X, Cao J, Li X, et al. Mining dual emotion for fake news detection[C]. WWW '21: Proceedings of the Web Conference 2021. New York, NY, USA: Association for Computing Machinery, 2021: 3465-3476. https://dl.acm.org/doi/10.1145/3442381.3450004.

[36] Wei L, Hu D, Zhou W, et al. Towards propagation uncertainty: Edge-enhanced bayesian graph convolutional networks for rumor detection[C]. Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers). Online: Association for Computational Linguistics, 2021: 3845-3854. https://aclanthology.org/2021.acl-long.297. DOI: 10.18653/v1/2021.acl-long.297.

[37] Pedregosa F, Varoquaux G, Gramfort A, et al. Scikit-learn: Machine learning in python [J]. J. Mach. Learn. Res., 2011, 12: 2825-2830.

[38] Hall M, Frank E, Holmes G, et al. The weka data mining software: an update[J]. SIGKDD Explor. Newsl., 2009, 11(1): 10–18. https://doi.org/10.1145/1656274.1656278.

[39] Contributors M. MMCV: OpenMMLab computer vision foundation[EB]. 2018. https://github.com/open-mmlab/mmcv.

[40] Chen K, Wang J, Pang J, et al. Mmdetection: Open mmlab detection toolbox and benchmark[EB]. 2019. https://arxiv.org/abs/1906.07155.

[41] Contributors M. MMSegmentation: Openmmlab semantic segmentation toolbox and benchmark[EB]. 2020. https://github.com/open-mmlab/mmsegmentation.

[42] Contributors M. MMEngine: Openmmlab foundational library for training deep learning models[EB]. 2022. https://github.com/open-mmlab/mmengine.

[43] Zhao W X, Mu S, Hou Y, et al. Recbole: Towards a unified, comprehensive and efficient framework for recommendation algorithms[C]. CIKM '21: Proceedings of the 30th ACM International Conference on Information & Knowledge Management. New York, NY, USA: Association for Computing Machinery, 2021: 4653-4664. https://doi.org/10.1145/3459637.3482016.

[44] Zhou X. Mmrec: Simplifying multimodal recommendation[C]. MMAsia '23 Workshops: Proceedings of the 5th ACM International Conference on Multimedia in Asia Workshops. New York, NY, USA: Association for Computing Machinery, 2023. https://doi.org/10.1145/3611380.3628561.

[45] Liu Z, Liu Q, Chen J, et al. pykt: A python library to benchmark deep learning based knowledge tracing models[C]. Koyejo S, Mohamed S, Agarwal A, et al. Advances in Neural Information Processing Systems: volume 35. Curran Associates, Inc., 2022: 18542-18555.

[46] Lan Y, Wang L, Zhang Q, et al. Mwptoolkit: An open-source framework for deep learning-based math word problem solvers[J]. Proceedings of the AAAI Conference on Artificial Intelligence, 2022, 36(11): 13188-13190. DOI: 10.1609/aaai.v36i11.21723.

[47] Wolf T, Debut L, Sanh V, et al. Transformers: State-of-the-art natural language processing[C]. Liu Q, Schlangen D. Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations. Online: Association for Computational Linguistics, 2020: 38-45. https://aclanthology.org/2020.emnlp-demos.6. DOI: 10.18653/v1/2020.emnlp-demos.6.

[48] Vaswani A, Shazeer N, Parmar N, et al. Attention is all you need[C]. NIPS'17: Proceedings of the 31st International Conference on Neural Information Processing Systems. Red Hook, NY, USA: Curran Associates Inc., 2017: 6000-6010.

[49] Wang J, Jiang J, Jiang W, et al. Libcity: An open library for traffic prediction[C]. SIGSPATIAL '21: Proceedings of the 29th International Conference on Advances in Geographic Information Systems. New York, NY, USA: Association for Computing Machinery, 2021: 145-148. https://doi.org/10.1145/3474717.3483923.

[50] Han H, Zhao T, Yang C, et al. Openhgnn: An open source toolkit for heterogeneous graph

neural network[C]. CIKM ’22: Proceedings of the 31st ACM International Conference on Information & Knowledge Management. New York, NY, USA: Association for Computing Machinery, 2022: 3993–3997. https://doi.org/10.1145/3511808.3557664.

[51] Singhal S, Kabra A, Sharma M, et al. Spotfake+: A multimodal framework for fake news detection via transfer learning (student abstract)[C]. Proceedings of the AAAI Conference on Artificial Intelligence: volume 34. 2020: 13915-13916. DOI: 10.1609/aaai.v34i10.7230.

[52] He K, Fan H, Wu Y, et al. Momentum contrast for unsupervised visual representation learning[C]. 2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). 2020: 9726-9735. DOI: 10.1109/CVPR42600.2020.00975.

[53] van den Oord A, Li Y, Vinyals O. Representation Learning with Contrastive Predictive Coding: arXiv:1807.03748[M]. arXiv, 2019. DOI: 10.48550/arXiv.1807.03748.

[54] He K, Zhang X, Ren S, et al. Deep residual learning for image recognition[C]. 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR). 2016: 770-778. https://ieeexplore.ieee.org/document/7780459. DOI: 10.1109/CVPR.2016.90.

[55] Shu K, Mahudeswaran D, Wang S, et al. Fakenewsnet: A data repository with news content, social context and dynamic information for studying fake news on social media[J]. arXiv preprint arXiv:1809.01286, 2018.

[56] Deng J, Dong W, Socher R, et al. Imagenet: A large-scale hierarchical image database [C]. 2009 IEEE Conference on Computer Vision and Pattern Recognition. 2009: 248-255. DOI: 10.1109/CVPR.2009.5206848.

[57] Zheng J, Zhang X, Guo S, et al. Mfan: Multi-modal feature-enhanced attention networks for rumor detection[C]. Thirty-First International Joint Conference on Artificial Intelligence: volume 3. 2022: 2413-2419. DOI: 10.24963/ijcai.2022/335.

[58] Zhu Y, Sheng Q, Cao J, et al. Generalizing to the future: Mitigating entity bias in fake news detection[C]. SIGIR ’22: Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval. New York, NY, USA: Association for Computing Machinery, 2022: 2120-2125. https://dl.acm.org/doi/10.1145/3477495.3531816.

[59] Zhu Y, Sheng Q, Cao J, et al. Memory-guided multi-view multi-domain fake news detection[J]. IEEE Transactions on Knowledge and Data Engineering, 2023, 35(7): 7178-7191. DOI: 10.1109/TKDE.2022.3185151.

[60] Kipf T N, Welling M. Semi-supervised classification with graph convolutional networks [C]. International Conference on Learning Representations. 2017. https://openreview.net/

forum?id=SJU4ayYgl.

[61] Veličković P, Cucurull G, Casanova A, et al. Graph attention networks[C]. International Conference on Learning Representations. 2018. https://openreview.net/forum?id=rJXMpikCZ.

[62] Monti F, Frasca F, Eynard D, et al. Fake news detection on social media using geometric deep learning[J]. ArXiv, 2019, abs/1902.06673. https://api.semanticscholar.org/CorpusID:62841478.

[63] Han Y, Karunasekera S, Leckie C. Continual learning for fake news detection from social media[C]. Artificial Neural Networks and Machine Learning –ICANN 2021: 30th International Conference on Artificial Neural Networks, Bratislava, Slovakia, September 14–17, 2021, Proceedings, Part II. Berlin, Heidelberg: Springer-Verlag, 2021: 372-384. https://doi.org/10.1007/978-3-030-86340-1_30.

[64] Liu L, Chen J, Cheng Z, et al. Towards trustworthy rumor detection with interpretable graph structural learning[C]. CIKM '23: Proceedings of the 32nd ACM International Conference on Information and Knowledge Management. New York, NY, USA: Association for Computing Machinery, 2023: 4089-4093. https://dl.acm.org/doi/10.1145/3583780.3615228.

[65] Sheng Q, Cao J, Zhang X, et al. Zoom out and observe: News environment perception for fake news detection[C]. Muresan S, Nakov P, Villavicencio A. Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). Dublin, Ireland: Association for Computational Linguistics, 2022: 4543-4556. https://aclanthology.org/2022.acl-long.311. DOI: 10.18653/v1/2022.acl-long.311.

[66] McClish D K. Analyzing a portion of the roc curve[J]. Medical Decision Making: An International Journal of the Society for Medical Decision Making, 1989, 9(3): 190-195. DOI: 10.1177/0272989X8900900307.

[67] Jin Z, Cao J, Guo H, et al. Multimodal fusion with recurrent neural networks for rumor detection on microblogs[C]. MM '17: Proceedings of the 25th ACM International Conference on Multimedia. New York, NY, USA: Association for Computing Machinery, 2017: 795-816. https://dl.acm.org/doi/10.1145/3123266.3123454.

[68] Song C, Yang C, Chen H, et al. Ced: Credible early detection of social media rumors [J]. IEEE Transactions on Knowledge and Data Engineering, 2021, 33(8): 3035-3047. https://ieeexplore.ieee.org/document/8939421. DOI: 10.1109/TKDE.2019.2961675.

[69] Li Y, Jiang B, Shu K, et al. Toward a multilingual and multimodal data repository for covid-19 disinformation[C]. 2020 IEEE International Conference on Big Data (Big Data). Atlanta, GA, USA: IEEE, 2020: 4325-4330. https://ieeexplore.ieee.org/document/9378472/.

DOI: 10.1109/BigData50022.2020.9378472.

[70] Boididou C, Papadopoulos S, Zampoglou M, et al. Detection and visualization of misleading content on twitter[J]. International Journal of Multimedia Information Retrieval, 2018, 7(1): 71-86. https://doi.org/10.1007/s13735-017-0143-x.

[71] Liu X, Nourbakhsh A, Li Q, et al. Real-time rumor debunking on twitter[C]. CIKM '15: Proceedings of the 24th ACM International on Conference on Information and Knowledge Management. New York, NY, USA: Association for Computing Machinery, 2015: 1867–1870. https://doi.org/10.1145/2806416.2806651.

[72] Ma J, Gao W, Wong K F. Detect rumors in microblog posts using propagation structure via kernel learning[C]. Barzilay R, Kan M Y. Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). Vancouver, Canada: Association for Computational Linguistics, 2017: 708-717. https://aclanthology.org/P17-1066. DOI: 10.18653/v1/P17-1066.

[73] Ma J, Gao W, Mitra P, et al. Detecting rumors from microblogs with recurrent neural networks[C]. IJCAI'16: Proceedings of the Twenty-Fifth International Joint Conference on Artificial Intelligence. New York, New York, USA: AAAI Press, 2016: 3818-3824.

[74] Kochkina E, Liakata M, Zubiaga A. Pheme dataset for rumour detection and veracity classification[EB]. 2018. https://figshare.com/articles/dataset/PHEME_dataset_for_Rumour_Detection_and_Veracity_Classification/6392078.

[75] Peng L, Jian S, Kan Z, et al. Not all fake news is semantically similar: Contextual semantic representation learning for multimodal fake news detection[J]. Information Processing & Management, 2024, 61(1): 103564.

附录 A 源代码

本文的 4.4.2 节开发新模型中介绍了如何基于 FaKnow 算法库来开发第 3 章提出的 AlLNet 模型，但是受限于篇幅原因，正文中只给出了部分关键的代码片段进行说明，本附录将给出完整的模型源代码，其中包含完整的模型定义、前向传播、损失计算和推理预测的代码实现，并辅以详细的注释说明，以便读者更好地理解和使用这个算法库。请注意，由于导入模块和类的 import 代码语句在程序中并不重要，因此这里省略了这些导入语句。

代码片段 A.1 基于 FaKnow 开发新模型

```
class AlLNet(AbstractModel):
    def __init__(self, bert: str, queue_size: int, momentum: float, temperature: float):
        super().__init__()
        kernel_sizes = [3, 4, 5]
        kernel_num = 100
        self.embedding_size = kernel_num * len(kernel_sizes) # 表征向量维度

        self.bert_encoder = BertEncoder(bert)
        self.text_cnn_layer = TextCNNLayer(self.bert_encoder.dim, kernel_num, kernel_sizes, nn.ReLU())
        self.resnet_encoder = ResNetEncoder(self.embedding_size)
        # 对比学习模块，其中还额外包含了图片特征编码器即查询编码器encoder_q
        self.image_moco = MoCo(nn.Linear, self.embedding_size, queue_size, momentum, temperature, in_features=2048, out_features=self.embedding_size)

        self.text_norm = nn.BatchNorm1d(self.embedding_size)
        self.image_norm = nn.BatchNorm1d(self.embedding_size)
        self.shared_head = nn.Sequential(
            nn.ReLU(),
            nn.Dropout(0.5),
            nn.Linear(self.embedding_size, 2),
        )

    def forward(self, token: Tensor, mask: Tensor, img: Tensor):
        logits = None # 对比学习输出

        if img is None: # 未传入图片数据
            img_out = None
        else: # 传入图片数据
            image = self.resnet_encoder(img)
            if self.training: # 训练时使用MoCo进行对比学习
                image_k = torch.dropout(image, 0.5, train=True)
                image_embedding, logits = self.image_moco(image, image_k)
            else: # 推理时直接使用查询编码器encoder_q
                image_embedding = self.image_moco.encoder_q(image)
            image_embedding = self.image_norm(image_embedding)
            img_out = self.shared_head(image_embedding)

        if token is None: # 未传入文本数据
            text_out = None
```

```
39          else: # 传入文本数据
40              text = self.bert_encoder(token, mask)
41              text_embedding = self.text_cnn_layer(text)
42              text_embedding = self.text_norm(text_embedding)
43              text_out = self.shared_head(text_embedding)
44
45          return text_out, img_out, logits
46
47      def calculate_loss(self, batch_data: Dict[str, Tensor]):
48          cross_entropy_fn = nn.CrossEntropyLoss()
49          info_nce_loss_fn = InfoNCELoss()
50          y_t, y_v, logits = self.forward(batch_data['text']['token_id'],
                batch_data['text']['mask'], batch_data['image'])
51          if y_t is None: # 未传入文本数据，则计算对比学习损失
52              out = y_v
53              info_nce = info_nce_loss_fn(logits)
54          else: # 否则对比学习损失为0
55              out = y_t
56              info_nce = torch.tensor(0)
57
58          clf_loss = cross_entropy_fn(out, batch_data['label'])
59          total_loss = clf_loss + info_nce # 将分类损失与对比学习损失相加
60          losses = {
61              'total': total_loss,
62              'class': clf_loss,
63              'contrast': info_nce_loss
64          }
65
66          return losses
67
68      def predict(self, batch_data: Dict[str, Tensor]) -> Tensor:
69          y_t, y_v, _ = self.forward(batch_data['text']['token_id'],
                batch_data['text']['mask'], batch_data['image'])
70          text_weight, image_weight = calculate_gating_weights(y_t, y_v)
71          y = y_t * text_weight + y_v * image_weight # 加权综合
72
73          return torch.softmax(y, dim=1)
```

致 谢

借用电视剧《潜伏》中的一句台词开头："时间就像一头野驴，跑起来就不停"。自2020年以来，生活好像就被按下了加速键。我仍能回想起研一入学的情景：前一夜在火车卧铺上的辗转反侧，离开南国初到北方的新奇，顶着烈日骑车逛校园的兴奋，以及因校区太偏而坐了2小时迎新大巴带来的晕车恶心和不适..... 从衡阳耒水河畔到湘潭雨湖再到西安秦岭，辗转3座城市（其实应该把太仓也算上），历经20年，我也要结束校园生活了。

在攻读硕士学位期间，首先要感谢我的导师李勇军，李老师站在本学科发展的前沿，以高瞻远瞩的眼光，从论文的选题、研究工作的逐步深入到论文的撰写，都给予我细致、富有建设性的意见。李老师渊博的知识、严谨的治学态度、开拓的学术思想、崇高的敬业精神、宽厚随和的为人以及睿智儒雅的学者风范给我留下了深刻的印象，使我受益匪浅。值此论文完成之际，向我的导师致以最诚挚的敬意和最衷心的感谢。

同时也要感谢418教研室的所有同学，从2020级的师兄师姐到2024级的师弟师妹，大家在一个教研室里互帮互助，共同学习和进步。读研期间的幸福感很大程度上来自于教研室和课题组的氛围，我很庆幸3年里跟418的同学们相处得很愉快。

然后还要感谢我的父母，这20多年，他们勤勤恳恳、辛苦工作，供我上学念书。现在解放的人不光是我，他们也了却了一桩事，可以安心过上退休生活了。

最后要感谢的是西北工业大学。虽然一开始它并不是我的梦中情校，我也是在阴差阳错间才来到这里，但是我现在的一切都是依靠瓜大而来，我的身上也刻下了学校的烙印。公诚勇毅、三实一新，这是我一辈子都会记住的校训校风。

最近看了李沐在上交计算机系的演讲[1]和访谈[2]视频，也读了他写的《博士这五年》[3]、《工作五年反思》[4]和《创业一年，人间三年》[5]三篇文章。在佩服对方是sys、ai等多方面大牛之余，也感慨良多。研一时，我靠他的视频教程入门深度学习；毕业之际，又从他的讲座和文章中收获启迪与反思。

COVID-19对全球经济的冲击仍没有结束，形势也在不断变化，现在的的确确是百年未有之大变局。正如大刘在《三体》里写的那样："在老张等待的这30分钟里，地球上还有约10000个婴儿出生，如果他们的哭声汇在一起，那肯定是一曲宏伟的合唱。在他们后面，黄金时代刚刚结束；在他们前面，人类的艰难岁月正在徐徐展开"。我曾在本科毕业论文的致谢里写过"感谢自己不忘初心"，如今却开始思考自己是否真的能做到。希望自己在工作后能像李沐一样，仍是个有理想、有情怀的人。

[1]https://www.bilibili.com/video/BV1dHWkewEWz
[2]https://www.bilibili.com/video/BV1zmymYHERs
[3]https://zhuanlan.zhihu.com/p/25099638
[4]https://zhuanlan.zhihu.com/p/374777591
[5]https://zhuanlan.zhihu.com/p/714533901

在学期间发表的学术成果和参加科研情况

学术成果：

[1] **Zhu Y**, Li Y, Wang J, et al. Faknow: A unified library for fake news detection[J]. Data Intelligence, 2024. https://doi.org/10.3724/2096-7004.di.2024.0026.（第一作者，对应本文第 4 章）

[2] **Zhu Y**, Li Y, Wang J, et al. Allnet: Alternative Learning is Better for Multi-modal Fake News Detection[J]. IEEE Transactions on Computational Social Systems, 2025.（第一作者，在审，对应本文第 3 章）

[3] 李勇军, **朱沂原**. 一种多模态虚假信息检测方法、系统、设备及存储介质 [P]. 中国专利: 2024.（第二申请人，第一申请人为导师，受理）

科研项目：

[1] 陕西省重点研发计划，编号 2024GX-YBXM-124，多模态融合的社交媒体虚假信息检测工具研究，2024/1-2025/12。